

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2015

Huấn luyện viên: Yu Linyun

China Computer Federation, 2015

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015

I0I China candidate team papers / National training team 2015 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2015. Tập được tạo bằng pdfTeX nhưng lớp văn bản không ánh xạ đúng về Unicode: ngay cả mục lục khi trích xuất bằng pdftotext cũng trở thành mojibake. Bản dịch này chuyển ngữ phần đầu sách, mục lục đọc được từ các trang đã render, và chín bài đầu tiên của tập sách, được đọc bằng OCR hoặc đối chiếu với trang render.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015*. Huấn luyện viên ghi trên bìa là Yu Linyun. Đơn vị xuất bản là China Computer Federation.

Ghi chú trích xuất. Do lớp văn bản của PDF nguồn không dùng được, các bài đã dịch được đọc từ ảnh render bằng pdftoppm và OCR bằng tesseract -l chi_sim+eng, sau đó đối chiếu trực quan ở các trang biên. Hiện bản dịch đã hoàn thành các bài đầu tiên: bài “Mở rộng suffix automaton trên trie” dùng các trang PDF vật lý 5–20, tương ứng trang in 1–16; bài “Bàn về ứng dụng tư tưởng heuristic trong thi tin học” dùng các trang PDF vật lý 21–40, tương ứng trang in 17–36; bài “Bàn về một số phương pháp khớp chuỗi” dùng các trang PDF vật lý 41–69, tương ứng trang in 37–65. Trang PDF vật lý 70, trang in 66, là trang trắng; trang PDF vật lý 71, trang in 67, là đầu bài kế tiếp về suffix automaton. Các bài tiếp theo dùng cùng cách xử lý OCR và đối chiếu trang render. Bài thứ sáu, “Báo cáo ra đề *ycd's bonus*”, dùng các trang PDF vật lý 107–116, tương ứng trang in 103–112; trang PDF vật lý 117, trang in 113, là đầu bài kế tiếp về chia khối. Bài thứ bảy, “Bàn về ứng dụng chia khối trong một lớp bài toán trực tuyến”, dùng các trang PDF vật lý 117–131, tương ứng trang in 113–127; trang PDF vật lý 132, trang in 128, là trang trắng; trang PDF vật lý 133, trang in 129, là đầu bài kế tiếp về cactus. Bài thứ tám, “Thuật toán liên quan tới cactus và ứng dụng”, dùng các trang PDF vật lý 133–152, tương ứng trang in 129–148; trang PDF vật lý 153, trang in 149, là đầu bài kế tiếp về matching trên đồ thị. Bài thứ chín, “Bàn về thuật toán matching trên đồ thị và ứng dụng”, dùng các trang PDF vật lý 153–178, tương ứng trang in 149–174; trang PDF vật lý 179, trang in 175, là đầu bài kế tiếp về các bài toán vật lý trong thi tin học.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Mở rộng suffix automaton trên trie	Liu Yanyi, Trường Yali, Trường Sa, Hồ Nam
17	Bàn về ứng dụng tư tưởng heuristic trong thi tin học	Ren Zhizhou, Trường Trung học số 1 Thiệu Hưng
37	Bàn về một số phương pháp khớp chuỗi	Wang Jianhao, Trường Trung học số 1 Thiệu Hưng
67	Suffix automaton và ứng dụng	Zhang Tianyang, Trường Trung học số 1 Trường Sa, Hồ Nam
81	Phép toán trên hàm sinh và bài toán đếm tổ hợp	Jin Ce, Trường Học Quân Hàng Châu, Chiết Giang
103	Báo cáo ra đề <i>yc's bonus</i>	Liu Jiancheng, Trường Yali, Trường Sa
113	Bàn về ứng dụng chia khối trong một lớp bài toán trực tuyến	Zou Daoyao, Trường Trung học Trần Hải, Ninh Ba
129	Thuật toán liên quan tới cactus và ứng dụng	Wang Yisong, Trường Học Quân Hàng Châu, Chiết Giang
149	Bàn về thuật toán matching trên đồ thị và ứng dụng	Chen Haobo, Trường Trường Quận, Trường Sa, Hồ Nam
175	Bàn về các bài toán vật lý trong thi tin học	Chen Siyu, Trường Trung học số 2 Hàng Châu
199	Báo cáo ra đề <i>Đề bài bị thất lạc</i>	Yu Jiping, Trường Trung học số 24 Đại Liên
205	Một số kỹ thuật tối ưu DP	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng
217	Báo cáo ra đề <i>Product</i>	Du Yuhao, Trường Trung học Trần Hải, Ninh Ba
231	Bước đầu tìm hiểu một lớp bài toán lấy mã nguồn làm đầu vào	Lu Xiaochen, Học viện Giao Xuyên Trần Hải, Ninh Ba
271	Tính chất, ứng dụng và thuật toán nhanh cho lũy thừa tập hợp	Lu Kaifeng, Trường Trung học số 2 Vũ Hán, Hồ Bắc

3 Mở rộng suffix automaton trên trie

Liu Yanyi

Trường Yali, Trường Sa, Hồ Nam

Tóm tắt

Bài viết phân tích chi tiết một automaton hữu hạn xác định có thể nhận ra mọi hậu tố của một trie. Tác giả đưa ra và chứng minh các kết luận rằng số trạng thái là tuyến tính, tổng số hàm chuyển không vượt quá số nút của trie nhân với kích thước bảng chữ cái, đồng thời trình bày một thuật toán xây dựng khả thi, có độ phức tạp tuyến tính khi được phép xử lý offline. Bài viết cũng phân tích độ phức tạp của thuật toán xây dựng online, giới thiệu một số ứng dụng liên quan của suffix automaton, với hy vọng gợi mở thêm

các hướng sử dụng cấu trúc này.

3.1 Lời nói đầu

Trong thi thuật toán, xử lý xâu là một mảng rất quan trọng. Nhìn chung có hai loại bài toán lớn: bài toán có mẫu cố định và bài toán có văn bản chính cố định. Khi mẫu cố định, có thể có nhiều văn bản chính khác nhau. Nếu chỉ có một mẫu, thuật toán KMP nổi tiếng [2, 3] xử lý được lớp bài toán này; cũng có thể dùng hashing xâu [13] để đếm số lần xuất hiện của mẫu. Nếu có nhiều mẫu, thuật toán AC automaton quen thuộc [1] đảm nhiệm tốt lớp bài toán đó. Nếu văn bản chính cố định, ta thường xây suffix array [11], suffix tree [4, 5] hoặc suffix automaton [7] cho văn bản chính. Lớp bài toán này đã được nghiên cứu đầy đủ trong cộng đồng thi thuật toán. Chẳng hạn, Chen Lijie đã nêu nhiều ứng dụng của suffix automaton trên một xâu trong bài giảng giao lưu trại đông [8], còn Luo Suijian giải thích chi tiết việc dùng suffix array trong thi tin học trong luận văn đội tuyển quốc gia của mình [12]. Khi đề bài cho phép xử lý offline, hai loại bài toán trên thường có thể chuyển đổi cho nhau, thể hiện tính linh hoạt của các thuật toán xử lý xâu.

Trong kỳ chọn đội tuyển tỉnh Chiết Giang năm 2015 xuất hiện bài toán sau: cho một cây có $n \leq 10^5$ nút, số lá không quá 20, mỗi cạnh mang một ký tự. Đường đi giữa hai nút trên cây tạo thành một xâu; hỏi có tất cả bao nhiêu xâu con khác nhau.¹ Bài này rõ ràng thuộc loại văn bản chính cố định, nhưng văn bản chính không còn là một xâu mà là một cây. Nếu trực tiếp liệt kê mọi cặp đỉnh rồi dùng trie [1, 13] để kiểm tra hai xâu có bằng nhau hay không, độ phức tạp là $O(n^3)$. Ngay cả dùng hashing xâu [13], thời gian vẫn là $O(n^2)$, không thể qua được dữ liệu.

Ta chuyển hóa bài toán như sau. Vì chỉ có 20 lá, với mỗi lá làm gốc, dựng một trie riêng rồi hợp nhất tất cả các trie này thành một trie mới. Khi đó mọi xâu xuất hiện trên cây ban đầu chắc chắn xuất hiện trong trie mới; văn bản chính đã trở thành một trie.

Để giải quyết vấn đề trên, bài viết phân tích automaton hữu hạn xác định nhận ra toàn bộ hậu tố của một trie và đề xuất một cách xây dựng. Nội dung chính gồm năm phần:

1. đưa một số khái niệm trên xâu sang trie, đồng thời giới thiệu suffix automaton của trie;
2. phân tích suffix link và xây dựng cấu trúc cây parent;
3. chứng minh số trạng thái là tuyến tính và nêu chặn trên cho tổng số hàm chuyển trong automaton;
4. mô tả thuật toán xây suffix automaton cho trie; khi được phép offline, độ phức tạp của nó là tuyến tính theo kích thước automaton;
5. trình bày một số ứng dụng của suffix automaton trên trie, qua đó làm rõ hơn cách nhìn về suffix automaton.

3.2 Suffix automaton của trie

Trong bài viết này, mọi ký tự của xâu và mọi ký tự trên cạnh của trie đều thuộc một bảng chữ cái hữu hạn A . Tập mọi xâu sinh bởi bảng chữ cái này là A^* ; xâu rỗng được ký hiệu là ε . Tập không chứa xâu rỗng là $A^+ = A^* \setminus \{\varepsilon\}$. Dùng $|s|$ để chỉ độ dài xâu s , s_i là ký tự thứ i , và $s_{i,j}$ là xâu gồm các ký tự từ vị trí i đến vị trí j của s . Ký hiệu $a \cdot b$ là xâu thu được khi nối a với b ; dấu nhân có thể được lược bỏ.

Tập mọi hậu tố của một xâu s là

$$S(s) = \{s_{i,|s|} \mid 1 \leq i \leq |s|\}.$$

Với một tập xâu S , $\text{Minl}(S)$ và $\text{Maxl}(S)$ lần lượt là độ dài của xâu ngắn nhất và dài nhất trong S .

¹ZJOI 2015 Day 1, *Vùng đất huyền tưởng được chut thần ưu ái*, tác giả Chen Lijie.

Dùng T để chỉ một trie, và dùng $T_x, T_y, T_z, \dots$ để chỉ các nút của T . Xây nhận được bằng cách nối theo thứ tự các ký tự trên đường đi từ nút T_x đến một nút T_y trong cây con của nó được ký hiệu là $T_{x,y}$. Nếu không nói rõ, khi dùng $T_{x,y}$, ta luôn ngầm hiểu T_y nằm trong cây con của T_x .

Một tiền tố của T là

$$P_{T_x} = T_{\text{root},x} \quad (\text{root là gốc của } T, T_x \in T).$$

Tập mọi xâu con của T là

$$F(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ nằm trong cây con của } T_x\}.$$

Tương tự, tập mọi hậu tố của T là

$$S(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ là nút lá}\}.$$

Xây suffix automaton cho trie nghĩa là xây một automaton hữu hạn có thể nhận ra mọi xâu trong $S(T)$, đồng thời cố gắng giảm số trạng thái và số chuyển xuống càng nhỏ càng tốt. Tương tự suffix automaton trên một xâu, ta định nghĩa vị trí xuất hiện của một xâu trên trie:

$$\text{Right}(s) = \{T_y \mid T_y \in T, \exists T_x \in T, T_y \text{ nằm trong cây con của } T_x, T_{x,y} = s\}.$$

Định nghĩa quan hệ tương đương R_T ; khi không gây nhầm lẫn viết là R :

$$aRb \iff \text{Right}(a) = \text{Right}(b) \quad (a, b \in A^*).$$

Gọi $R(s)$ là tập mọi xâu tương đương với s , tức một lớp tương đương. Trong automaton, mỗi trạng thái tương ứng với một lớp tương đương.

Gọi suffix automaton xây từ T là $\Phi(T)$, trạng thái đầu là $\Phi(T)_\varepsilon$. Ký hiệu $\Phi(T)_s$, với $s \in F(T)$, là trạng thái sau khi đọc s . Hàm chuyển được định nghĩa bởi chuyển nhân a từ $\Phi(T)_s$ tới $\Phi(T)_{sa}$ khi $a \in A$ và $sa \in F(T)$. Khi nối hai xâu s_1, s_2 mà $s_1, s_2, s_1s_2 \in F(T)$, ta cũng có $\Phi(T)_{s_1}$ đọc s_2 tới $\Phi(T)_{s_1s_2}$. Theo định nghĩa,

$$\Phi(T)_a = \Phi(T)_b \iff aRb \quad (a, b \in F(T)).$$

Nếu a là một xâu tương ứng với trạng thái $\Phi(T)_b$, thì tập xâu tương ứng với trạng thái $\Phi(T)_b$ chính là $R(b)$.

Hình a. Trong trie minh họa của bản gốc, b tương đương với ab ; các nút màu đỏ là tập Right của chúng.

Để phân tích số trạng thái của toàn bộ cấu trúc, tức số lớp tương đương R trong $F(T)$, ta cần xét một số tính chất của các trạng thái suffix automaton. Mục tiêu là làm số trạng thái nhỏ nhất có thể.

Bổ đề 2.1. Với $x, y, z, u \in F(T)$, ta có:

- (i) $xRy \Rightarrow (x \in S(y) \text{ hoặc } y \in S(x))$,
- (ii) $x \in S(z) \Rightarrow \text{Right}(z) \subseteq \text{Right}(x)$,
- (iii) $(x \in S(z), z \in S(u), xRu) \Rightarrow zRuRx$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$ và $Right(x) = Right(y)$. Lấy tùy ý $T_v \in Right(x)$. Khi đó $x \in S(P_{T_v})$ và $y \in S(P_{T_v})$, nên $x \in S(y)$.

(ii) Với mọi $T_v \in Right(z)$, ta có $z \in S(P_{T_v})$. Vì $x \in S(z)$, suy ra $x \in S(P_{T_v})$, do đó $T_v \in Right(x)$. Vậy $Right(z) \subseteq Right(x)$.

(iii) Từ (ii), $z \in S(u)$ cho $Right(u) \subseteq Right(z)$, và $x \in S(z)$ cho $Right(z) \subseteq Right(x)$. Lại có xRu , nên $Right(z) = Right(u) = Right(x)$, tức $zRuRx$.

Mệnh đề (i) cũng cho thấy trong cùng một lớp tương đương R , không thể có hai xâu khác nhau nhưng cùng độ dài.

Hệ quả 2.2. Với một lớp tương đương $R(a)$, gọi x là xâu dài nhất trong $R(a)$, và y là xâu ngắn nhất trong $R(a)$. Khi đó

$$R(a) = \{z \mid z \in S(x) \text{ và } |y| \leq |z| \leq |x|\}.$$

Điều này suy ra trực tiếp từ Bổ đề 2.1.

3.3 Suffix link

Trong thuật toán xây suffix automaton cho một xâu, suffix link giữ vai trò then chốt [7, 6], và toàn bộ thuật toán rất giống KMP [3]. Khi xây suffix automaton cho trie, ta cũng giữ suffix link và so sánh quá trình xây dựng với thuật toán xây automaton tối thiểu xác định nhận ra tập $A^*\{P_{T_x}\}$, tức cách xây AC automaton quen thuộc [1]. Tương tự hàm fail trong AC automaton, ta định nghĩa suffix link của suffix automaton trên trie như sau.

Suffix link là một hàm $s_T : A^* \setminus \{\varepsilon\} \rightarrow A^*$, khi không nhầm lẫn viết là s :

$$s_T(x) = \{z \mid z \in S(x), |z| = \text{Minl}(R_T(x)) - 1\}.$$

Nói cách khác, đó là hậu tố dài nhất y của x sao cho xR_Ty không đúng.

Bổ đề 3.1. Với $x, y \in F(T)$, ta có:

- (i) $xRy \Rightarrow s(x) = s(y)$,
- (ii) $Right(x) \subset Right(s(x))$,
- (iii) $\text{Minl}(R(x)) - 1 = |s(x)| = \text{Maxl}(R(s(x)))$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$. Theo định nghĩa, $|s(x)| = \text{Minl}(R(x)) - 1$, $|s(y)| = \text{Minl}(R(y)) - 1$, mà $R(x) = R(y)$, nên $|s(x)| = |s(y)| < |x| \leq |y|$. Theo Bổ đề 2.1(i), $x \in S(y)$. Lại có $s(x) \in S(x)$ và $s(y) \in S(y)$, do đó $s(x) = s(y)$.

(ii) Theo Bổ đề 2.1(ii), $Right(x) \subseteq Right(s(x))$. Vì $xRs(x)$ là sai, hai tập này không bằng nhau, nên bao hàm là nghiêm ngặt.

(iii) Vì $s(x) \in R(s(x))$, ta có $|s(x)| \leq \text{Maxl}(R(s(x)))$. Giả sử tồn tại xâu y tương đương với $s(x)$ và dài nhất trong lớp đó. Khi ấy $Right(y) = Right(s(x))$, đồng thời $s(x) \in S(y)$. Dùng phản chứng: nếu $|y| > |s(x)|$, lấy $T_v \in Right(x)$. Từ (ii), $T_v \in Right(s(x))$, suy ra $T_v \in Right(y)$, nên $y \in S(P_{T_v})$ và $x \in S(P_{T_v})$. Nếu $|y| > |x|$ thì $x \in S(y)$, theo Bổ đề 2.1(ii) sẽ có $Right(s(x)) = Right(y) \subseteq Right(x)$, mâu thuẫn với (ii). Nếu $|y| \leq |x|$, vì xRy không đúng nên $s(x) = y$, mâu thuẫn với $|y| > |s(x)|$. Vậy mệnh đề được chứng minh.

Từ (i), mọi xâu trong cùng một lớp tương đương $R(a)$ có suffix link trở tới cùng một xâu. Vì vậy ta có thể gán suffix link cho từng trạng thái của suffix automaton. Cụ thể, suffix link của $R(a)$ trở tới $R(s(a))$.

Theo (ii) hoặc (iii), các suffix link này tạo thành một cây; ta gọi nó là **cây parent**. Định nghĩa $Fa(\Phi(T)_s)$ là suffix link của trạng thái $\Phi(T)_s$, tức

$$Fa(\Phi(T)_a) = \Phi(T)_b \iff s(a)Rb.$$

Bổ đề 3.2. Với $x \in F(T)$, lấy tùy ý $T_v \in \text{Right}(x)$. Nếu $|P_{T_v}| > \text{Maxl}(R(x))$ và đặt $a = P_{T_v}$, thì

$$s(a_{|a|-\text{Maxl}(R(x)), |a|})Rx.$$

Chứng minh. Đặt $b = a_{|a|-\text{Maxl}(R(x)), |a|}$. Từ giả thiết có $x \in S(b)$. Theo Hệ quả 2.2, vì $|b| > \text{Maxl}(R(x))$, nên bRx không đúng, còn hậu tố độ dài $\text{Maxl}(R(x))$ của b lại tương đương với x . Vì vậy b không tương đương với hậu tố đó; theo định nghĩa suffix link, $s(b)$ chính là hậu tố này, và $s(b)Rx$.

Hình b. Cây trong bản gốc là cây parent của suffix automaton được xây từ trie ở Hình a.

3.4 Số trạng thái tuyến tính

Với suffix automaton xây cho một xâu x , kích thước automaton là $O(|x|)$ [7, 6]. Ta cũng mong suffix automaton xây cho trie có quy mô trạng thái tuyến tính. Sau khi định nghĩa suffix link, cần phân tích kỹ hơn $\Phi(T)$.

Bổ đề 4.1. Với $x, y \in F(T)$, ta có:

- (i) $\forall y, \neg(xRs(y)) \Rightarrow |\text{Right}(x)| = 1$,
- (ii) $|\text{Right}(x)| = 1 \Rightarrow P_{T_v}Rx \quad (\text{Right}(x) = \{T_v\})$,
- (iii) $\forall T_v \in \text{Right}(x), \neg(xRP_{T_v})$
 $\Rightarrow \exists u, v, s(u)Rs(v)Rx \text{ và } \neg(uRv).$

Chứng minh. (i) Chứng minh bằng phản chứng. Giả sử $|\text{Right}(x)| > 1$. Lấy $T_{v_1}, T_{v_2} \in \text{Right}(x)$ và đặt $a = P_{T_{v_1}}, b = P_{T_{v_2}}$. Khi đó $|a|, |b| \geq \text{Maxl}(R(x))$; nếu một trong hai độ dài nhỏ hơn $\text{Maxl}(R(x))$, xâu trong $R(x)$ có độ dài $\text{Maxl}(R(x))$ không thể xuất hiện ở nút tương ứng, mâu thuẫn. Nếu $a \notin R(x)$, theo Bổ đề 3.2 suy ra suffix link của một hậu tố thích hợp của a trở vào lớp của x . Nếu $a \in R(x)$, vì $T_{v_1} \neq T_{v_2}$, ta có $a \neq b$, và $|b| \geq \text{Maxl}(R(x))$, nên một lập luận tương tự cũng cho một xâu có suffix link trở vào $R(x)$. Mệnh đề được chứng minh.

(ii) Biết $x \in S(P_{T_v})$, do đó $|\text{Right}(P_{T_v})| \leq |\text{Right}(x)|$. Vì $|\text{Right}(x)| = 1$, suy ra $\text{Right}(P_{T_v}) = \{T_v\} = \text{Right}(x)$, tức $P_{T_v}Rx$.

(iii) Từ phản đảo của (i), nếu $|\text{Right}(x)| \neq 1$ thì $|\text{Right}(x)| \geq 2$. Theo giả thiết, với mọi $T_v \in \text{Right}(x)$ ta có $|P_{T_v}| > \text{Maxl}(R(x))$. Tồn tại T_{v_1}, T_{v_2} sao cho hậu tố chung dài nhất của $P_{T_{v_1}}$ và $P_{T_{v_2}}$ có độ dài đúng bằng $\text{Maxl}(R(x))$. Nếu không, mọi cặp đều có hậu tố chung dài hơn $\text{Maxl}(R(x))$; theo Bổ đề 2.1(iii), hậu tố chung dài hơn đó cũng tương đương với x , mâu thuẫn với định nghĩa $\text{Maxl}(R(x))$.

Đặt

$$a = (P_{T_{v_1}})_{|P_{T_{v_1}}|-\text{Maxl}(R(x)), |P_{T_{v_1}}|}, \quad b = (P_{T_{v_2}})_{|P_{T_{v_2}}|-\text{Maxl}(R(x)), |P_{T_{v_2}}|}.$$

Từ cách chọn, $a \neq b$, $a \notin S(b)$, $b \notin S(a)$. Theo Bổ đề 2.1(i), $\neg(aRb)$. Dùng Bổ đề 3.2 ta có $s(a)Rx$ và $s(b)Rx$. Lấy $u = a$, $v = b$ là được.

Định lý 4.2. Suffix automaton xây cho một trie có số trạng thái tuyến tính.

Chứng minh. Xét cây parent tạo bởi suffix link. Từ Bổ đề 4.1(i) và (ii), với mọi lá $\Phi(T)_s$ trên cây parent đều tồn tại một tiền tố P_{T_v} sao cho $P_{T_v}Rs$. Từ Bổ đề 4.1(iii), mọi trạng thái $\Phi(T)_s$ mà với mọi T_v đều không có sRP_{T_v} thì phải có ít nhất hai con trong cây parent. Nói cách khác, các lá và các nút chỉ có một con trên cây parent đều tương ứng với một tiền tố nào đó P_{T_v} ; các nút còn lại có ít nhất hai con.

Số tiền tố chỉ là $|T|$, và trạng thái đạt được khi đọc từng P_{T_v} là duy nhất. Vì vậy tổng số lá và nút một con trên cây parent không vượt quá $|T|$; toàn bộ số nút của cây không vượt quá $2|T|$. Do đó số trạng thái là tuyến tính.

Nếu không chỉ số trạng thái mà tổng số hàm chuyển, tức số cạnh, cũng tuyến tính, thì toàn bộ automaton sẽ tuyến tính. Tuy nhiên, khác với suffix automaton của một xâu, tổng số hàm chuyển của suffix automaton trên trie có thể ở cấp số trạng thái nhân với kích thước bảng chữ cái. Ta chứng minh nó không vượt quá cấp này, và đưa ra một trie đạt cùng cấp độ.

Định lý 4.3. Với một trie, chặn trên của số hàm chuyển trong suffix automaton là

$$O((\text{số nút của trie}) \cdot (\text{kích thước bảng chữ cái})).$$

Chứng minh. Gọi trie là T . Theo Định lý 4.2, $|\Phi(T)| \leq 2|T|$. Với mỗi trạng thái $\Phi(T)_s$, số chuyển nhiều nhất là $|A|$. Vì vậy tổng số hàm chuyển không vượt quá $2|T||A|$.

Ta cũng dựng được một trie khiến số hàm chuyển đạt chặn này theo cấp độ. Chỉ cần xét những ký tự thực sự xuất hiện trên cạnh của T , nên $|A| \leq |T|$. Một phần của T là một dây dài $|T|/2$, mọi cạnh trên dây đều mang cùng ký tự a . Các nút còn lại nối từ nút cuối của dây bằng các ký tự đôi một khác nhau; gọi tập ký tự này là B . Khi đó $a^ib \in S(T)$ với $0 \leq i \leq |T|/2$ và $b \in B$, đồng thời hiển nhiên $\neg(a^iRa^j)$ nếu $i \neq j$. Vì thế với mỗi trạng thái $\Phi(T)_{a^i}$, mọi $b \in B$ đều tạo một hàm chuyển. Tổng số hàm chuyển đạt $|T|/2 \cdot |B|$, tức cùng cấp với $|T|/2 \cdot |A|/2$, trùng cấp với chặn trên trong Định lý 4.3.

Hình c. Ví dụ trie trong bản gốc gồm một dây toàn ký tự a , rồi tỏa ra nhiều cạnh ký tự khác nhau, làm tổng số hàm chuyển đạt chặn trên theo cấp độ.

Dù số chuyển khi xây suffix automaton cho trie không luôn tuyến tính, trong phần lớn đề thi và bài toán thực tế, bảng chữ cái không lớn; thường chỉ là chữ cái Latin thường hoặc chữ số. Khi bảng chữ cái là hằng số, số chuyển vẫn tuyến tính, nên đây vẫn là một phương pháp hiệu quả để xử lý các bài toán xâu liên quan đến trie.

3.5 Thuật toán xây dựng

Trong thuật toán xây suffix automaton cho xâu [7, 8], ta dùng một thuật toán tăng dần online: từ suffix automaton của s , xây suffix automaton của sa , với $a \in A$. Khi xây cho trie, ta cũng xét thuật toán tăng dần online. Từ phân tích số trạng thái và số chuyển ở trên, ta biết cận dưới độ phức tạp của thuật toán xây dựng là $|T||A|$.

Giả sử từ trie T , ta thêm vào nút T_v một cạnh nhãn a đi tới nút con mới T_w , thu được trie mới T_N . Đặt $s = P_{T_v}$, nên tiền tố mới là $sa = P_{T_w}$. Xét những phần của $\Phi(T_N)$ cần sửa trên nền $\Phi(T)$. Trước hết phân tích số trạng thái.

3.5.1 Thay đổi trạng thái

Với mọi xâu đã có thể tiếp tục mở rộng thành hậu tố của trie mới, trạng thái tương ứng trong $\Phi(T_N)$ phải tồn tại. Ngược lại, nếu một xâu không thể mở rộng thành hậu tố mới, trạng thái đó không cần xuất hiện thêm. Nếu một lớp tương đương không đổi sau khi thêm T_w , trạng thái suffix automaton tương ứng cũng không đổi, tức $\Phi(T_N)_x = \Phi(T)_x$.

Trường hợp 1. Nếu T_v đã có một con nối bằng cạnh nhãn a , ta không cần sửa $\Phi(T)$.

Trường hợp 2. Nếu $sa \in F(T)$, nghĩa là trong $\Phi(T)$ đã có chuyển nhãn a từ trạng thái của s , thì lớp tương đương $R_T(sa)$ có thể bị tách làm hai lớp.

Thật vậy, với $x, y \in F(T)$ và $xR_T y$, nếu trong tập $Right_{T_N}$ của x và y đều xuất hiện nút mới T_w , thì khi bỏ T_w ra, hai tập vẫn bằng nhau, nên chắc chắn $xR_{T_N} y$. Vì vậy mọi xâu tương đương với sa trong T_N vốn đều nằm trong cùng lớp $R_T(sa)$. Với $x \in R_T(sa)$, nếu $xR_{T_N} sa$, không có trạng thái mới. Nếu tồn tại $x \in R_T(sa)$ nhưng $\neg(xR_{T_N} sa)$, cần tách $R_T(sa)$ thành hai lớp: một lớp tương đương với sa trong T_N , một lớp không tương đương với nó. Lớp không tương đương có tập $Right$ không chứa T_w , nên tập $Right$ của nó giữ nguyên như trong T .

Xét suffix link của hai trạng thái mới. Gọi $u, v \in F(T)$, trong đó $uR_{T_N} sa$, $\neg(vR_{T_N} sa)$, nhưng $uR_T sa$ và $vR_T sa$. Nếu đặt u vào lớp thứ nhất và v vào lớp thứ hai, vì $sa = P_{T_w}$ và $uR_{T_N} sa$, mọi hậu tố của u nhận thêm cùng vị trí mới nên suffix link của lớp thứ nhất không đổi. Với v , phần mới bị tách ra đúng tại lớp của sa , do đó suffix link của lớp thứ hai trở tới lớp mới chứa sa . Nói ngắn gọn, nếu một trạng thái bị tách, một bản sao giữ suffix link cũ, bản sao còn lại có suffix link trở tới trạng thái của sa .

Hình d. Suffix automaton của trie trong Hình a.

Hình e, f, g. Sau khi thêm một nút vào trie của Hình a, bản gốc minh họa lần lượt trie mới, suffix automaton mới và cây parent mới. Nút và cạnh màu cam biểu thị phần mới trong automaton/cây parent.

Khi đã biết lớp tương đương chứa sa trong T_N , mọi hậu tố của sa trước đó đã được đưa vào các lớp liên quan, và các trạng thái ấy không đổi, nên không cần xét lại.

Trường hợp 3. Nếu $sa \notin F(T)$, ta tạo một lớp mới $R_{T_N}(sa)$. Vì xuất hiện một hậu tố mới, vẫn có khả năng một lớp tương đương cũ bị tách.

Có một kết luận khá rõ ràng: $s_{T_N}(sa)$ bằng hậu tố dài nhất của sa có xuất hiện trong $F(T)$. Chứng minh đơn giản: nếu $x \in S(sa)$ nhưng $x \notin F(T)$, thì x chỉ xuất hiện tại nút mới T_w , và sa cũng chỉ xuất hiện tại T_w ; do đó $Right(sa) = Right(x) = \{T_w\}$, nên $xRsa$. Ngược lại, nếu $x \in S(sa)$ và $x \in F(T)$, thì $|Right(x)| > 1$, nên $\neg(xRsa)$.

Kết luận này giúp quá trình xây dựng thuận tiện hơn. Các hậu tố

$$y \in S(sa), \quad y \notin S(s_{T_N}(sa))$$

tạo thành lớp mới $R_{T_N}(sa)$, và ta tạo một trạng thái mới cho lớp đó. Với những $y \in S(s_{T_N}(sa))$, tức phần hậu tố còn lại, có thể xem $s_{T_N}(sa)$ như sa trong Trường hợp 2. Dù Trường hợp 2 dùng tính chất $sa = P_{T_w}$, bản thân $s_{T_N}(sa)$ đã có tính cực đại cần thiết, nên không gây sai lệch.

Bài toán còn lại là tìm $s_{T_N}(sa)$. Theo kết luận trên, ta cần hậu tố dài nhất của sa đã xuất hiện ít nhất một lần trong $F(T)$. Bắt đầu từ trạng thái $\Phi(T)_s$, đi lên cây parent qua các suffix link cho đến khi gặp trạng thái đầu tiên có chuyển nhãn a . Nếu trạng thái đó đại diện bởi một xâu r , thì ra là hậu tố dài nhất của sa xuất hiện ít nhất hai lần, tức $s_{T_N}(sa) = ra$.

Từ ba trường hợp trên, khi thêm một ký tự vào trie, số trạng thái tăng nhiều nhất hai. Điều này một lần nữa chứng minh số trạng thái tuyến tính, đồng thời mô tả được thay đổi của suffix link. Ta vẫn chưa xét cách sửa hàm chuyển; phần tiếp theo xử lý điểm này.

3.5.2 Thay đổi hàm chuyển

Trong Trường hợp 2, nếu không tạo trạng thái mới, chắc chắn không có thay đổi hàm chuyển. Nếu có, lớp cũ $R_T(sa)$ bị tách thành hai lớp $R_{T_N}(sa)$ và một lớp còn lại. Vì nút mới T_w là lá, nó không làm thay đổi các chuyển vốn có từ các xâu cũ. Các chuyển của hai trạng thái mới có thể sao chép từ trạng thái cũ tương ứng.

Tuy nhiên, vì lớp $R_T(sa)$ cũ bị xóa, có thể tồn tại các trạng thái có chuyển nhãn a trở tới trạng thái cũ đó; những chuyển này phải được sửa. Theo phân tích ở Trường hợp 2, các chuyển dọc theo chuỗi suffix link từ trạng thái của s lên trên, chừng nào chúng còn đi tới lớp cũ, sẽ được đổi để trở tới lớp mới phù hợp. Những chuyển khác giữ nguyên. Cách làm trực tiếp là giữ một bản sao đại diện cho phần không đổi, tạo một trạng thái mới cho phần bị tách, rồi bạo lực sửa các chuyển nhãn a trên chuỗi suffix link cho tới khi gặp trạng thái mà chuyển nhãn a không còn trở tới lớp cũ.

Trong Trường hợp 3, ta tạo trạng thái mới $R_{T_N}(sa)$; phần còn lại giống Trường hợp 2. Vì T_w là lá, trạng thái mới ban đầu không có chuyển ra. Do có trạng thái mới, một số trạng thái trên chuỗi suffix link có thể cần thêm chuyển nhãn a trở tới trạng thái mới của sa . Theo phân tích Trường hợp 3, chỉ cần thêm các chuyển này dọc theo chuỗi từ trạng thái của s đi lên cho tới khi gặp trạng thái đã có chuyển nhãn a thích hợp.

3.5.3 Phân tích độ phức tạp

Tương tự trường hợp một xâu, toàn bộ thuật toán là thuật toán tăng dần online. Với trie T , số lần thực hiện là $|T|$. Mỗi lần chỉ thêm nhiều nhất hai trạng thái, nên phần trạng thái là $O(|T|)$. Suffix link của mỗi trạng thái mới chỉ sửa một lần, phần này cũng là $O(|T|)$.

Xét hàm chuyển. Trong Trường hợp 2 có thể cần sao chép một tập chuyển; mỗi trạng thái có nhiều nhất $|A|$ chuyển, nên phần này là $O(|T| |A|)$. Phần phức tạp hơn là các lần bạo lực sửa chuyển khác. Có thể chứng minh tổng thời gian cho phần này không vượt quá tổng độ sâu các lá trong trie; gọi tổng đó là $G(T)$.

Ý tưởng chứng minh giống phân tích thế năng. Gọi $Dep(y)$, với $y \in F(T)$, là độ sâu của trạng thái $\Phi(T)_y$ trong cây parent. Xét đường từ trạng thái $\Phi(T)_s$ và trạng thái mới tới gốc trên cây parent. Theo Bổ đề 2.1(iii), nếu tồn tại lớp của sa thì cũng tồn tại lớp của s , nên $Dep(sa) \leq Dep(s) + 1$. Khi bạo lực sửa chuyển, ta liên tục đặt $c = s(c)$ và sửa chuyển của c ; mỗi bước như vậy làm $Dep(c)$ giảm 1. Khi dừng, độ sâu của trạng thái liên quan chỉ có thể tăng thêm một hằng số so với $Dep(s)$. Vì mỗi lần sửa đều làm giảm thế năng, tổng số lần sửa là cấp độ độ sâu.

Có thể xem Dep là hàm thế năng của trạng thái hiện tại. Với mỗi lá của T , nếu thực hiện riêng quá trình chèn theo đường từ gốc tới lá đó, mỗi phép chèn làm Dep tăng nhiều nhất 3, còn mỗi lần bạo lực sửa chuyển làm Dep giảm 1. Do đó số lần bạo lực sửa chuyển thực chất bị chặn bởi độ sâu; cộng trên mọi lá sẽ được kết luận trên.

Độ phức tạp này chưa phải đẹp, nhưng độ phức tạp của xây dựng online có thể đạt đến cấp đó.

Hình h. Nếu chèn các nút theo thứ tự chỉ số nút trong ví dụ của bản gốc, độ phức tạp đạt $O(G(T))$.

Khi không yêu cầu online, ta có thể thêm các nút trên trie theo thứ tự FIFO, tức thứ tự duyệt rộng. Khi đó chứng minh độ phức tạp của J. A. Blumer và cộng sự [9, 10] cho thuật toán xây trên xâu cũng áp

dụng cho trie. Chỉ những tổ hợp xâu con cố định mới khiến ta phải bạo lực sửa hàm chuyển [6], gợi ý rằng phần độ phức tạp này không vượt quá $O(|T|)$.

Tóm lại, nếu yêu cầu online, độ phức tạp xây dựng là $O(G(T))$. Nếu cho phép offline, ta có thể xây theo thứ tự FIFO và đạt độ phức tạp $O(|T| |A|)$, đúng với cận dưới đã nêu.

3.6 Ứng dụng

3.6.1 ZJOI 2015: Vùng đất huyền tưởng được chú thần ưu ái

Đề bài đã xuất hiện ở phần lời nói đầu, nên không nhắc lại nữa.

Sau chuyển hóa, bài toán trở thành: đếm số xâu con khác nhau về bản chất của một trie. Bằng phương pháp xây suffix automaton cho trie ở trên, ta thu được một automaton có thể nhận ra mọi hậu tố của trie. Mỗi xâu con của trie chắc chắn là tiền tố của một hậu tố nào đó, giống hệt tính chất khi xây suffix automaton cho một xâu. Vì vậy, với một trạng thái $\Phi(T)_s$, số xâu con khác nhau mà nó đại diện là $|R(s)|$; cộng $|R(s)|$ trên mọi trạng thái sẽ cho đáp án.

Cũng có cách khác. Sau khi xây automaton, trực tiếp đếm có bao nhiêu xâu khác nhau không bị automaton từ chối; điều này tương đương đếm số xâu con khác nhau, hay đếm số đường chuyển khác nhau tồn tại khi bắt đầu từ trạng thái đầu. Xem automaton như một đồ thị có hướng không chu trình, ta chỉ cần quy hoạch động đơn giản trên đồ thị này.

Theo phân tích độ phức tạp ở trên, đề bài không yêu cầu online, nên có thể dùng thuật toán offline với độ phức tạp $O(n|A|)$. Thực ra điều kiện số lá là hằng số rất mạnh; có thể chứng minh automaton xây bằng phương pháp trên có thời gian $O(n)$. Lý do là ta có thể tách riêng đường từ gốc đến từng lá thành các xâu, rồi dùng phân tích độ phức tạp giống trường hợp xâu [9]. Vì điều này không liên quan trực tiếp đến nội dung chính của bài viết, tác giả bỏ qua chứng minh chi tiết.

Đến đây bài toán trong phần lời nói đầu đã được giải quyết trọn vẹn.

Từ bài toán này có thể thấy hầu hết bài toán xâu xử lý được bằng suffix automaton đều có thể mở rộng lên trie: tìm xâu con thứ K nhỏ nhất của trie, tìm xâu con chung dài nhất của nhiều trie, đếm số lần xuất hiện của xâu vòng trên trie, v.v. Cũng có những bài rất dễ trên xâu nhưng phức tạp hơn khi mở rộng lên trie, chẳng hạn đếm số xâu khác nhau mà tiền tố của chúng là một xâu cho trước. Các bài này đều có thể giải sau khi xây suffix automaton cho trie.

3.6.2 So sánh với AC automaton

Trong các thuật toán xâu quen thuộc, AC automaton xây một trie từ m mẫu rồi xây các con trỏ fail, sau đó có thể nhận diện số lần xuất hiện của từng mẫu trong xâu đầu vào. Ta có thể thay thế cách dùng này bằng suffix automaton trên trie: xây trie từ m xâu, rồi xây suffix automaton cho trie đó. Khi đọc xâu cần nhận diện, nếu trạng thái hiện tại $\Phi(T)_s$ không có chuyển nhãn a , ta nhảy tới $fa(\Phi(T)_s)$ và tiếp tục nhận diện. Rõ ràng tổng số lần nhảy không vượt quá độ dài xâu đang nhận diện.

Về độ phức tạp, trie T xây từ m xâu có $G(T)$ không vượt quá tổng độ dài của m xâu, nên độ phức tạp không kém hơn AC automaton. Nhưng thuật toán xây ở đây có thể là online, trong khi thuật toán xây AC automaton chỉ là offline.

3.6.3 Cây parent

Hình i. Bản gốc minh họa việc bổ sung các ký tự trên cạnh của Hình b để nhìn cây parent như một trie của các tiền tố đảo.

Thực chất, ta có thể đảo toàn bộ tiền tố trong trie rồi xây lại thành một trie, tương tự suffix tree của một xâu. Với mỗi trạng thái $\Phi(T)_s$, sắp xếp các con của nó trong cây parent. Cụ thể, nếu

$$Fa(\Phi(T)_x) = \Phi(T)_s,$$

ta sắp xếp các trạng thái $\Phi(T)_x$ theo thứ tự của đoạn

$$x_{1, |x| - \text{Maxl}(R(s))}.$$

Khi đó ta thu được một mảng tiền tố của trie, tức thứ tự sau khi đảo mọi tiền tố. Ta cần duy trì cho $\Phi(T)_s$ một nút $T_v \in \text{Right}(s)$; kết hợp nhân đôi trên cây, có thể nhanh chóng tìm được đoạn $x_{1, |x| - \text{Maxl}(R(s))}$.

Với tính chất này, có thể nhanh chóng xác định trạng thái tương ứng khi thêm một ký tự $a \in A$ vào trước xâu đang nhận diện. Giả sử hiện ở trạng thái $\Phi(T)_s$. Ta tìm nhị phân trong các con của nó trên cây parent để tìm $\Phi(T)_{as}$, hoặc kết luận $\Phi(T)_{as} = \Phi(T)_s$.

3.7 Lời cảm ơn

Xin cảm ơn thầy Qu Yunhua của Trường Yali đã lâu dài hướng dẫn và quan tâm tới việc học tập cũng như cuộc sống của tôi.

Xin cảm ơn cha mẹ đã nuôi dưỡng tôi.

Xin cảm ơn các thầy Zhu Quanmin và Wang Xingming của Trường Yali đã chỉ dạy.

Xin cảm ơn CCF đã cung cấp nền tảng và cơ hội giao lưu.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự tận tụy.

Xin cảm ơn các anh Huang Zhixiang của Đại học Thanh Hoa và Huang Yuyang của Đại học Giao thông Thượng Hải đã giúp đỡ tôi.

Xin cảm ơn các bạn Yang Dingcheng, Kuang Zhengfei và Liu Jiancheng của Trường Yali đã đọc duyệt bài viết này.

Xin cảm ơn tất cả những bạn học từng giúp đỡ tôi.

Tài liệu tham khảo của bài viết

- [1] A. V. Aho and M. J. Corasick, Efficient string matching: An aid to bibliographic research.
- [2] R.-S. Boyer and J. S. Moore, A fast string searching algorithm.
- [3] D. E. Knuth, J. H. Morris and V. I. Pratt, Fast pattern-matching in strings.
- [4] P. Weiner, Linear pattern-matching algorithms.
- [5] E. M. McCreight, A space-economical suffix-tree construction algorithm.
- [6] M. Mohri, P. Moreno and E. Weinstein, General Suffix Automaton Construction Algorithm and Space Bounds.
- [7] M. Crochemore, Transducers and repetitions.
- [8] Chen Lijie, *Suffix automaton*.
- [9] J. A. Blumer, Algorithms for the directed acyclic word graph and related structures.
- [10] A. Blumer, J. Blumer, D. Haussler, R. M. McConnell and A. Ehrenfeucht, Complete inverted files for efficient text retrieval and analysis.
- [11] U. Manber and G. Myers, Suffix arrays: A new method for on-line string searches.

[12] Luo Suijian, *Suffix array: một công cụ mạnh để xử lý xâu*.

[13] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*.

4 Bàn về ứng dụng tư tưởng heuristic trong thi tin học

Ren Zhizhou

Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Thuật toán heuristic là các thuật toán được thiết kế dựa trên kinh nghiệm. So với các thuật toán tối ưu hóa truyền thống, heuristic thường đánh đổi độ chính xác hoặc tính ổn định để lấy tốc độ. Bài viết thử đưa ra một định nghĩa cụ thể hơn cho heuristic trong thi tin học, đồng thời thảo luận một số thuật toán heuristic thường dùng và một số mô hình bài toán. Trong đó có một thuật toán xấp xỉ cho cây Steiner nhỏ nhất với độ phức tạp $O(|S||V|^2)$; bài viết cũng chứng minh tỉ lệ xấp xỉ của thuật toán này đạt

$$2 \left(1 - \frac{1}{\text{leaf}} \right) \leq 2 \left(1 - \frac{1}{|S|} \right),$$

trong đó leaf là số lá của cây Steiner tối ưu.

4.1 Dẫn nhập

Thuật toán heuristic là một lớp thuật toán được nêu ra trong tương quan với các thuật toán tối ưu hóa. Chúng được xây dựng từ trực giác hoặc kinh nghiệm, với mục tiêu tìm được một nghiệm khá tốt trong chi phí thời gian và bộ nhớ chấp nhận được. Khái niệm này rất rộng; đa số heuristic khá phức tạp và không thể đưa thẳng vào bài thi OI.

Tác giả giản lược và cải tiến một số thuật toán heuristic để chúng phù hợp hơn với OI, rồi định nghĩa thêm hai kiểu heuristic trong bối cảnh thi tin học:

- dựa vào trực giác để điều chỉnh và tối ưu thuật toán, hy sinh tính ổn định để đổi lấy tốc độ;
- dựa vào trực giác để trực tiếp dựng nghiệm tốt, hy sinh tính đúng tuyệt đối để giải nhanh.

Nhiều thuật toán quen thuộc cũng chứa tư tưởng heuristic. Sau tối ưu, chúng không làm giảm độ phức tạp xấu nhất, nhưng hiệu suất thực tế thường tăng rõ rệt, chẳng hạn:

- truy vấn điểm gần nhất trên k - d tree;
- Bellman–Ford tối ưu bằng hàng đợi, tức SPFA, và các tối ưu tham lam liên quan;
- mô phỏng tôi luyện, thuật toán di truyền và các thuật toán tìm kiếm ngẫu nhiên lấy cảm hứng từ khoa học tự nhiên.

Bài viết thảo luận ba mô hình heuristic:

- thuật toán A^* và các biến thể, dùng để nối giữa thuật toán xấp xỉ và lời giải tối ưu;
- phân loại Bayes ngây thơ, một thuật toán xác suất kết hợp phán đoán trực quan với suy luận toán học;
- gộp heuristic, một tối ưu đơn giản cho các bài toán cần hợp nhất dữ liệu.

Ngoài việc áp dụng mô hình kinh điển, nhiều khi ta cũng có thể tự thiết kế heuristic. Bài viết dành nhiều dung lượng cho hai bài toán cổ điển:

- bài toán chia đều có trọng số, hay *2-partition problem*, từ đó có một thuật toán xấp xỉ có xác suất đúng rất cao và quy trình thiết kế đáng tham khảo;
- bài toán cây Steiner, được trình bày kỹ hơn, với một thuật toán hiệu quả có thể đánh giá sai số xấp xỉ, rồi kết hợp với A^* để thu được thuật toán xấp xỉ cho K cây Steiner nhỏ nhất.

Ở mỗi phần, tác giả đều đưa ví dụ với hy vọng giúp người đọc nắm được cách sử dụng và tiếp tục phát triển các ý tưởng này.

4.2 Thuật toán A^*

Trong một số bài toán rất khó tìm thuật toán đa thức hiệu quả, ta buộc phải dùng tìm kiếm, trong khi số trạng thái lại rất lớn. Khi đó có thể thiết kế hàm ước lượng để cắt tỉa; phần này trình bày một số cách dùng ước lượng để tăng tốc tìm kiếm.

4.2.1 Tìm kiếm heuristic

Giả sử trạng thái hiện tại là s , chi phí thực tế đã đi tới s là $g(s)$, ước lượng chi phí từ s tới trạng thái đích là $h(s)$, còn chi phí tối ưu thật sự từ s tới đích là $h^*(s)$. Đặt

$$F(s) = g(s) + h(s).$$

Ta có thể dựa vào giá trị này, mỗi lần mở rộng trạng thái có $F(s)$ tốt nhất, và cắt tỉa theo tính chất của hàm ước lượng. Trong bài viết, các bài toán được mặc định là tối thiểu hóa chi phí.

4.2.2 $h(s) = h^*(s)$

Khi hàm ước lượng cho đúng chi phí còn lại, mỗi lần mở rộng trạng thái tốt nhất theo $F(s)$ sẽ nhanh chóng thu được nghiệm tối ưu.

Sau khi đã có nghiệm tối ưu, nếu tiếp tục mở rộng các trạng thái còn lại thì có thể nhận được nghiệm tốt thứ K , đồng thời tránh tính các trạng thái thừa. Phần tổng quát và các tối ưu cho bài toán đặc biệt đã được Yu Yili trình bày trong luận văn đội tuyển quốc gia năm 2014.

4.2.3 $h(s) \leq h^*(s)$

Đôi khi, để tính ước lượng thuận tiện, ta bỏ qua một số ràng buộc trong đề bài. Kết quả thu được là một nghiệm thô “tốt hơn” nghiệm tối ưu thật, tức ước lượng không vượt quá chi phí còn lại. Khi đó ước lượng vẫn cung cấp căn cứ cắt tỉa đúng, giúp giảm phạm vi trạng thái phải xét.

IDA*. IDA*, hay tìm kiếm sâu dần lặp có heuristic, là một biến thể của A^* kết hợp ý tưởng lặp sâu dần với hàm ước lượng. Trong các bài có số trạng thái lớn và khó lưu trữ, cách này tiết kiệm rất nhiều bộ nhớ, đồng thời mã cài đặt cũng gọn.

Với lặp sâu dần đơn giản, mỗi lượt thường chọn một bước tăng cố định và dùng giới hạn độ sâu để cắt tỉa. Với IDA*, ta cắt tỉa bằng ước lượng. Vì $h(s) \leq h^*(s)$, giá trị nhỏ nhất của $g(s) + h(s)$ trong các trạng thái bị cắt có thể dùng để ước lượng đáp án kế tiếp, giảm số lần lặp so với bước tăng cố định.

Ví dụ 1: UVa 10181, 15-Puzzle Problem. Bài toán 15-puzzle mở rộng từ 8-puzzle. Trên bảng 4×4 có 15 ô đánh số và một ô trống; mỗi bước có thể đưa một ô kề ô trống vào vị trí trống. Cần tìm số bước ít nhất để tới trạng thái đích. Dữ liệu bảo đảm mọi trường hợp có lời giải đều tìm được trong không quá 45 bước.

Việc phán đoán vô nghiệm cần phân tích riêng về puzzle số, bài viết không đi sâu. Với phần tìm kiếm, một ước lượng rất đơn giản là tổng khoảng cách Manhattan từ mỗi ô không trống tới vị trí đích. Cách này tương đương với việc bỏ qua ràng buộc “chỉ được đổi chỗ với ô trống”, đồng thời bỏ qua ảnh hưởng của một phép đổi chỗ tới ô còn lại.

Vì đã bỏ bớt ràng buộc, ta có $h(s) \leq h^*(s)$. Với dữ liệu có lời giải trong 45 bước, hiệu quả cắt tỉa và hiệu quả giảm số vòng lặp đều khá tốt.

4.2.4 Anytime algorithm

Lớp này còn được gọi là thuật toán có thể ngắt giữa chừng. Không phải bài nào cũng dễ tìm hàm ước lượng hiệu quả. Đôi khi ta chỉ ước lượng rất thô, không kiểm soát được $h(s) \leq h^*(s)$, nên cũng không thể thu hẹp không gian tìm kiếm một cách đúng đắn. Những bài như vậy thường xuất hiện dưới dạng nộp đáp án. Ta có thể dùng thuật toán cải thiện dần, trong thời gian thi cố gắng thu được nghiệm càng tốt càng tốt.

Ví dụ 2: CTSC 2009 Day 2, trò chơi số $N \times N$. Trò chơi gồm $N^2 - 1$ ô trượt và một ô trống. Mục tiêu là di chuyển ô trống lên, xuống, trái, phải để đưa các ô về một thứ tự quy định. Đây là bài nộp đáp án, với $3 \leq N \leq 6$ và 10 bộ dữ liệu.

So với ví dụ 1, miền dữ liệu lớn hơn nhiều. Đổi lại, bài không yêu cầu nghiệm chính xác; chỉ cần trong thời gian cho phép tìm được nghiệm đủ tốt thì vẫn có điểm đáng kể.

Nếu dùng trực tiếp ước lượng Manhattan của ví dụ 1 rồi chạy IDA*, ta có thể nhanh chóng tìm được nghiệm tối ưu cho 5 bộ nhỏ. Nhưng giá trị ước lượng vẫn cách đáp án thật khá xa, nên các bộ còn lại không xử lý được.

Một cách sửa đơn giản là đặt

$$h'(s) = c h(s),$$

với c là hằng số điều chỉnh theo từng bộ dữ liệu. So với $h(s)$, $h'(s)$ gần số bước thật hơn, nên có thể đưa vào A^* . Tuy vậy, A^* tốn bộ nhớ rất lớn, đó cũng là lý do ví dụ 1 không dùng A^* .

Với bài nộp đáp án, ta có thể thỏa hiệp: chỉ giữ lại một phần trạng thái có ước lượng tốt nhất. Mỗi lần chọn trạng thái tốt nhất để mở rộng; nếu không còn chỗ lưu trạng thái mới thì bỏ trạng thái kém nhất. Vì bài có rất nhiều phương án lời giải, việc loại bỏ một số trạng thái “xấu” ít ảnh hưởng đến đáp án cuối. Ngoại trừ một bộ đặc biệt, cách này có thể đạt khoảng 9 đến 10 điểm mỗi bộ.

4.3 Gộp heuristic

Gộp heuristic là một chiến lược hợp nhất đơn giản, có thể tối ưu hiệu quả cho các thuật toán cần gộp dữ liệu.

4.3.1 Gộp heuristic trên cấu trúc dữ liệu

Giả sử cần gộp hai cấu trúc dữ liệu A, B , số phần tử đang được chúng quản lý lần lượt là a, b và $a \leq b$. Ta tách toàn bộ thông tin trong cấu trúc nhỏ A , rồi lần lượt chèn chúng vào cấu trúc lớn B .

Định lý 3.1. Giả sử sau một số phép gộp ta thu được một cấu trúc dữ liệu có n phần tử. Khi dùng chiến lược gộp heuristic, tổng số lần chèn phát sinh là $O(n \log n)$.

Chứng minh. Xét riêng từng phần tử. Khi một phần tử bị chèn sang cấu trúc khác, kích thước cấu trúc chứa nó sau phép gộp ít nhất gấp đôi trước đó. Sau $O(\log n)$ lần chèn, phần tử đó chắc chắn đã nằm trong cấu trúc cuối cùng. Cộng trên mọi phần tử, ta được $O(n \log n)$.

Chiến lược này có thể gộp các cấu trúc dữ liệu phức tạp, và cũng dễ mở rộng cho các kiểu tập dữ liệu khác.

Ví dụ 3: HNOI 2009, Bánh pudding huyền ảo. Có n chiếc pudding xếp thành một hàng và m thao tác. Mỗi lần đổi toàn bộ pudding của một màu thành màu khác, rồi hỏi hiện có bao nhiêu đoạn màu liên tiếp. Dữ liệu có $n \leq 10^6$.

Ta dùng danh sách liên kết hoặc vector để lưu các vị trí của từng màu. Khi đổi màu, chọn danh sách nhỏ hơn để tách thô, sửa từng vị trí và cập nhật đóng góp vào số đoạn. Theo Định lý 3.1, tổng số lần sửa là $O(n \log n)$, nên có thể bảo trì trực tiếp.

Ví dụ 4: BZOJ 3510, Thủ đô. Cần duy trì một rừng n đỉnh với ba thao tác:

- nối hai đỉnh u, v , bảo đảm sau khi nối vẫn là rừng;
- hỏi trọng tâm của cây chứa đỉnh x , nếu có hai trọng tâm thì lấy chỉ số nhỏ hơn;
- hỏi xor chỉ số trọng tâm của tất cả các cây.

Trước hết xét việc thêm một lá vào một cây. Trọng tâm chỉ có thể di chuyển về phía lá mới, và nhiều nhất di chuyển một lần.

Khi nối hai cây, ta tách thô cây có ít đỉnh hơn và thêm từng đỉnh vào cây lớn dưới dạng lá, đồng thời duy trì trọng tâm. Theo Định lý 3.1, số lần thêm lá là $O(n \log n)$; chỉ cần thêm một cấu trúc dữ liệu để kiểm tra trọng tâm có di chuyển hay không.

4.3.2 Hợp nhất theo hạng trong DSU

Hợp nhất theo hạng là một cách duy trì DSU rất thực dụng: mỗi lần nối gốc của tập nhỏ vào gốc của tập lớn. Từ Định lý 3.1 suy ra chiều cao cây là $O(\log n)$. Khi cài đặt, cũng có thể trực tiếp duy trì độ cao rồi hợp nhất theo hạng.

Cách này bảo đảm độ sâu cây DSU ngay cả khi không nén đường đi. Trong một số bài, nó tránh được việc mất thông tin khi nén đường đi.

Ví dụ 5: NOIP 2013, Vận tải bằng xe tải. Có n thành phố, m đường hai chiều; mỗi đường có giới hạn trọng lượng z_i . Với q xe tải, cần biết mỗi xe có thể chở tối đa bao nhiêu mà không vượt quá giới hạn trên đường đi. Dữ liệu:

$$n \leq 10000, \quad m \leq 50000, \quad q \leq 30000, \quad z_i \leq 100000.$$

Cách thông thường là dùng Kruskal dựng cây khung lớn nhất. Giá trị nhỏ nhất trên đường đi trong cây khung lớn nhất chính là giới hạn lớn nhất có thể giữa hai đỉnh; có thể trả lời bằng nhân đôi, với độ phức tạp $O(m \log m + q \log n)$.

Nếu dùng hợp nhất theo hạng thay cho nén đường đi, ta có thể ghi lại với mỗi đỉnh trong DSU trọng số cạnh nối nó với cha. Giá trị nhỏ nhất trên đường trong cây DSU vẫn biểu diễn giới hạn rộng nhất giữa

hai đỉnh. Trả lời thô có độ phức tạp $O(m \log m + q \log n)$, nhưng cài đặt ngắn hơn cách dựng cây rồi nhân đôi.

Vì bài toán chuyển hóa vẫn là truy vấn giá trị nhỏ nhất trên đường đi, ta vẫn có thể dùng nhân đôi; về lý thuyết phần hồi đạt $O(\log \log n)$. Nếu chọn thuật toán sắp xếp tuyến tính và vẫn nén đường đi trong DSU theo cách nối cạnh ban đầu, độ phức tạp lý thuyết có thể đạt

$$O(n + m\alpha(n) + q \log \log n).$$

4.4 Bài toán trừu tượng và học máy

Khi gặp các bài toán phân loại trừu tượng, cảm giác trực quan thường không cho căn cứ phán đoán chính xác. Khi đó có thể dùng học máy thay cho quan sát thủ công. Phần này thông qua một bài Google Code Jam để giới thiệu ngắn gọn một thuật toán phân loại thực dụng.

4.4.1 Bayes ngây thơ

Công thức Bayes. Ký hiệu $P(A | B)$ là xác suất sự kiện A xảy ra khi đã biết B xảy ra; ký hiệu $P(A \cap B)$ là xác suất cả A và B cùng xảy ra.

Định lý 4.1.

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Chứng minh. Ta có

$$P(A \cap B) = P(A)P(B | A) = P(B)P(A | B).$$

Chuyển về sẽ thu được công thức Bayes.

Phán đoán ngây thơ. Gọi S là sự kiện cần phân loại, A và B là hai lớp có thể chọn. Nếu $P(A | S) > P(B | S)$, ta cho rằng S thuộc lớp A ; ngược lại thuộc lớp B . Khi có nhiều lớp hơn, chọn lớp có xác suất lớn nhất.

Giả định ngây thơ. Giả sử sự kiện cần phân loại có các thuộc tính

$$S = \{S_1, S_2, \dots, S_m\}.$$

Nếu giả định các thuộc tính đặc trưng này độc lập với nhau khi đã biết lớp, ta có thể xấp xỉ

$$P(S | A) \approx \prod_{i=1}^m P(S_i | A).$$

So sánh các xác suất xấp xỉ như vậy sẽ cho kết quả phân loại. Các giá trị $P(S_i | A)$ có thể tính bằng lấy mẫu ngẫu nhiên hoặc bằng một thuật toán riêng cho bài toán.

4.4.2 Ví dụ 6: GCJ Round 1A 2014, *Proper Shuffle*

Có hai cách sinh hoán vị. Đề cho 120 hoán vị độ dài 1000, mỗi hoán vị được sinh độc lập bởi một trong hai thuật toán. Cần nhận diện hoán vị nào do thuật toán nào sinh ra; nếu trả lời đúng ít nhất 109 trong 120 hoán vị thì được chấp nhận.


```

Algorithm 1 GOOD
for k = 0 to n - 1 do
    a[k] = k
end for
for k = 0 to n - 1 do
    p = random_int(k..n - 1)
    swap(a[k], a[p])
end for

```

```

Algorithm 2 BAD
for k = 0 to n - 1 do
    a[k] = k
end for
for k = 0 to n - 1 do
    p = random_int(0..n - 1)
    swap(a[k], a[p])
end for

```

Thuật toán GOOD sinh mọi hoán vị với xác suất bằng nhau, còn BAD thì không.

Phân tích đơn giản. Với GOOD, sau khi quyết định vị trí k , xác suất xuất hiện của mọi giá trị tại vị trí đó đều là $1/n$, và các bước sau không ảnh hưởng vị trí đó.

Với BAD, khi quyết định phép đổi tại vị trí k , nếu $\text{swap}(a[k], a[p])$ có $p < k$, giá trị vốn ở vị trí $a[k]$ bị đổi tới một vị trí cao hơn, rồi sau đó còn có thể tiếp tục bị đổi tới vị trí cao hơn nữa. Điều này tạo ảnh hưởng tinh vi tới xác suất một giá trị xuất hiện tại từng vị trí.

Một đặc trưng đơn giản. Gọi S là hoán vị đầu vào. Có thể dùng số lượng vị trí thỏa $S_i < i$ làm đặc trưng. Phân tích của cuộc thi cho thấy phân bố đặc trưng này giữa GOOD và BAD khác nhau rõ rệt; nếu dùng một ngưỡng thích hợp thì tỉ lệ đúng có thể vượt 90%.

Phân loại Bayes ngây thơ. Ta muốn tính $P(\text{GOOD} \mid S)$. Theo Định lý 4.1:

$$P(\text{GOOD} \mid S) = \frac{P(S \mid \text{GOOD})P(\text{GOOD})}{P(S \mid \text{GOOD})P(\text{GOOD}) + P(S \mid \text{BAD})P(\text{BAD})}.$$

Vì $P(\text{GOOD}) = P(\text{BAD})$, công thức rút gọn thành

$$P(\text{GOOD} \mid S) = \frac{P(S \mid \text{GOOD})}{P(S \mid \text{GOOD}) + P(S \mid \text{BAD})},$$

và tương tự

$$P(\text{BAD} \mid S) = \frac{P(S \mid \text{BAD})}{P(S \mid \text{GOOD}) + P(S \mid \text{BAD})}.$$

Nếu $P(\text{GOOD} \mid S) > P(\text{BAD} \mid S)$, xác suất S do GOOD sinh ra lớn hơn. Điều kiện này tương đương

$$P(S \mid \text{GOOD}) > P(S \mid \text{BAD}).$$

Với GOOD, mọi hoán vị cùng xác suất:

$$P(S \mid \text{GOOD}) = \frac{1}{n!}.$$

Việc tính chính xác $P(S \mid \text{BAD})$ lại rất khó, nên ta xấp xỉ:

$$P(S \mid \text{BAD}) \approx \prod_{k=0}^{n-1} P(S_k \mid \text{BAD}).$$

Để so sánh công bằng hơn, ta cũng dùng xấp xỉ tương tự cho GOOD:

$$P(S \mid \text{GOOD}) \approx \prod_{k=0}^{n-1} \frac{1}{n}.$$

Nếu $P_{i,j}$ là xác suất sau khi chạy BAD ta có $S_i = j$, ta dễ thiết kế một DP $O(n^2)$ để tính toàn bộ bảng này. Từ đó có thể tính $P(S \mid \text{BAD})$ xấp xỉ rồi so sánh với $P(S \mid \text{GOOD})$. Khi tính trực tiếp xác suất có thể gặp yêu cầu độ chính xác cao; thực tế có thể so sánh $P(S \mid \text{BAD}) n^n$ với 1.0, hoặc dùng ngưỡng khác vì bản thân quá trình đã là xấp xỉ.

4.5 Bài toán chia đều có trọng số

Ví dụ 7. Cho một đa tập số nguyên dương S . Ký hiệu $f(S)$ là tổng các phần tử trong S . Chia S thành hai tập S_1, S_2 , với $S_1 + S_2 = S$, sao cho $|f(S_1) - f(S_2)|$ nhỏ nhất, đồng thời đưa ra phương án chia.

4.5.1 Ví dụ đơn giản

Với $S = \{3, 1, 1, 2, 2, 1\}$, có hai cách chia hợp lệ làm $f(S_1) = f(S_2) = 5$, tức $|f(S_1) - f(S_2)| = 0$:

$$S_1 = \{1, 1, 1, 2\}, \quad S_2 = \{2, 3\};$$

$$S_1 = \{3, 1, 1\}, \quad S_2 = \{2, 2, 1\}.$$

4.5.2 Một thuật toán quy hoạch động

Có thể thiết kế DP giống ba lô: đặt $d[i][j]$ cho biết sau khi dùng i phần tử đầu tiên, có thể đạt $f(S_1) = j$ hay không. Độ phức tạp là $O(nf(S))$.

Khi mọi phần tử trong S đều nhỏ, đây đúng là thuật toán tốt. Nhưng thực chất đây chỉ là thuật toán giả đa thức.

4.5.3 Một ý tưởng tham lam đơn giản

Ý tưởng tự nhiên là đọc lần lượt từng phần tử và đặt nó vào tập hiện có tổng nhỏ hơn. Để hai tổng dần áp sát nhau, ta sắp xếp các phần tử giảm dần trước khi xử lý; độ phức tạp là $O(n \log n)$.

Thuật toán này rất tự nhiên và kết quả thường không tệ, nhưng phân tích kỹ sẽ thấy nó có lỗ hổng rõ ràng.

4.5.4 Thuật toán sai phân

Thuật toán sai phân cải tiến thuật toán tham lam ở trên. Ý tưởng ngắn gọn, nhưng chất lượng nghiệm xấp xỉ tốt hơn nhiều.

Tư tưởng cơ bản. Khi đặt hai số X, Y vào hai tập khác nhau, phần ảnh hưởng tới đáp án chỉ là độ lớn của hiệu giữa chúng. Điều này tương đương tạo ra một số mới $|X - Y|$, rồi tiếp tục xem nó như một số thường trong thuật toán.

Nhìn lại thuật toán tham lam trước đó, xét trạng thái ở mỗi quyết định. Giả sử phần tử mới là X , còn các phần tử đã quyết định tạo ra sai phân Y . Việc đưa X vào tập có tổng nhỏ hơn tương đương thay X, Y bằng $|X - Y|$. Nhưng mục đích ban đầu của ta là xử lý phần tử lớn trước, nên quyết định theo thứ tự đọc vào là chưa tối ưu.

Quy trình thuật toán. Theo tư tưởng sai phân, ta dùng heap và DSU để bảo trì:

1. Đưa mọi phần tử của S vào heap. Trong DSU, mỗi phần tử ban đầu là một tập riêng.
2. Lấy hai phần tử lớn nhất khỏi heap, gọi là X, Y ; trong DSU chúng tương ứng với hai tập S_X, S_Y . Quyết định đặt S_X và S_Y ở hai phía khác nhau, đưa $|X - Y|$ trở lại heap, và nối hai tập trong DSU bằng một cạnh trọng số 1 biểu diễn việc đổi phía.
3. Nếu heap chỉ còn một phần tử thì dừng, ngược lại lặp bước trước.
4. Chọn gốc của cây DSU thuộc S_1 . Các phần tử khác dựa vào tính chẵn lẻ của độ dài đường tới gốc để quyết định thuộc S_1 hay S_2 : chẵn cùng phía, lẻ khác phía.

Khi cài đặt DSU, có thể duy trì tính chẵn lẻ của độ dài đường trong lúc nén đường đi, hoặc dùng hợp nhất theo hạng để bảo đảm chiều cao $O(\log n)$.

Hiệu quả xấp xỉ. Thuật toán có độ phức tạp $O(n \log n)$, rất hiệu quả; điều quan trọng là nghiệm có đủ tốt hay không.

Khi n không quá nhỏ, chẳng hạn $n > 1000$, với tập số sinh ngẫu nhiên $S_i \in [0, 2^{30})$, thuật toán gần như chắc chắn chia được S_1, S_2 sao cho

$$|f(S_1) - f(S_2)| \leq 1.$$

Nói cách khác, nếu $f(S)$ lẻ thì sai khác bằng 1, còn nếu $f(S)$ chẵn thì sai khác bằng 0. Có thể thấy xác suất đúng của thuật toán này rất cao.

Khi n nhỏ, có thể dùng thuật toán này để thiết kế hàm ước lượng rồi chạy tìm kiếm heuristic nhằm thu được nghiệm tốt hơn.

4.5.5 Ví dụ 8: POI 2013, *Polarization*

Cho một cây n đỉnh. Hãy định hướng mỗi cạnh thành cạnh có hướng. Nếu từ đỉnh u đi được tới đỉnh v , gọi (u, v) là một cặp hợp lệ. Cần tìm số cặp hợp lệ nhỏ nhất và lớn nhất trong mọi cách định hướng. Dữ liệu có $n \leq 250000$.

Chuyển hóa bài toán. Cây là đồ thị hai phía, nên giá trị nhỏ nhất chắc chắn là $n - 1$.

Với giá trị lớn nhất, nghiệm tối ưu luôn tồn tại một đỉnh x sao cho mọi đỉnh còn lại hoặc có đường đi tới x , hoặc có đường đi từ x tới nó. Phân tích tiếp cho thấy x phải chọn ở trọng tâm của cây.

Lấy x làm gốc, cây tách thành một số cây con. Bài toán còn lại là chia các cây con này thành hai tập sao cho tích kích thước của hai tập lớn nhất.

Thuật toán tối ưu truyền thống. Chú ý rằng tổng các kích thước ở đây là n , nên số giá trị phần tử khác nhau chỉ là $O(\sqrt{n})$. Ta có thể chuyển thành bài toán ba lô đa trọng, một bài toán kinh điển không trình bày lại ở đây. Độ phức tạp là $O(n\sqrt{n})$.

Thuật toán xấp xỉ. Nếu dùng trực tiếp thuật toán sai phân, có thể qua khoảng 80% dữ liệu. Nguyên nhân chính là khi số phần tử ít, sai lệch của nghiệm xấp xỉ lớn hơn.

Ta đặt một ngưỡng. Khi số cây con nhỏ hơn ngưỡng này, dùng ba lô vét cạn; các trường hợp còn lại dùng thuật toán sai phân. Cách ghép này có thể qua toàn bộ dữ liệu chính thức của POI, với độ phức tạp $O(n \log n)$.

Khi thuật toán tối ưu truyền thống đủ nhanh, không nên ưu tiên thuật toán xấp xỉ. Ví dụ này chỉ nhằm cho thấy thuật toán sai phân có thể tạo ra nghiệm xấp xỉ rất tốt.

4.6 Bài toán cây Steiner nhỏ nhất

Ví dụ 9. Cho đồ thị vô hướng có trọng số $G = (V, E)$. Tập đỉnh đặc biệt S là một tập con của V . Cần tìm một đồ thị liên thông

$$T = (V_T, E_T)$$

sao cho $S \subseteq V_T \subseteq V$, $E_T \subseteq E$, $|E_T| = |V_T| - 1$, và tổng trọng số các cạnh trong E_T là nhỏ nhất.

4.6.1 Một thuật toán tối ưu

Cây Steiner nhỏ nhất có thuật toán trạng thái nén kinh điển, chuyển về mô hình đường đi ngắn nhất; có thể xem luận văn đội tuyển quốc gia năm 2014 của Jiang Jinye. Độ phức tạp là

$$O(|V| 3^{|S|} + |E| 2^{|S|}),$$

đây là thuật toán tốt nhưng chỉ áp dụng được cho miền dữ liệu nhỏ.

4.6.2 Một ý tưởng tham lam đơn giản

Một tham lam đơn giản là tìm cây khung nhỏ nhất của đồ thị gốc, rồi xóa các cạnh thừa trên cây khung, tức xóa nhiều nhất có thể nhưng vẫn giữ cho tập S liên thông.

Thuật toán này có lỗi hổng lớn. Trong ví dụ của bản gốc, với

$$V = \{A, B, C, 1, 2, 3, 4, 5\}, \quad S = \{A, B, C\}, \quad x > 2,$$

tham lam theo cây khung nhỏ nhất chọn các cạnh trọng số x và cho nghiệm kém, trong khi nghiệm tối ưu đi qua các cạnh trọng số 1 và các cạnh $x + 1$ như hình minh họa.

4.6.3 Thuật toán heuristic

Dựa trên ví dụ trên, ta cải tiến có chủ đích.

Phân tích thêm. Trong ví dụ, nếu tạm bỏ qua các điểm 1 và 2, rồi xét khoảng cách ngắn nhất giữa các điểm còn lại, cây khung nhỏ nhất trên đồ thị rút gọn đã có thể cho nghiệm tối ưu. Để tổng quát hơn, ta loại cả những điểm không thuộc S khỏi đồ thị rút gọn.

Vấn đề là các đường đi ngắn nhất giữa các cặp đỉnh của S có thể giao nhau. Khi cuối cùng tạo cây, các cạnh giao nhau không được tính lặp.

Quy trình thuật toán. Sau khi chỉnh lý, ta có thuật toán sau:

1. Từ đồ thị gốc $G = (V, E)$, dựng đồ thị đầy đủ $G_1 = (S, E_1)$. Trọng số cạnh giữa hai đỉnh trong S là độ dài đường đi ngắn nhất giữa chúng trong G .
2. Tìm cây khung nhỏ nhất T_1 của G_1 . Nếu có nhiều nghiệm thì chọn tùy ý.
3. Dựng một đồ thị con $G_2 = (V_2, E_2)$ của G : với mỗi cạnh của T_1 , đưa vào G_2 một đường đi ngắn nhất tương ứng trong G . Nếu có nhiều đường đi ngắn nhất thì chọn tùy ý.
4. Tìm cây khung nhỏ nhất T_2 của G_2 . Nếu có nhiều nghiệm thì chọn tùy ý.
5. Xóa khỏi T_2 các cạnh vô dụng sao cho mọi lá còn lại đều là đỉnh thuộc S . Kết quả là cây Steiner T .

Phân tích từng bước:

- Dựng G_1 cần chạy đường đi ngắn nhất một nguồn $|S|$ lần, độ phức tạp $O(|S||V|^2)$ nếu dùng cách cơ bản.
- Tìm cây khung nhỏ nhất của G_1 ; vì G_1 là đồ thị đầy đủ, nên dùng Prim với độ phức tạp $O(|S|^2)$.
- Khi dựng G_2 , mỗi đường đi ngắn nhất có thể qua $O(|V|)$ đỉnh, nhưng tổng số cạnh đường đi được thêm vẫn bị chặn bởi $O(|V|)$ theo phân tích của bản gốc cho trường hợp này.
- Tìm cây khung nhỏ nhất của G_2 , độ phức tạp $O(|V|^2)$ hoặc $O(|V| \log |V|)$ tùy cài đặt.
- Xóa cạnh vô dụng trong $O(|V|)$.

Có thể thấy thuật toán chạy rất hiệu quả, đồng thời chất lượng xấp xỉ cũng tốt. Phần tiếp theo chúng minh chặn trên tương đối cho nghiệm xấp xỉ của nó. Có thể thay bước đường đi ngắn nhất bằng thuật toán hiệu quả hơn để tối ưu thêm độ phức tạp.

Hiệu quả xấp xỉ. Gọi D_T là tổng trọng số cây Steiner T do thuật toán trên thu được, và $D_{\min}$ là tổng trọng số cây Steiner nhỏ nhất $T_{\min}$. Ta sẽ chặn tỉ số $D_T/D_{\min}$.

Bổ đề 6.1. Với một cây T có $m > 1$ cạnh, tồn tại một vòng

$$u_0, u_1, \dots, u_{2m} = u_0$$

thỏa các tính chất:

- mỗi cạnh của T xuất hiện đúng hai lần trong vòng;
- mỗi lá của T chỉ xuất hiện một lần trong vòng, không tính việc u_0 lặp lại ở cuối;
- nếu u_i và u_j , với $i < j$, là hai lá liên tiếp trên vòng, và giữa chúng không có lá nào khác, thì $u_i, u_{i+1}, \dots, u_j$ là một đường đi đơn trên T .

Chứng minh. Chọn một đỉnh làm gốc, rồi duyệt Euler trên cây T . Dãy Euler thu được thỏa các tính chất trên nếu xem mỗi cạnh vô hướng như hai cạnh có hướng ngược nhau.

Định lý 6.1. Gọi leaf là số lá của cây Steiner nhỏ nhất $T_{\min}$. Khi đó

$$\frac{D_T}{D_{\min}} < 2 \left(1 - \frac{1}{\text{leaf}} \right) \leq 2 \left(1 - \frac{1}{|S|} \right).$$

Chứng minh. Vì $\text{leaf} \leq |S|$, bất đẳng thức bên phải là hiển nhiên.

Theo Bổ đề 6.1, trong $T_{\min}$ có thể tạo một vòng L bằng cách đi mỗi cạnh hai lần. Nếu trọng số vòng là D_L , thì $D_L = 2D_{\min}$.

Trên vòng L , chọn đường đơn dài nhất giữa hai lá liên tiếp rồi xóa nó đi; phần còn lại là một đường đi P . Vì có leaf đoạn giữa các lá, đoạn dài nhất có trọng số ít nhất D_L/leaf , do đó

$$D_P \leq \left(1 - \frac{1}{\text{leaf}}\right) D_L = 2 \left(1 - \frac{1}{\text{leaf}}\right) D_{\min}.$$

Trong G_1 , cạnh giữa hai đỉnh thuộc S luôn lấy đường đi ngắn nhất trong G , nên cây khung nhỏ nhất trên G_1 không nặng hơn đường P . Các bước khử trùng cạnh và xóa cạnh vô dụng sau đó không làm tăng tổng trọng số. Vì vậy

$$D_T \leq D_P < 2 \left(1 - \frac{1}{\text{leaf}}\right) D_{\min}.$$

Định lý 6.2. Khi $\text{leaf} > 2$, tỉ lệ xấu nhất đạt đúng

$$\frac{D_T}{D_{\min}} = 2 \left(1 - \frac{1}{\text{leaf}}\right).$$

Chứng minh. Dựng đồ thị $G = (V, E)$ như sau:

$$V = \{v_1, v_2, \dots, v_{\text{leaf}}, v_{\text{leaf}+1}\}, \quad S = \{v_1, v_2, \dots, v_{\text{leaf}}\}.$$

Đặt trọng số

$$d(v_i, v_j) = \begin{cases} 2, & j = i + 1, i < \text{leaf}, \\ 1, & i \leq \text{leaf}, j = \text{leaf} + 1, \\ \infty, & \text{các trường hợp khác.} \end{cases}$$

Trong trường hợp xấu nhất, thuật toán có thể chọn đường đi nối các đỉnh $v_1, v_2, \dots, v_{\text{leaf}}$ thành một chuỗi với tổng trọng số $2(\text{leaf} - 1)$. Trong khi đó, cây Steiner tối ưu là ngôi sao qua $v_{\text{leaf}+1}$, có tổng trọng số leaf . Do đó

$$\frac{D_T}{D_{\min}} = \frac{2(\text{leaf} - 1)}{\text{leaf}} = 2 \left(1 - \frac{1}{\text{leaf}}\right),$$

đúng bằng chặn trên.

4.6.4 Ví dụ 10: kiểm tra lẫn nhau đội tuyển 2015, *Marketing network*

Cho đồ thị $G = (V, E)$. Cần tìm K rừng sinh nhỏ nhất đầu tiên, với ràng buộc tập đỉnh đặc biệt S phải liên thông trong rừng sinh. Dữ liệu:

$$|V|, K < 50, \quad |E| < 100, \quad |S| < 15,$$

cạnh, trọng số cạnh và tập S đều được sinh ngẫu nhiên.

Thuật toán tối ưu. Bài toán nghiệm tốt thứ k có thể dùng A^* . Ta mô tả trạng thái bằng việc mỗi cạnh đã được chọn hay chưa; khi tìm kiếm, quyết định cạnh tiếp theo có được chọn hay không.

Nếu dùng thuật toán trạng thái nén cho cây Steiner làm hàm ước lượng, ta có $h(s) = h^*(s)$. Mỗi lần mở rộng trạng thái tốt nhất. Vì yêu cầu là rừng sinh, sau mỗi $O(n)$ quyết định hiệu lực có thể tìm được nghiệm tốt nhất hiện tại; tìm K nghiệm đầu chỉ cần $O(Km)$ lần tính ước lượng.

Thuật toán này bảo đảm tính đúng và độ phức tạp, nhưng hiệu suất thực tế thấp, không đủ cho miền dữ liệu của bài này.

Thuật toán xấp xỉ. Tiếp tục dùng tư tưởng A^* , nhưng lấy thuật toán heuristic ở phần trước làm ước lượng. Bất lợi là giá trị ước lượng có thể lớn hơn $h^*(s)$, nên không còn thể cất tĩa một cách chắc chắn.

Ta xử lý xấp xỉ bằng cách chọn một hằng số c , dùng $ch(s)$ làm tham khảo cho cận dưới, rồi cất các trạng thái có ước lượng kém. Điều chỉnh c cho phép cân bằng giữa tốc độ chạy và độ chính xác.

4.7 Lời cảm ơn

Xin cảm ơn Hiệp hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy Chen Heli, Dong Yehua, Shao Hongxiang và You Guanghui của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và hướng dẫn tôi.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yu Linyun và Chen Xuji đã chỉ dẫn.

Xin cảm ơn các anh Yu Yili, Dong Anhua và He Qizheng của Đại học Thanh Hoa đã giúp đỡ tôi.

Xin cảm ơn các bạn Zhang Hengjie, Wang Jianhao và Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi ý cho tôi.

Tài liệu tham khảo của bài viết

- [1] Wikipedia, Heuristic (computer science).
- [2] Wikipedia, Partition problem.
- [3] Wikipedia, Steiner tree problem.
- [4] L. Kou, G. Markowsky and L. Berman, “A fast algorithm for Steiner trees”, *Acta Informatica*.
- [5] Jiang Jinye, *Công cụ mạnh cho giải lập: tối ưu và ứng dụng thuật toán SPFA*, luận văn đội tuyển quốc gia 2009.
- [6] Zhou Erjin, *Bàn về ứng dụng hàm ước lượng trong thi tin học*, luận văn đội tuyển quốc gia 2009.
- [7] Yu Yili, *Một số phương pháp tìm nghiệm tốt thứ k* , luận văn đội tuyển quốc gia 2014.

5 Bàn về một số phương pháp khớp chuỗi

Wang Jianhao

Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài toán khớp chuỗi là một bài toán kinh điển trong thi tin học. Bài viết phân loại và tổng kết các cách giải cho lớp bài toán này. Trước hết tác giả điểm lại một số cấu trúc và thuật toán thường dùng, sau đó tập trung giới thiệu một cấu trúc khớp chuỗi mới là *Border Tree*, từ đó rút ra một cách giải tổng quát cho một lớp bài toán khớp chuỗi. Mỗi thuật toán và kỹ thuật đều đi kèm ví dụ để thuận tiện cho việc hiểu và phân tích.

5.1 Dẫn nhập

Bài toán khớp chuỗi thường xuất hiện trong xử lý văn bản. Trong thời đại dữ liệu lớn, nhiều lĩnh vực phải xử lý các thao tác đa dạng trên lượng dữ liệu lớn. Bài toán khớp chuỗi trong thi tin học là một mô hình

tiêu biểu của kiểu vấn đề đó; bài viết này giới thiệu một số cách giải cho chúng.

Mục 2 nhắc lại các định nghĩa về chuỗi để thuận tiện cho phần sau. Tiếp đó, bài viết tổng kết ngắn gọn một số cấu trúc và thuật toán, chia chúng thành hai nhóm: cách giải dựa trên tiền tố và cách giải dựa trên hậu tố.

Mục 3 giới thiệu các cấu trúc, thuật toán tiền tố gồm trie, KMP và AC automaton. Mục 4 giới thiệu các cấu trúc hậu tố gồm suffix array, suffix tree, suffix cactus và suffix automaton. Mục 5 trình bày chi tiết cấu trúc mới Border Tree. Mục 6 dùng Border Tree để đưa ra một cách giải tổng quát cho một lớp bài toán khớp chuỗi.

Nội dung ở Mục 3 và Mục 4 khá phổ biến, một số thuật toán cũng đã từng được trình bày trong các luận văn đội tuyển quốc gia trước đây, nên ở đây tác giả chỉ tóm tắt và phân tích một số bài toán kinh điển. Mục 5 và Mục 6 được trình bày kỹ hơn. Border Tree là một cấu trúc tương đối mới; các ứng dụng nêu ra trong bài viết nhằm gợi mở để độc giả quan tâm tiếp tục nghiên cứu.

5.2 Định nghĩa

Để tiện mô tả, trước hết nêu một số định nghĩa.

Định nghĩa 2.1 (tiền tố, hậu tố, xâu con, Border, LBorder). Xét xâu $s = s_1s_2 \dots s_n$ có độ dài n .

Với $1 \leq i \leq j \leq n$, ký hiệu $s[i : j] = s_is_{i+1} \dots s_j$. Khi $i > j$, quy ước $s[i : j]$ là xâu rỗng. Nếu $u = s[i : j]$, ta gọi u là một xâu con của s .

Với $1 \leq i \leq n$, $s[i : n]$ là một hậu tố của s , ký hiệu $\text{suf}[i]$; $s[1 : i]$ là một tiền tố của s , ký hiệu $\text{pre}[i]$.

Với $1 \leq i < n$, nếu $s[1 : i] = s[n - i + 1 : n]$, thì $s[1 : i]$ là một *Border* của s . Thông thường xâu rỗng cũng được xem là một Border. Trong mọi Border của s , Border dài nhất được gọi là *LBorder* của s . Tương tự có thể nói $\text{pre}[i]$ có LBorder, hay một xâu con có LBorder.

Định nghĩa 2.2 (xâu tuần hoàn, gốc, độ dài chu kỳ). Với xâu $s = s_1s_2 \dots s_n$, nếu LBorder của s có độ dài L , thì $s[L + 1 : n]$ được gọi là một gốc của s . Nếu $2L > n$, ta gọi s là xâu tuần hoàn. Khi s là xâu tuần hoàn, độ dài của gốc được gọi là độ dài chu kỳ.

Định nghĩa 2.3 (xâu lặp). Nếu một xâu t nhận được bằng cách lặp một xâu s nguyên số lần $k > 1$, thì t được gọi là xâu lặp, ký hiệu $t = s^k$. Rõ ràng xâu lặp là một lớp đặc biệt của xâu tuần hoàn.

5.3 Các cách giải theo tiền tố

Trong các phương pháp khớp chuỗi của thi tin học, có một số cấu trúc và thuật toán dựa trên việc tận dụng hiệu quả thông tin tiền tố để khớp nhanh. Bài viết gọi chúng là nhóm cách giải theo tiền tố. Mục này giới thiệu ba cấu trúc tiền tố và các biến thể của chúng.

5.3.1 Trie

Trie là cây tra cứu từ; dưới đây gọi là cây chữ cái. Cây chữ cái hỗ trợ tìm kiếm và khớp nhiều xâu mẫu. Định nghĩa và phương pháp xây dựng có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan hoặc luận văn năm 2009 của Dong Huaxing.

Cây chữ cái là một cây có gốc, mỗi cạnh biểu diễn một ký tự. Ghép các ký tự trên đường đi từ gốc tới một đỉnh theo thứ tự duyệt sẽ thu được một xâu w . Vì vậy mỗi đỉnh trên cây chữ cái biểu diễn một xâu; gốc biểu diễn xâu rỗng.

Thông thường, khi khớp nhiều xâu mẫu, ta chèn từng xâu mẫu vào cây chữ cái. Theo định nghĩa của trie, mọi tiền tố của mỗi xâu mẫu đều được một đỉnh biểu diễn. Nhờ đó ta có thể dùng tính chất cây để giải nhanh nhiều bài toán khớp chuỗi, chẳng hạn tính nhanh tiền tố chung dài nhất của hai xâu mẫu.

Ví dụ 1 (USACO 2012 Dec Gold, First). Cho m xâu. Thứ tự từ điển của 26 chữ cái thường có thể được chọn tùy ý. Hỏi mỗi xâu có thể trở thành xâu nhỏ nhất theo thứ tự từ điển trong một cách chọn thứ tự chữ cái nào đó hay không. Các xâu đôi một khác nhau.

Dữ liệu:

$$1 < m < 30000, \quad 1 < \text{tổng độ dài các xâu} < 300000.$$

Giới hạn thời gian là 1 giây.

Trước hết xây trie cho m xâu. Với xâu thứ i , giả sử đỉnh u của trie biểu diễn xâu đó. Để xâu thứ i có thể là xâu nhỏ nhất theo một thứ tự từ điển nào đó, trước hết không được có xâu khác là tiền tố của nó. Ngoài ra, trên đường từ gốc tới u , mỗi ký tự được chọn phải nhỏ hơn mọi ký tự khác ở cùng tầng có thể gây cạnh tranh.

Với mỗi xâu, lấy các ràng buộc này ra rồi kiểm tra chúng có mâu thuẫn hay không. Khi cài đặt, xem mỗi ràng buộc là một cạnh có hướng giữa hai chữ cái và kiểm tra đồ thị có chu trình hay không. Độ phức tạp là

$$O(26^2 m + 26 \cdot \text{tổng độ dài các xâu}).$$

5.3.2 KMP

Trong thi tin học thường gặp bài toán khớp một xâu mẫu với một xâu chính. Cách làm vét cạn bắt đầu so từ ký tự đầu tiên; nếu bằng nhau thì so tiếp, đến khi gặp ký tự khác nhau. Lúc đó nó đơn giản bỏ toàn bộ thông tin đã khớp và thử lại từ vị trí kế tiếp của xâu chính với ký tự đầu tiên của xâu mẫu. Việc bỏ thông tin khớp trước đó gây lãng phí lớn và làm hiệu suất thấp.

Thuật toán KMP tăng tốc quá trình khớp xâu mẫu với xâu chính. Chi tiết của KMP có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan. Dưới đây nêu định nghĩa mảng thất bại của xâu mẫu.

Định nghĩa 3.1 (mảng thất bại). Cho xâu mẫu $s = s_1 s_2 \dots s_n$ có độ dài n . Đặt

$$\text{next}[i] = |\text{LBorder}(s[1 : i])|.$$

Mảng next được gọi là mảng thất bại của s .

Ý tưởng cốt lõi của KMP là tiền xử lý mảng thất bại trên xâu mẫu, rồi với xâu chính, dựa vào mảng này để tính với mỗi vị trí kết thúc thì có thể khớp tối đa bao nhiêu ký tự của xâu mẫu. Thời gian của KMP là tuyến tính.

KMP automaton. Sau khi có mảng next, nếu nối cạnh từ i tới $\text{next}[i]$ thì toàn bộ cấu trúc tạo thành một cây $n + 1$ đỉnh. Trong đó đỉnh 0 là gốc, đỉnh i biểu diễn $\text{pre}[i]$. Các đỉnh trên đường từ gốc tới đỉnh i chính là các Border của $s[1 : i]$. Vì vậy có thể dựa vào cây này để xây mảng chuyển cho mỗi đỉnh, cho biết nếu sau xâu hiện tại thêm một ký tự thì xâu mới có thể khớp nhiều nhất bao nhiêu ký tự của mẫu. Khi khớp xâu, ta dùng mảng chuyển để tính nhanh. Cấu trúc này được gọi là KMP automaton.

Ví dụ 2 (NOI 2014, Zoo). Có T bộ dữ liệu. Mỗi bộ cho một xâu mẫu $s = s_1 s_2 \dots s_n$. Với mỗi $\text{pre}[i]$, cần đếm số Border không rỗng B thỏa

$$2|B| \leq i.$$

Gọi số đó là $\text{num}[i]$, yêu cầu tính mảng num .

Dữ liệu:

$$1 < T < 5, \quad 1 < n < 1000000.$$

Giới hạn thời gian là 1 giây.

Một cách giải là xây mảng next , rồi từ đó xây cây trong KMP automaton. Với mỗi đỉnh, ta chỉ cần biết trên đường từ gốc tới đỉnh đó có bao nhiêu đỉnh có nhãn không vượt quá một nửa nhãn hiện tại. Vì nhãn trên đường đi tăng đơn điệu, có thể nhị phân trực tiếp. Mảng next được tính trong thời gian tuyến tính, nên tổng độ phức tạp là $O(Tn \log n)$.

5.3.3 AC automaton

AC automaton có thể xem là cấu trúc kết hợp trie và KMP automaton. Để xây AC automaton cho nhiều xâu mẫu, trước hết xây trie, rồi giống KMP automaton, xây mảng next để tạo cây và mảng chuyển. Tuy nhiên mảng next này dành cho nhiều xâu, khác với mảng next của KMP automaton. Trong AC automaton, cây tạo bởi các cạnh kiểu này được gọi là *fail tree*. Trên fail tree, một đỉnh biểu diễn một xâu, còn các đỉnh trong cây con của nó biểu diễn những xâu có xâu đó làm hậu tố. Dựa vào fail tree ta xây được mảng chuyển của từng đỉnh và khớp nhanh khi quét xâu.

Tính chất và cách xây AC automaton có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan. Nhờ kết hợp tính chất của trie và KMP automaton, AC automaton giải được các bài toán khớp nhiều mẫu phức tạp hơn.

Ví dụ 3 (COCI 2015, Divljak). Ban đầu cho n xâu $s_1, s_2, \dots, s_n$. Có một tập xâu T , ban đầu rỗng. Có q thao tác thuộc hai loại:

1. thêm một xâu p vào tập T ;
2. chọn một xâu s_i , hỏi trong T có bao nhiêu xâu chứa s_i làm xâu con.

Đặt sum là tổng độ dài các xâu ban đầu và

$$\text{sum}_p = \sum |p|$$

trên các xâu được thêm.

Dữ liệu:

$$1 < n, q < 100000, \quad 1 < \text{sum}, \text{sum}_p < 2000000.$$

Giới hạn thời gian là 1 giây.

Trước hết xây AC automaton cho $s_1, s_2, \dots, s_n$, thu được fail tree và mảng chuyển. Giả sử trên fail tree, đỉnh u_i biểu diễn xâu s_i .

Ta chuyển đổi thao tác như sau. Với thao tác thêm xâu p , xem như tô cùng một màu cho các đỉnh của AC automaton biểu diễn các tiền tố của p . Với thao tác hỏi xâu s_i , đáp án là số màu khác nhau trong cây con của u_i trên fail tree. Chuyển đổi này suy ra trực tiếp từ tính chất của AC automaton.

Khi cài đặt, dùng thứ tự DFS của fail tree để thống kê. Mỗi cây con tương ứng một đoạn liên tiếp trên thứ tự DFS. Sau mỗi lần tô màu, nếu một cây con có điểm mang màu đó, tổng trọng số trên đoạn DFS tương ứng cần tăng một. Với thao tác hỏi, lấy đoạn DFS của cây con u_i rồi dùng Fenwick tree để lấy tổng đoạn.

Với thao tác thêm, lấy ra các đỉnh cần tô; số lượng là $O(|p|)$. Khử trùng, sắp theo thứ tự DFS, sau đó cộng 1 tại mỗi đỉnh và trừ 1 tại LCA của hai đỉnh kề nhau trong danh sách. Sau thao tác này, trong mỗi

cây con, nếu có đỉnh được tô thì chỉ đỉnh có thứ tự DFS nhỏ nhất đóng góp 1. Có thể hiểu cách làm này qua hình dạng của virtual tree.

Nhờ đó bài toán giải được trong

$$O(26\text{sum} + \text{sum}_p \log \text{sum} + \text{sum} \log \text{sum})$$

thời gian.

5.4 Các cách giải theo hậu tố

Trong các phương pháp khớp chuỗi, nhóm hậu tố giữ vị trí quan trọng. So với nhóm tiền tố, các cấu trúc hậu tố thường cao cấp và phức tạp hơn, đồng thời giải được các bài toán khó hơn. Mục này giới thiệu bốn cấu trúc dữ liệu hậu tố: suffix array, suffix tree, suffix cactus và suffix automaton.

5.4.1 Suffix array

Suffix array là một cấu trúc dữ liệu rất tốt trong xử lý chuỗi, được ứng dụng rộng rãi ở nhiều loại bài toán.

Ý tưởng cốt lõi là với xâu mẫu b , sắp xếp mọi hậu tố của b theo thứ tự từ điển và ghi bằng mảng sa . Trong đó sa_i là vị trí của hậu tố đứng thứ i theo thứ tự từ điển. Tương ứng có mảng $rank$, trong đó $rank_i$ là vị trí của hậu tố $\text{suf}[i]$ trong mảng sa . Sau khi có sa và $rank$, ta dùng tính chất của suffix array để tính mảng $height$ trong thời gian tuyến tính; $height_i$ là độ dài LCP của hai hậu tố sa_{i-1} và sa_i . Mảng $height$ giúp tính hiệu quả nhiều bài toán, ví dụ LCP của hai hậu tố.

Suffix array có thể xây bằng thuật toán nhân đôi hoặc DC3. Do khuôn khổ bài viết, phần này không trình bày chi tiết tính chất và cách xây suffix array; độc giả quan tâm có thể xem luận văn đội tuyển quốc gia năm 2004 của Xu Zhilei hoặc luận văn năm 2009 của Luo Suiqian.

Bài toán 1: LCP. Cho xâu b độ dài n . Có q truy vấn; mỗi truy vấn cho hai xâu con của b , hỏi độ dài tiền tố chung dài nhất của chúng.

Trước hết xây suffix array của b trong $O(n \log n)$, thu được sa , $rank$ và $height$. Với mỗi truy vấn LCP của hai xâu con, kéo chúng thành hai hậu tố, lấy LCP của hai hậu tố rồi lấy min với độ dài hai xâu con. LCP của hai hậu tố trở thành truy vấn RMQ trên mảng $height$. Tiền xử lý sparse table trong $O(n \log n)$, mỗi truy vấn trả lời trong $O(1)$. Tổng độ phức tạp là $O(n \log n + q)$.

Bài toán 2: ghép xâu con. Cho xâu b độ dài n . Có q truy vấn; mỗi truy vấn chọn hai xâu con a và c của b , đặt $s = ac$, hỏi s có phải xâu con của b không. Nếu có, cần đưa ra một vị trí hợp lệ.

Xây suffix array của b trong $O(n \log n)$. Nếu s là xâu con của b , thì s là tiền tố của một hậu tố nào đó của b . Trước hết tìm toàn bộ hậu tố có tiền tố a ; theo định nghĩa suffix array, các hậu tố này tạo thành một đoạn liên tiếp trên sa , tìm được bằng nhị phân. Trên đoạn đó, tiếp tục nhị phân theo thứ tự từ điển để tìm hậu tố có tiền tố ac . Trong quá trình nhị phân, dùng mảng $rank$ và phép tính LCP $O(1)$ từ bài toán 1 để so sánh nhanh. Vì vậy mỗi truy vấn mất $O(\log n)$, tổng độ phức tạp là $O(n \log n + q \log n)$.

Bài toán 3: tiền tố lớn nhất trên đoạn. Cho xâu b độ dài n . Có q truy vấn; mỗi truy vấn cho một đoạn $[l, r]$, yêu cầu tính

$$\max_{l \leq i \leq r} \text{LCP}(\text{suf}[i], b).$$

Xây suffix array của b trong $O(n \log n)$, thu được sa , $rank$ và $height$. Bài toán trở thành RMQ trên các giá trị $\text{LCP}(\text{suf}[i], b)$. Tương tự bài toán 1, tiền xử lý sparse table trong $O(n \log n)$, mỗi truy vấn trả lời $O(1)$. Tổng độ phức tạp là $O(n \log n + q)$.

Một mở rộng là vừa tính giá trị lớn nhất vừa đếm số vị trí đạt giá trị đó. Khi đó không dùng trực tiếp sparse table kiểu chỉ lấy min/max đơn giản nữa; có thể dùng nhân đôi để trả lời mỗi truy vấn trong $O(\log n)$. Độ phức tạp trở thành $O(n \log n + q \log n)$.

5.4.2 Suffix tree

Suffix tree là cấu trúc dữ liệu kinh điển cho xử lý chuỗi. Bản chất của suffix tree là trie. Với một chuỗi b độ dài n , nếu đưa tất cả hậu tố của nó vào trie thì thu được suffix tree của b . Tuy nhiên trie trực tiếp có thể có $O(n^2)$ đỉnh, nên cần nén các đường đi. Ban đầu mỗi cạnh của trie biểu diễn một ký tự; sau khi nén, mỗi cạnh biểu diễn một chuỗi. Nếu thêm ký tự kết thúc vào cuối b , xây trie của mọi hậu tố rồi gộp các đỉnh chỉ có một con với con của nó, ta thu được cây có $O(n)$ đỉnh.

Phân tích tính chất và cách xây suffix tree có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan.

Sau khi xây suffix tree, có thể thấy mảng sa ở mục trước chính là thứ tự DFS của các lá trong suffix tree. Vì vậy có thể dùng suffix tree để xây suffix array. Ngược lại, mục sau giới thiệu một cách xây suffix tree từ suffix array. Hai cấu trúc này có thể chuyển đổi qua lại; các bài toán suffix array giải được thì suffix tree cũng hỗ trợ được. Vì thế phần này không phân tích ví dụ riêng.

5.4.3 Suffix cactus

Suffix cactus, giống suffix array, cũng dựa trên suffix tree. Nó cũng là cấu trúc dạng cây tương tự suffix tree. Trong suffix tree, ta gộp các đỉnh chỉ có một con với con của chúng; còn trong suffix cactus, ta gộp mỗi đỉnh không phải lá với một con của nó, tạo thành một khối liên thông gọi là một nhánh.

Để xây suffix cactus, trước hết xây suffix array. Dựa vào định nghĩa của sa và height, khi duyệt sa tăng dần, duy trì một ngăn xếp đơn điệu giảm của mảng height. Mỗi lần duyệt tới vị trí i , nối sa_i với phần tử hiện ở đỉnh ngăn xếp.

Do khuôn khổ bài viết, tính đúng đắn của cách xây dựng trên không được trình bày chi tiết. Tính chất cụ thể của suffix cactus có thể xem trong tài liệu giao lưu trại đông năm 2014 của Wang Yuetong, Xu Duo và Xu Zihan.

Từ suffix array có thể thu được suffix cactus. Nếu cắt mỗi nhánh của suffix cactus tại các vị trí nối cạnh, ta nhận được suffix tree. Vì vậy chỉ cần ghi lại vị trí nối cạnh khi dựng suffix cactus, ta có thể xây suffix tree từ suffix array trong thời gian tuyến tính.

5.4.4 Suffix automaton

Cây của suffix automaton là suffix tree của chuỗi mẫu đảo ngược. Vì suffix tree là một trie nén, suffix automaton cũng hỗ trợ duy trì nhiều chuỗi mẫu. Do đó ta có một cách khác để xây suffix tree của một chuỗi mẫu: trước hết xây suffix automaton, rồi xây suffix tree từ suffix automaton. Suffix automaton có thể được xây bằng phương pháp tăng dần. Do khuôn khổ bài viết, cách xây và phân tích tính chất có thể xem trong tài liệu giao lưu trại đông năm 2012 của Chen Lijie.

Ví dụ 7 (ZJOI 2015, Vùng đất huyền tưởng được chư thần ưu ái). Cho một cây n đỉnh, mỗi đỉnh mang một ký tự. Mỗi lần chọn hai đỉnh x, y , ghép các ký tự trên đường đi từ x tới y theo thứ tự để được một chuỗi. Hỏi tất cả các đường đi như vậy tạo ra bao nhiêu chuỗi khác nhau. Số lá của cây không vượt quá 20, kích thước bảng chữ cái không vượt quá 10.

Dữ liệu:

$$1 < n < 100000.$$

Giới hạn thời gian là 1 giây.

Cách vét cạn: vì chỉ có 20 lá, lấy xâu trên đường đi giữa mọi cặp lá. Mọi xâu con của các xâu giữa lá đều tương ứng với một đường đi trên cây, và mỗi đường đi đều được một xâu con như vậy biểu diễn. Do đó có thể xây suffix automaton cho 400 xâu này rồi đếm số xâu con khác nhau trên automaton. Mỗi xâu có độ dài $O(n)$, nên độ phức tạp là $O(20^2 n \cdot 10)$.

Cách tốt hơn là xét từng lá làm điểm xuất phát. Nếu lấy một lá làm gốc, toàn bộ xâu từ lá đó tới các lá khác có thể hiểu là duyệt qua cả cây một lần. Vì vậy tổng độ dài xâu khi cố định một lá làm gốc là $O(n)$. Độ phức tạp trở thành $O(20n \cdot 10)$.

5.5 Border Tree

Mục này giới thiệu cấu trúc dữ liệu Border Tree. Cấu trúc này dựa trên KMP automaton và xử lý hiệu quả một lớp bài toán khớp một xâu mẫu.

Xét xâu mẫu b độ dài m . Nếu xây KMP automaton cho b , chiều sâu của cây trong automaton có thể đạt $O(m)$. Border Tree là một biến thể của cây đó: nó gộp một số tiền tố cùng loại, làm chiều sâu giảm xuống $O(\log m)$. Nói cách khác, mỗi đỉnh của Border Tree không còn biểu diễn một tiền tố duy nhất mà biểu diễn một nhóm tiền tố; nhóm đó gọi là *Border Group*.

5.5.1 Border Group

Số Border của $b[1 : i]$ có thể đạt $O(i)$, chẳng hạn $b[1 : i] = 11 \dots 1$. Ta cần nén lượng thông tin này. Ý tưởng cơ bản là phân loại Border sao cho số loại Border của $b[1 : i]$ chỉ còn $O(\log i)$.

Định nghĩa 5.1 (Border Group). Các Border của $b[1 : i]$ được chia thành một dãy

$$g = [B_1, B_2, \dots, B_{|g|}], \quad 1 \leq |g| \leq O(\log i).$$

Mỗi B_j là một dãy các Border, và mọi Border trong B_j dài hơn mọi Border trong B_{j+1} .

Mỗi B_j có một trong hai dạng:

$$[x] \quad (k_j = 2),$$

hoặc

$$[x^k r, x^{k-1} r, \dots, x r] \quad (k_j > 1),$$

trong đó $x^k r$ là một Border của $b[1 : i]$, còn r là một tiền tố của x . Ta gọi mỗi B_j là một Border Group, và g là dãy BG.

Ví dụ 8. Cho

$$b = \text{bababbababcbababbabab}.$$

Các Border là

$$[\text{bababbabab}, \text{babab}, \text{bab}, \text{b}].$$

Có thể chia chúng thành ba nhóm:

$$[\text{bababbabab}], \quad [\text{babab}, \text{bab}], \quad [\text{b}].$$

Nhóm thứ nhất có thể viết với $r = \text{babab}, x = \varepsilon$; nhóm thứ hai với $r = \text{ab}, x = \text{b}$; nhóm thứ ba với $r = \text{b}, x = \varepsilon$.

Dãy BG được xây theo ý tưởng: dùng Border Group để phân phối mọi Border vào các dãy khác nhau; trong cùng một Border Group, các Border có cùng x và r . Quy trình:

1. Lấy $a = \text{LBorder}_i$, phân tích $a = x^k r$, rồi đưa $x^k r, x^{k-1} r, \dots, x r$ vào B_1 .
2. Đặt $s = x r$. Nếu s là xâu tuần hoàn thì lấy LBorder của s làm phần tử đầu của B_2 ; nếu không thì lấy chính s làm phần tử đầu của B_2 .
3. Lặp lại cho đến khi cả x và LBorder đều rỗng.

Cách xây này bảo đảm $1 \leq |g| \leq O(\log m)$. Lý do là ở bước 2, nếu s tuần hoàn thì s là phần tử đầu của Border Group kế tiếp. Theo định nghĩa, $|r| < |x|$, nên $|s| < \frac{k+1}{k}|x|$; trường hợp lớn nhất khi $k = 2$, do đó với mọi k đều có $|s| < \frac{3}{2}|x|$. Nếu s không tuần hoàn, phần tử đầu của nhóm kế tiếp là $\text{LBorder}(s)$. Theo định nghĩa LBorder ,

$$2|\text{LBorder}(s)| < |s|.$$

Vì $|s| = |x r|$, độ dài phần tử đầu giảm theo hằng số, nên số nhóm là $O(\log m)$.

5.5.2 Xây Border Tree

Để xây Border Tree, trước hết xây KMP automaton, sau đó chuyển cây trong automaton thành Border Tree.

Với mỗi đỉnh i của KMP automaton, định nghĩa bộ ba (x_i, y_i, z_i) :

1. Nếu $b[1 : i]$ là xâu tuần hoàn và $b[1 : i] = x^k r$ với $k > 1$, đặt $(x_i, y_i, z_i) = (|r|, |x|, k)$.
2. Nếu $b[1 : i]$ không tuần hoàn và $b[1 : i] = x r$, đặt $(x_i, y_i, z_i) = (|r|, |x|, 1)$.
3. Nếu $b[1 : i]$ không tuần hoàn nhưng có đúng một con j thỏa $z_j = 2$, đặt $(x_i, y_i, z_i) = (x_j, y_j, 1)$.

Trong trường hợp 3, định lý sau bảo đảm nhiều nhất chỉ có một con j thỏa $z_j = 2$.

Định lý 5.1. Mỗi xâu không tuần hoàn $s = b[1 : i]$ không thể có từ hai con trở lên là xâu tuần hoàn.

Chứng minh. Giả sử ngược lại s có ít nhất hai con là xâu tuần hoàn. Khi đó dựa trên hai con tuần hoàn đó, có thể biểu diễn

$$s = x^k r = y^\ell t,$$

với $|x| < |y|$ và $|y| \bmod |x| \neq 0$. Nếu $|y| \bmod |x| = 0$, thì s cũng có dạng tuần hoàn, mâu thuẫn giả thiết.

Đặt $d = \gcd(|x|, |y|)$. Khi đó s có một cách biểu diễn với chu kỳ độ dài d . Điều này dẫn đến $|x| \bmod d = 0$, mâu thuẫn với điều kiện đã chọn. Vậy giả thiết sai.

Bộ ba (x_i, y_i, z_i) có thể tính trong thời gian tuyến tính bằng hai lượt duyệt:

1. Lượt duyệt thứ nhất xử lý các đỉnh thuộc trường hợp 1 và 2 theo định nghĩa Border Group:

$$x_i = |r|, \quad y_i = |x|, \quad z_i = \left\lfloor \frac{i}{|x|} \right\rfloor.$$

2. Lượt duyệt thứ hai xử lý trường hợp 3 và cập nhật (x_i, y_i, z_i) .

Trường hợp 3 có ý nghĩa duy trì Border Group dạng $x^k r$. Ví dụ, với

$$b = abaaba,$$

ban đầu $(x_3, y_3, z_3) = (1, 2, 1)$, nhưng vì $(x_6, y_6, z_6) = (0, 3, 2)$, nên (x_3, y_3, z_3) được đổi thành $(0, 3, 1)$.

Sau khi mỗi đỉnh trong KMP automaton có một bộ ba, ta sửa cây của automaton để xây Border Tree. Những đỉnh có cùng x, y thuộc cùng một loại theo định nghĩa Border Group; chỉ giữ lại đỉnh có z nhỏ nhất và ghi thông tin của các đỉnh còn lại vào đó. Chẳng hạn nếu trong một loại có đỉnh biểu diễn r thì giữ đỉnh đó, nếu không thì giữ đỉnh biểu diễn $x r$. Cuối cùng duyệt thêm một lần để thu được Border Tree.

5.5.3 Phân tích tính chất

Mỗi đỉnh i trong Border Tree tương ứng với một Border Group, có thể viết dưới dạng

$$[x_i^{z_i} r_i, x_i^{z_i-1} r_i, \dots, x_i r_i].$$

Tuy nhiên, với một đỉnh i , các Border Group trên đường từ gốc tới i không nhất thiết biểu diễn chính xác mọi Border của $b[1 : i]$. Có thể có nhóm chứa cả những xâu không phải Border thật của $b[1 : i]$. Do đó khi cần tìm mọi Border của $b[1 : i]$, ta phải duy trì thêm một cận độ dài x trong quá trình duyệt lên tổ tiên để lọc các phần tử hợp lệ.

Ký hiệu $\text{fa}[i]$ là cha của i trong Border Tree. Trong Border Group của $\text{fa}[i]$, các Border thật của $b[1 : i]$ có dạng

$$x_j^t r_j$$

với số mũ t nằm trong một đoạn liên tiếp; biên trên của đoạn được lấy từ bộ ba tương ứng với $\text{next}[i]$ trong KMP automaton. Vì vậy khi duyệt các Border Group trên đường đi, ta chỉ cần cập nhật cận này.

5.5.4 Ví dụ

Cho

$$b = \text{babababcbabab},$$

độ dài 13. Trong bản gốc, Hình 1 là cây của KMP automaton và Hình 2 là Border Tree tương ứng. Cây ban đầu có 14 đỉnh, chiều sâu 5. Sau khi phân loại Border và xây Border Tree, cây còn 10 đỉnh, chiều sâu 3. Các đỉnh hình vuông trong cây KMP automaton được co lại khi xây Border Tree.

Bảng trong bản gốc so sánh trạng thái của từng đỉnh trong KMP automaton với bộ ba (x_i, y_i, z_i) của Border Tree. Ý nghĩa chính là: các đỉnh biểu diễn các Border cùng dạng $x^z r$ được gộp theo cùng x, y , chỉ giữ thông tin cần thiết để khi truy vấn có thể sinh lại các Border hợp lệ.

5.6 Một lớp bài toán khớp văn bản

Như đã biết, khớp xâu văn bản động với xâu mẫu tĩnh là một bài toán kinh điển. Mục này đưa ra một cách giải tổng quát hiệu quả dựa trên Border Tree. Nội dung về Border Tree đã được trình bày ở mục trước.

5.6.1 Các bài toán con

Xét xâu mẫu b độ dài m . Trước khi xử lý lớp bài toán này, cần giải ba bài toán con trên b :

1. **LCP.** Trên b , cho hai xâu con, hỏi độ dài tiền tố chung dài nhất. Cách giải xem ở Ví dụ 4. Tiền xử lý $O(m \log m)$, mỗi truy vấn $O(1)$.
2. **Ghép xâu con.** Trên b , cho hai xâu con b' và b'' , đặt $s = b' b''$, hỏi s có phải xâu con của b hay không; nếu có, đưa ra một vị trí hợp lệ. Cách giải xem ở Ví dụ 5. Tiền xử lý $O(m \log m)$, mỗi truy vấn $O(\log m)$.
3. **Tiền tố lớn nhất trên đoạn.** Trên b , cho một đoạn $[l, r]$, tính

$$\max_{l \leq i \leq r} \text{LCP}(\text{suf}[i], b).$$

Cách giải xem ở Ví dụ 6. Tiền xử lý $O(m \log m)$, mỗi truy vấn $O(1)$. Nếu còn cần đếm số vị trí đạt cực đại, mỗi truy vấn mất $O(\log m)$.

5.6.2 Chuyển đổi mô hình

Trước hết đơn giản hóa bài toán khớp văn bản động với mẫu tĩnh thành bài toán cơ sở: cho xâu văn bản a và xâu mẫu b , sau mỗi lần sửa a , cần khớp a với b . Thao tác sửa có thể là đổi một ký tự, chèn một ký tự hoặc xóa một ký tự trong a .

Chuyển bài toán khớp a với b thành tính hàm sau. Định nghĩa

$$F(a, b) = (x, y),$$

trong đó

$$x = \max_{1 \leq i \leq |a|} \text{LCP}(a[i : |a|], b),$$

còn y là số vị trí i đạt giá trị lớn nhất đó. Bài toán trở thành: khi a thay đổi động và b tĩnh, duy trì $F(a, b)$. Trong phần sau, a là xâu văn bản và b là xâu mẫu.

5.6.3 Phủ

Mục này đưa ra khái niệm *phủ*. Giả sử xâu mẫu b có độ dài m , xâu văn bản a có độ dài n . Ý tưởng cơ bản là dùng các xâu con của b để phủ chính xác a .

Định nghĩa 6.1. Một phủ S của a theo b là một dãy xâu

$$[v_1, v_2, \dots, v_t],$$

trong đó mỗi v_i là một phần tử phủ, thỏa:

$$v_1 v_2 \dots v_t = a,$$

và với mọi $1 \leq i \leq t$:

- *tính chất xâu con*: v_i là xâu con của b ;
- *tính chất cực đại*: nếu $i < t$, thì $v_i v_{i+1}$ không phải xâu con của b .

Đặc biệt, nếu một ký tự của a không xuất hiện trong b , ta cũng xem ký tự đó là một v_i , nhưng trong b nó tương ứng với xâu rỗng.

Một xâu có thể có nhiều cách phủ hợp lệ; khi cài đặt chỉ cần chọn một cách bất kỳ. Có thể hiểu đơn giản là dãy v_i chia a thành các khối nhỏ. Với mỗi v_i , dùng bộ ba (s_i, l_i, r_i) để biểu diễn:

- nếu v_i tương ứng với xâu con của b , thì $v_i = b[l_i : r_i]$;
- nếu v_i là ký tự không xuất hiện trong b , đặt $l_i = r_i = 0$;
- s_i là vị trí của v_i trong a , tức $s_i = 1 + \sum_{j < i} |v_j|$.

Khởi tạo phủ S như sau:

1. Tạo mảng c theo bảng chữ cái. Với ký tự i , nếu nó xuất hiện ở vị trí x trong b , đặt $c_i = x$, nếu không đặt $c_i = 0$.
2. Duyệt từng ký tự a_i . Nếu $c_{a_i} = 0$, ký tự đó không xuất hiện trong b ; nếu không, tạo $v_i = (i, c_{a_i}, c_{a_i})$.
3. Cách dựng này chưa thỏa tính chất cực đại. Duyệt theo thứ tự tăng của s_i , lần lượt kiểm tra $v_i v_{i+1}$ có phải xâu con của b hay không; nếu có thì gộp, nếu không thì giữ nguyên. Phép kiểm tra dùng bài toán con 2.

Sau khi có phủ ban đầu, xét cách duy trì động. Dưới đây chỉ mô tả thao tác đổi một ký tự; chèn và xóa tương tự. Khi đổi vị trí x thành ký tự y :

1. Tìm phần tử v_i đang phủ vị trí x . Vì phủ là chính xác, phần tử này là duy nhất.
2. Tách v_i thành ba phần trái, giữa, phải:

$$v_l = (s_i, l_i, l_i + x - s_i - 1),$$

$$v_m = (x, 0, 0),$$

$$v_r = (x + 1, l_i + x + 1 - s_i, r_i).$$

Phần giữa đúng là vị trí x . Theo ký tự mới, đặt nó thành phần tử tương ứng với y , rồi xóa các phần có độ dài 0 trong a .

3. Do tính chất cực đại, sau khi tách chỉ các phần kề v_l và v_r mới có khả năng thay đổi; đưa chúng vào danh sách cần xét.
4. Lúc này chỉ có hằng số phần tử có thể chưa cực đại. Duyệt theo thứ tự tăng của s_i , dùng bài toán con 2 để kiểm tra hai phần kề nhau có gộp được không, từ đó tái dựng các phần tử cho thỏa tính chất cực đại.

Ví dụ 10. Cho

$$a = \text{cbabbabbbacb}, \quad b = \text{abbacb}.$$

Phủ ban đầu là

$$[\text{cb}, \text{abba}, \text{bb}, \text{bacb}].$$

Nếu đổi ký tự thứ 4 của a thành c , phần tử phủ vị trí này là abba . Tách nó thành a , b , ba , đổi phần giữa thành c , rồi gộp được a với c . Phủ mới là

$$[\text{cb}, \text{ac}, \text{ba}, \text{bb}, \text{bacb}],$$

thỏa định nghĩa.

Phân tích độ phức tạp. Bài toán con 2 cần tiền xử lý $O(m \log m)$, mỗi truy vấn $O(\log m)$. Khi khởi tạo, ta thực hiện $O(n)$ lần kiểm tra kiểu bài toán con 2, nên tốn $O(n \log m)$. Với q lần sửa, mỗi lần duy trì phủ chỉ cần hằng số lần kiểm tra, tốn $O(q \log m)$.

Cần một cấu trúc dữ liệu hỗ trợ chèn, xóa, tìm tiền nhiệm, hậu nhiệm và tìm vị trí đầu tiên có chỉ số không nhỏ hơn một giá trị cho trước. Nếu dùng cây tìm kiếm nhị phân để duy trì phủ S , mỗi thao tác tốn $O(\log n)$. Vì mỗi lần sửa chỉ thực hiện hằng số thao tác trên cây, thời gian duy trì phủ là

$$O(q \log n + q \log m).$$

Kết hợp lại, bài toán duy trì phủ giải được trong

$$O((n + m + q) \log m + q \log n).$$

Nếu ghi trực tiếp s_i làm chỉ số, có thể dùng cây đoạn thay cây tìm kiếm nhị phân; độ phức tạp không đổi nhưng hằng số nhỏ hơn. Tốt hơn nữa, dùng van Emde Boas tree thay cây tìm kiếm nhị phân thì thao tác trên cấu trúc thứ tự có thể giảm xuống $O(\log \log n)$.

5.6.4 Kỹ thuật tính toán

Mục trước đã cho cách duy trì phủ S hiệu quả. Mục này đơn giản hóa bài toán để giới thiệu kỹ thuật tính $F(a, b)$ dựa trên Border Tree và phủ S .

Bài toán đơn giản hóa. Bản chất của $F(a, b)$ là biểu diễn giá trị lớn nhất của

$$\text{LCP}(a[i : |a|], b) \quad (1 \leq i \leq |a|)$$

và số vị trí đạt giá trị đó thành một cặp. Phủ S chia a thành các khối $v_1, \dots, v_t$, trong đó $v_i = b[l_i : r_i]$ nếu nó xuất hiện trong b . Với mỗi khối, đặt w_i là cặp gồm giá trị lớn nhất và số lần đạt giá trị lớn nhất của

$$\text{LCP}(v_i[j : |v_i|] \cdot v_{i+1} \cdots v_t, b)$$

trên các vị trí j trong v_i . Khi đã biết các w_i , ta gộp chúng để thu được $F(a, b)$.

Do tính chất cực đại của phủ, giá trị w_i không đổi nếu phần đuôi $v_{i+1} \cdots v_t$ được rút gọn theo cùng thông tin biên cần thiết. Mục trước đã cho cách tìm tiền nhiệm và hậu nhiệm của một phần tử phủ trong $O(\log n)$, nên mỗi lần cần tính w_i , ta chỉ mất $O(\log n)$ để tìm các phần tử kề.

Vì vậy bài toán tính $F(a, b)$ được rút về tính từng w_i . Với w_i , ta xét xâu

$$s = s_1 s_2 s_3,$$

trong đó s_1 là một xâu con của b , còn $s_2 s_3$ là phần được ghép từ các phần tử kế tiếp. Cần gộp giá trị lớn nhất và số lần đạt của

$$\text{LCP}(s[i : |s|], b) \quad (1 \leq i \leq |s_1|).$$

Bài toán mới. Đặt $s = s_1 s'$ và $s_1 = b[L : R]$. Cần tính giá trị lớn nhất Max và số lần đạt Num của

$$\text{LCP}(s[i : |s|], b) \quad (1 \leq i \leq |s_1|).$$

Với mỗi i , đặt

$$j = i + \text{LCP}(s[i : |s|], b) - 1.$$

Khi đó $s[i : j]$ là một tiền tố của b . Nếu s_1 chỉ là một ký tự và ký tự đó không xuất hiện trong b , thì $\text{Max} = 0, \text{Num} = 1$. Trường hợp còn lại chia theo $j < |s_1|$ và $j \geq |s_1|$.

Trường hợp 1: $j < |s_1|$. Khi $j < |s_1|$, bài toán chuyển thành

$$\max \text{LCP}(s_1[i : |s_1|], b).$$

Vì $s_1 = b[L : R]$, các hậu tố $s_1[i : |s_1|]$ tương ứng với đoạn con trong b . Nếu không có ràng buộc $j < |s_1|$, ta có thể dùng trực tiếp bài toán con 3. Ràng buộc này yêu cầu vị trí khớp xa nhất trong b phải nằm nghiêm ngặt trước R .

Cách đơn giản là khi tính bài toán con 3, chỉ xét các hậu tố có vị trí khớp xa nhất trong b nhỏ hơn R . Xét mọi Border của $b[L : R]$, tìm x lớn nhất sao cho $b[1 : x]$ là Border của $b[L : R]$ và

$$0 < x < R - L + 1.$$

Khi đó chỉ cần làm bài toán con 3 trên đoạn $[L, R - x]$; đoạn còn lại $[R - x + 1, R]$ cho đáp án nhỏ hơn x nên có thể bỏ qua.

Để tối đa hóa x , có thể xây KMP automaton rồi từ đỉnh R nhảy lên gốc bằng nhân đôi. Cũng có thể dùng Border Tree: vì chiều sâu Border Tree là $O(\log m)$, duyệt trực tiếp các Border Group trên đường lên gốc vẫn bảo đảm độ phức tạp.

Ví dụ 11. Cho

$$a = \text{babaca}, \quad b = \text{abcabacabc}.$$

Phủ hiện tại là

$$[b, \text{abaca}],$$

trong đó $v_1 = b$, $v_2 = \text{abaca}$, và v_2 tương ứng với bộ ba $(2, 4, 8)$ trong b . Khi cần tính w_2 , ta có $\text{LCP}(v_2, b) = 2$, tiền tố tương ứng là ab , thuộc trường hợp 1.

Xét đoạn $[4, 8]$ trong b . Nếu trực tiếp làm bài toán con 3 trên $[4, 8]$, đáp án sai sẽ là 3, ứng với tiền tố abc và đoạn khớp $[8, 10]$. Cần tìm x như trên; ở ví dụ này $x = 1$. Làm bài toán con 3 trên $[4, 8 - x]$ thu được đáp án 2, tiền tố ab , đoạn khớp $[3, 5]$, vị trí xa nhất nhỏ hơn 8. Vì vậy với w_2 ,

$$\text{Max} = 2, \quad \text{Num} = 1.$$

Trường hợp 2: $j \geq |s_1|$. Vì s_1 là xâu con của b , tìm một đoạn $[L, R]$ sao cho

$$s_1 = b[L : R].$$

Nếu $j \geq |s_1|$, thì phần khớp vượt qua cuối s_1 , nên

$$s_1[i : |s_1|]$$

phải là một Border của $b[L : R]$. Một cách vét cạn là xây KMP automaton, duyệt mọi Border của $b[L : R]$. Với mỗi Border có độ dài hợp lệ, ghép Border đó với phần s' ở sau để được xâu mới rồi tính LCP với b . Vì xâu mới được ghép từ ba xâu con của b , chỉ cần dùng hằng số lần bài toán con 1 để tính được giá trị; khi đó $\text{Num} = 1$.

Nhược điểm là chiều sâu KMP automaton có thể đạt $O(m)$, nên vét cạn mỗi lần có thể tốn $O(m)$. Dùng Border Tree để tối ưu: nếu xử lý được cả một Border Group trong $O(1)$, độ phức tạp mỗi lần chỉ còn $O(\log m)$.

Xét một Border Group có dạng

$$[x^k r, x^{k-1} r, \dots, x r].$$

Bản chất của chuyển đổi là thay vì xét từng Border trong KMP automaton, ta xét cả nhóm Border có cùng tính chất. Nếu Border Group đặc biệt có r rỗng, ta xem $x^0 r$ là xâu rỗng và xử lý riêng Border rỗng; phần còn lại của nhóm vẫn có cùng dạng.

Gọi phần sau s' là xâu cần ghép sau Border. Có hai loại chính:

1. Nếu s' là xâu tuần hoàn có độ dài chu kỳ $|x|$, đặt

$$\text{len} = \text{LCP}(s', b[|x| + 1 : |b|]).$$

Với một Border trong nhóm, độ dài khớp nhận được có dạng

$$\min\{|b|, |x^t r| + \text{len}\}.$$

Trực giác là khi LCP còn nằm trong phần tuần hoàn của b , các Border trong cùng nhóm chỉ khác số lần lặp x , còn phần ghép sau đóng góp cùng một len . Nếu độ dài khớp lớn nhất đạt $|b|$, thì $\text{Max} = |b|$ và Num là số giá trị t hợp lệ; nếu không, Max là độ dài khớp lớn nhất và $\text{Num} = 1$.

2. Nếu không thuộc loại trên, tồn tại vị trí sai khác đầu tiên q . Từ thông tin Border Group và một số phép tính LCP, ta xác định được Border nào cho khớp dài nhất. Việc phân tích chia thành các trường hợp theo vị trí của q trong khối tuần hoàn; mỗi trường hợp hoặc trả lời trực tiếp, hoặc rút về loại 1 trên một đoạn ngắn hơn.

Ví dụ 12. Cho

$$b = ababababa, \quad s_1 = ababab, \quad s_2 = ab, \quad s_3 = ba.$$

Khi đó $s' = abba$. Vì s_1 là xâu con của b , có thể chọn $[L, R] = [3, 8]$. Trong KMP automaton, các Border của $b[1 : 8]$ có độ dài hợp lệ là

$$ababab, \quad abab, \quad ab.$$

Ghép chúng với s' lần lượt cho các độ dài khớp 8, 6, 4, nên $\text{Max} = 8, \text{Num} = 1$.

Trong Border Tree, mọi Border của $b[1 : 8]$ thuộc cùng một Border Group, có bộ ba $(0, 2, 4)$. Vì b là xâu tuần hoàn với chu kỳ 2, đây là loại 1. Nhóm gồm ba Border $ababab, abab, ab$. Từ Border ab , tính vết cận được độ dài khớp cơ sở và $\text{len} = 2$; công thức trên cho ngay độ dài 8, 6 của hai Border còn lại.

Ví dụ 13. Cho

$$b = ababababacde, \quad s_1 = ababab, \quad s_2 = ababac, \quad s_3 = de.$$

Khi đó $s' = ababacde$, và có thể chọn $[L, R] = [3, 8]$. Trong Border Tree, mọi Border của $b[1 : 8]$ cũng thuộc Border Group có bộ ba $(0, 2, 4)$, nhưng b không còn là xâu tuần hoàn hoàn toàn, nên thuộc loại 2. Nhóm có ba Border:

$$ababab, \quad abab, \quad ab.$$

Tính theo các đại lượng trong phân tích loại 2 cho thấy Border $abab$ cho giá trị tốt nhất, vì vậy

$$\text{Max} = 12, \quad \text{Num} = 1.$$

Phân tích độ phức tạp. Theo phân tích của hai trường hợp trên, nếu dùng Border Tree, số Border Group cần duyệt là $O(\log m)$. Mỗi Border Group được xử lý bằng cách phân loại và chuyển đổi cụ thể, về mặt độ phức tạp tương đương hằng số lần gọi bài toán con 1. Vì bài toán con 1 trả lời $O(1)$, tính một w_i tốn $O(\log m)$.

Kết hợp với phân tích duy trì phủ, bài toán khớp văn bản động a với mẫu tĩnh b , tức duy trì $F(a, b)$, giải được trong

$$O((n + m + q) \log m + q \log n).$$

5.6.5 Ví dụ

Ví dụ 14 (kiểm tra lẫn nhau đội tuyển 2015, Sone2). Cho xâu chính a độ dài n và xâu mẫu b độ dài m . Có q thao tác thuộc bốn loại:

1. sửa một vị trí của a , rồi hỏi $F(a, b)$;
2. chọn một đoạn $[L, R]$ của a , hỏi $F(a[L : R], b)$;
3. bài toán con 1;
4. bài toán con 2.

Dữ liệu:

$$1 < n, m, q, \Sigma < 100000.$$

Giới hạn thời gian là 2 giây.

Trong bài này, ngoài thao tác loại 2, các phép tính khác đã được nêu ở trên. Với loại 2, ý tưởng đơn giản là duy trì phủ S' của xâu $a[L : R]$ rồi tính trên đó. Vì $a[L : R]$ là xâu con của a , bỏ hai phần tử đầu

và cuối có thể bị cắt ra thì phần còn lại của S' là một dãy con liên tiếp của phủ S . Do đó với loại 2 chỉ cần xử lý riêng hai phần tử đầu cuối, còn phần giữa lấy trực tiếp từ S bằng cấu trúc dữ liệu. Nhờ vậy có thể dựng S' trong

$$O(\log n + \log m)$$

cho mỗi truy vấn. Nếu đồng thời duy trì các giá trị w_i , có thể tính ngay đáp án từ S' . Tổng độ phức tạp vẫn là

$$O((n + m + q) \log m + q \log n).$$

Bài này còn có một phần điểm trong đó xâu mẫu là ngẫu nhiên. Khi b ngẫu nhiên, chiều sâu của KMP automaton dựng trên b là $O(\log m)$. Vì thế ở dữ liệu này không cần dùng Border Tree; có thể dùng trực tiếp KMP automaton thay thế. Độ phức tạp tiệm cận không giảm, nhưng độ khó cài đặt giảm đáng kể.

Có thể tăng cường bài toán như sau. Cho một cây n đỉnh, mỗi đỉnh có một ký tự. Với một đường đi trên cây, ghép ký tự các đỉnh theo thứ tự để được xâu a . Cho thêm xâu mẫu b độ dài m . Mỗi thao tác đổi ký tự của một đỉnh trên cây và hỏi $F(a, b)$ cho một đường đi. Tăng cường hơn nữa, cây có thể trở thành cây động.

Cách giải là dùng Link-Cut Tree để duy trì phủ S . Trong mỗi splay, duy trì phủ S' của xâu tạo bởi heavy path tương ứng. Mỗi thao tác access ghép hoặc tách các splay, chỉ làm tái dựng hằng số phần tử v_i và w_i . Vì vậy bài toán trên cây có thể giải trong

$$O((n + m) \log m + q \log n \log m).$$

5.7 Lời cảm ơn

Xin cảm ơn Hiệp hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy Chen Heli, Dong Yehua, You Guanghui và Shao Hongxiang của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và hướng dẫn tôi.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yu Linyun và Chen Xuji đã chỉ dẫn.

Xin cảm ơn các anh Yu Yili, Dong Anhua và He Qizheng của Đại học Thanh Hoa đã giúp đỡ tôi.

Xin cảm ơn các bạn Zhang Hengjie, Ren Zhizhou và Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn các bạn Yang Jiacheng, Guo Yu, Shao Jiechang, Weng Tiandong, Tao Yuanzheng, Hong Huadun và các bạn khác ở Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn bạn Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã đọc duyệt bài viết này.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi ý cho tôi.

Xin cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo của bài viết

- [1] Wikipedia, Suffix array.
- [2] Zhu Zeyuan, *Thuật toán khớp nhiều xâu và những gợi mở của nó*.
- [3] Yang Yu, *Một lớp ứng dụng của hashing trong thi tin học*.
- [4] Luo Suiqian, *Suffix array: công cụ mạnh để xử lý chuỗi*.
- [5] Dong Huaxing, *Bàn về ứng dụng trie trong thi tin học*.
- [6] Chen Lijie, *Suffix Automaton*.

- [7] Wang Yuetong, Xu Duo and Xu Zihan, *Suffix Cactus*.
- [8] Amihood Amir, Gad M. Landau, Moshe Lewenstein and Dina Sokol, “Dynamic Text and Static Pattern Matching”.
- [9] Mokhtar S. Bazaraa, John J. Jarvis and Hanif D. Sherali, *Linear Programming and Network Flows*, 4th ed., John Wiley & Sons.
- [10] Liu Rujia and Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Tsinghua University Press.
- [11] Liu Rujia, *Nhập môn kinh điển về thi thuật toán*, Tsinghua University Press.

6 Suffix automaton và ứng dụng

Zhang Tianyang

Trường Trung học số 1 Trường Sa, Hồ Nam

Tóm tắt

Suffix automaton là một công cụ xử lý xâu mới nổi trong OI, ngày càng được nhiều người ra đề ưa chuộng. Bài viết này nhằm tổng kết các tính chất và một số ứng dụng của suffix automaton, đồng thời thông qua một số ví dụ để nghiên cứu sâu hơn các tính chất của nó.

6.1 Định nghĩa và xây dựng suffix automaton

6.1.1 Định nghĩa suffix automaton

Suffix automaton (SAM) của một xâu S là một automaton hữu hạn xác định (DFA); nó có thể, và chỉ có thể, chấp nhận mọi hậu tố của S , đồng thời có số trạng thái và số chuyển nhỏ nhất.

Ta sẽ chứng minh bên dưới rằng số trạng thái và số chuyển của suffix automaton của một xâu S đều là $O(|S|)$.

6.1.2 Cách xây dựng suffix automaton

Một vài định nghĩa. Gọi xâu mẹ của suffix automaton là S . Ký hiệu $S[l, r]$ là xâu con gồm các ký tự từ vị trí l đến vị trí r trong S . Ký hiệu hậu tố bắt đầu tại vị trí i là suf_i , và tiền tố kết thúc tại vị trí i là pre_i .

Ký hiệu SAM_{str} là trạng thái (hay nút) nhận được sau khi bắt đầu từ trạng thái đầu và đọc xâu str .

Với một xâu con str của S , ký hiệu right_{str} là tập các vị trí đầu mút phải của mọi lần xuất hiện của str trong S .

Nếu không nói khác, ta giả sử bảng chữ cái gồm mọi chữ cái thường. Kích thước bảng chữ cái được xem là hằng số và không tính vào độ phức tạp.

Chứng minh độ phức tạp. Xét một xâu str . Nếu str là một xâu con của S , thì SAM_{str} phải là một trạng thái hợp lệ. Lý do là ta có thể thêm một ký tự vào sau str để biến nó thành một hậu tố của S ; hậu tố này phải là một trạng thái chấp nhận được, nên SAM_{str} cũng phải là một trạng thái hợp lệ. Ngược lại, nếu str không phải xâu con của S , thì SAM_{str} phải là trạng thái không hợp lệ (hay trạng thái rỗng), vì dù thêm ký tự thế nào nó cũng không thể trở thành một hậu tố của S .

Nói cách khác, với mỗi xâu con của S , trong suffix automaton đều phải có một trạng thái tương ứng. Tuy nhiên rõ ràng ta không thể dựng riêng một trạng thái cho từng xâu con, vì số xâu con là $O(|S|^2)$.

Xét một xâu con str của S . Nếu $right_{str}$ và $right_{a+str}$ (trong đó a là một xâu) hoàn toàn giống nhau, thì ta có thể gộp chúng vào cùng một trạng thái. Từ hai xâu này, các ký tự có thể thêm vào để thu được hậu tố là như nhau (hay nói khác đi, ngoài phần xâu a ở phía trước thì giống nhau). Tương đương, từ các trạng thái của chúng, các trạng thái chấp nhận có thể chuyển tới cũng giống nhau. Do đó hai xâu này có bản chất như nhau trong suffix automaton và có thể gộp vào một trạng thái.

Tiếp theo xét hai xâu con A, B của S . Nếu $right_A$ và $right_B$ giao nhau, thì hiển nhiên một xâu là hậu tố của xâu còn lại; giả sử A là hậu tố của B , khi đó dễ thấy

$$right_A \supset right_B.$$

Nghĩa là, với hai xâu con bất kỳ của S , hai tập $right$ của chúng hoặc rời nhau, hoặc một tập chứa tập kia. Ta định nghĩa trạng thái cha của một trạng thái s , ký hiệu fa_s , là trạng thái thỏa

$$right_s \subset right_{fa_s}$$

và $|right_{fa_s}|$ nhỏ nhất. Khi đó mọi trạng thái tạo thành một cấu trúc cây, gọi là cây parent.

Rõ ràng tập $right$ của trạng thái đầu chứa mọi số nguyên từ 1 đến $|S|$. Với mỗi nút lá, kích thước tập $right$ của nó là 1. Do đó số nút lá là $O(|S|)$. Mỗi nút không phải lá có ít nhất hai con; nếu chỉ có một con thì có thể gộp nó với con đó. Vì thế số nút không phải lá nhiều nhất cũng là $O(|S|)$, và số trạng thái của suffix automaton là $O(|S|)$.

Xét các chuyển. Vì số trạng thái là $O(|S|)$, ta xét một đồ thị cây của automaton bắt đầu từ trạng thái đầu; số cạnh trên cây hiển nhiên là $O(|S|)$. Vậy chỉ cần xét các cạnh không thuộc cây.

Xét một cạnh không thuộc cây $u \rightarrow v$. Từ trạng thái đầu, đi theo cạnh cây tới u , rồi đi qua cạnh không thuộc cây này tới v , sau đó tiếp tục đi xuống thì chắc chắn tới được ít nhất một hậu tố. Nói cách khác, nếu mỗi hậu tố phủ cạnh không thuộc cây đầu tiên mà nó đi qua, thì mọi cạnh không thuộc cây đều được phủ. Vì vậy số cạnh không thuộc cây không vượt quá số hậu tố. Ta đã chứng minh được số chuyển của suffix automaton là $O(|S|)$.

Phương pháp xây dựng. Suffix automaton có một cách xây dựng rất đơn giản: phương pháp tăng dần.

Với mỗi trạng thái s , ghi len_s là độ dài xâu con dài nhất mà nó đại diện.

Giả sử hiện tại ta đã có suffix automaton của $|S| - 1$ ký tự đầu, và trong automaton hiện tại, xâu $S[1, |S| - 1]$ nằm ở trạng thái $last$. Bây giờ thêm ký tự thứ $|S|$, giả sử là c . Ta tạo trạng thái mới np ; rõ ràng $len_{np} = |S|$.

Sau đó xét các chuyển. Hiển nhiên ta cần thêm một chuyển $last \rightarrow np$, và cũng cần thêm một chuyển $fa_{last} \rightarrow np$, cứ như vậy cho đến khi gặp một trạng thái đã có chuyển bằng ký tự c . Giả sử trạng thái này là p , và trạng thái sau khi đi bằng c là q .

Lúc này $right_q$ sẽ có thêm một xâu: xâu có đầu mút phải là $|S|$ và độ dài dài nhất là $len_p + 1$. Có hai trường hợp:

1. $len_q = len_p + 1$. Khi đó mọi xâu do q đại diện vẫn có cùng tập $right$, nên chỉ cần đặt $fa_{np} = q$.
2. $len_q > len_p + 1$. Trong các xâu do q đại diện, các xâu có độ dài không quá $len_p + 1$ sẽ có thêm giá trị $|S|$ trong tập $right$, còn các xâu dài hơn thì không. Để duy trì tính chất mọi xâu trong một trạng thái có cùng tập $right$, ta tạo một trạng thái mới nq . Trạng thái nq đại diện cho mọi xâu do q đại diện trước đây có độ dài không quá $len_p + 1$, nên $len_{nq} = len_p + 1$; các thuộc tính khác của nq (fa và các chuyển) giống hệt q . Đồng thời dễ thấy $fa_q = fa_{np} = nq$.

Sau đó ta lại bắt đầu từ p . Trước đây chuyển bằng ký tự c của p tới q ; bây giờ nó phải chuyển tới nq . Tương tự, nếu chuyển bằng ký tự c của fa_p tới q , thì nó cũng phải chuyển tới nq , cứ như vậy cho đến khi gặp một điểm mà chuyển bằng ký tự c không tới q .

Đến đây, ta hoàn thành việc xây dựng suffix automaton.

6.1.3 Một vài tính chất cơ bản của suffix automaton

Các tính chất cơ bản đã được nhắc tới ở trên, tổng kết như sau:

Tính chất 1. Các độ dài của những xâu mà mỗi trạng thái s đại diện tạo thành khoảng

$$(\text{len}_{\text{fa}_s}, \text{len}_s].$$

Tính chất 2. Mọi xâu mà một trạng thái s đại diện có cùng số lần xuất hiện trong xâu gốc và cùng các vị trí đầu mút phải của mỗi lần xuất hiện.

Tính chất 3. Trong cây parent, tập right của mỗi trạng thái là tập con của tập right của trạng thái cha.

6.1.4 Cách tính tập right

Trước hết, thêm i vào tập right của trạng thái chứa pre_i .

Sau đó, theo thứ tự độ sâu giảm dần trong cây parent, lần lượt gộp tập right của mỗi trạng thái vào tập right của trạng thái cha.

Như vậy có thể tính được tập right của mỗi trạng thái, nhưng làm trực tiếp tốn $O(|S|^2)$.

Tùy bài toán cụ thể, ta có thể chỉ cần kích thước, giá trị lớn nhất, hay một số đại lượng khác của tập right; các đại lượng này có thể tính trong $O(|S|)$. Nếu thật sự cần tập right của từng trạng thái, có thể dùng cây cân bằng để duy trì, khi gộp dùng heuristic merging,² đạt độ phức tạp $O(|S| \log |S|)$. Nếu cần online, có thể dùng cây cân bằng persistent.

6.1.5 Dùng suffix automaton để tìm suffix tree và suffix array

Ngoài suffix automaton, suffix tree và suffix array cũng là các cấu trúc dữ liệu hậu tố thường dùng. Tuy nhiên việc xây dựng chúng thường không tiện: xây suffix tree tuyến tính có thể nói là khá rắc rối, còn xây suffix array tuyến tính có hằng số tương đối lớn. Vì suffix automaton có hằng số nhỏ và cách xây dựng khá ngắn gọn, ta có thể dùng nó để tìm suffix tree và suffix array.

Tính chất 4. Cây parent của suffix automaton chính là cây tiền tố ngược của xâu gốc.³

Trước hết, cây tiền tố ngược được định nghĩa như sau: đưa xâu đảo của mỗi tiền tố vào một trie, rồi gộp các dây không phân nhánh giống như trong suffix tree.

Tính chất này dễ nhận ra. Xét trạng thái của một tiền tố trên suffix automaton. Ta liên tục đi theo con trở fa; mỗi lần, xâu hiện tại trở thành một hậu tố của xâu trước đó, cho tới khi thành xâu rỗng. Nếu nhìn ngược quá trình này, đó chính là đi từ trên xuống dưới trong cây tiền tố ngược.

Vậy nếu ta xây suffix automaton của xâu đảo của xâu ban đầu, thì cây parent của nó chính là suffix tree của xâu ban đầu.

Để khôi phục suffix tree, ta có thể tìm một giá trị bất kỳ trong tập right của mỗi trạng thái trên suffix automaton, chẳng hạn giá trị lớn nhất. Sau đó dễ dàng xác định xâu nằm trên cạnh nối mỗi nút với cha của nó trong suffix tree.

²Vấn đề heuristic merging trên cây cân bằng không liên quan trực tiếp tới bài viết này nên không bàn thêm; độc giả quan tâm có thể xem lời giải bài *Cây non và tập hợp*.

³Thực chất chính là suffix tree của xâu đảo.

Cuối cùng, chỉ cần DFS suffix tree một lần là có thể thu được suffix array.

6.2 Ứng dụng của suffix automaton

6.2.1 Một cách diễn giải

Khi nói “đưa một xâu chạy trên suffix automaton”, ta có thể liên tưởng tới cách chạy KMP và AC automaton: bắt đầu từ trạng thái đầu; nếu tồn tại chuyển tương ứng thì đi theo chuyển đó, nếu không thì nhảy ngược theo con trỏ fa, cho tới khi tồn tại chuyển tương ứng hoặc đã nhảy ra khỏi suffix automaton.

6.2.2 Một vài bài toán kinh điển

Trước hết ta thảo luận lời giải của một vài bài toán kinh điển.

Xâu con chung dài nhất. Ta xây suffix automaton cho một trong hai xâu. Đưa xâu còn lại chạy trên suffix automaton và luôn duy trì độ dài xâu con hiện tại.

Với xâu con chung dài nhất của nhiều xâu (nhiều hơn 2), ta xây suffix automaton cho một xâu, rồi lần lượt đưa các xâu còn lại chạy trên automaton. Với mỗi trạng thái, duy trì độ dài khớp dài nhất của trạng thái đó đối với từng xâu. Sau đó, với mỗi trạng thái lấy min của các độ dài khớp dài nhất trên mọi xâu, rồi lấy max trên mọi trạng thái.

Xâu con thứ k theo thứ tự từ điển. Ta topo một lượt suffix automaton để tính từ mỗi điểm có thể đi tới bao nhiêu xâu con. Sau đó DFS từ trạng thái đầu: nếu số xâu con mà trạng thái hiện tại có thể đi tới không nhỏ hơn k thì tiếp tục DFS; nếu không, trừ k đi số xâu con đó rồi quay lui.

Nếu cần tìm xâu con thứ k khác nhau thì không cần kích thước tập right; nếu tính cả số lần xuất hiện thì cần kích thước right của mỗi điểm.

Hậu tố nhỏ nhất theo thứ tự từ điển (biểu diễn vòng nhỏ nhất). Sao chép xâu gốc một lần và nối vào phía sau, rồi xây suffix automaton.

Bắt đầu từ trạng thái đầu, mỗi lần đi theo chuyển nhỏ nhất theo thứ tự từ điển; sau khi đi $|S|$ lần, xâu nhận được chính là biểu diễn vòng nhỏ nhất.

Nếu cần hậu tố nhỏ nhất, ta thêm vào sau xâu gốc một ký tự kết thúc nhỏ hơn mọi ký tự trong bảng chữ cái, rồi sao chép xâu một lần nữa là được.

Tiếp theo ta dùng một số ví dụ đơn giản để thảo luận các ứng dụng cơ bản của suffix automaton.

Ví dụ 1: Strings. Nguồn bài: Adera 1 Cup, bài mô phỏng trại đông.

Tóm tắt đề. Cho n xâu, hỏi với mỗi xâu có bao nhiêu xâu con không rỗng là xâu con của ít nhất k trong n xâu.

$$1 \leq n, k \leq 10^5, \quad \sum |S_i| \leq 10^5.$$

Phân tích. Trước hết nối n xâu lại với nhau, giữa các xâu dùng một ký tự không nằm trong bảng chữ cái gốc để phân tách. Sau đó xây suffix automaton của xâu nối này. Với mỗi nút trong suffix automaton, ta cần tính nó là xâu con của bao nhiêu trong n xâu ban đầu.

Ý tưởng cơ bản là đưa từng xâu chạy trên suffix automaton. Với mỗi nút đi tới, tăng số lần xuất hiện của mọi nút trên đường đi theo con trỏ fa lên gốc thêm 1. Nhưng làm vậy sẽ gây trùng lặp. Ta có thể ghi lại ở mỗi trạng thái xâu cuối cùng đã đi tới nó; khi tới một điểm, chỉ cần đi ngược lên tới điểm đầu

tiên mà xuất hiện tại đã từng đi tới là đủ. Nếu làm thô bạo, độ phức tạp là $O(n\sqrt{n})$.⁴ Dùng heavy-light decomposition có thể đạt $O(n \log^2 n)$.

Sau đó chỉ cần quy hoạch một lượt để tính trên đường từ mỗi nút theo fa lên gốc có bao nhiêu xâu xuất hiện ít nhất k lần. Cuối cùng đưa từng xâu chạy trên suffix automaton, cộng giá trị của mỗi nút mà nó đi tới là được.

Ví dụ 2: Palindrome. *Nguồn bài: APIO 2014.*

Tóm tắt đề. Cho một xâu độ dài n , tìm giá trị lớn nhất của số lần xuất hiện nhân với độ dài trên mọi xâu con palindrome của nó.

$$1 \leq n \leq 3 \times 10^5.$$

Phân tích. Xây suffix automaton của xâu gốc và tính giá trị lớn nhất rmax trong tập right của mỗi nút.

Sau đó đưa xâu đảo chạy trên suffix automaton. Nếu xâu khớp hiện tại trong xâu đảo, ký hiệu $[l, r]$, phủ rmax của nút hiện tại, thì $[l, rmax]$ là một palindrome.⁵

Chi tiết xem tài liệu tham khảo [4].

Ví dụ 3: Nhận diện xâu con. *Tóm tắt đề.* Cho một xâu độ dài n . Định nghĩa xâu con nhận diện của vị trí i là một xâu con chứa vị trí này và chỉ xuất hiện một lần trong xâu gốc. Hãy tìm độ dài xâu con nhận diện ngắn nhất cho mỗi vị trí.

$$1 \leq n \leq 10^5.$$

Phân tích. Hiển nhiên mọi xâu con nhận diện đều nằm ở những nút có kích thước tập right bằng 1 trong suffix automaton. Vì thế trước hết ta tìm tất cả các nút như vậy. Với mỗi nút có kích thước tập bằng 1, ta dễ dàng biết đầu mút phải duy nhất của lần xuất hiện.

Nếu một xâu xuất hiện đúng một lần, thì mở rộng nó sang trái thêm một đoạn vẫn xuất hiện đúng một lần. Do đó, với mỗi nút có kích thước tập right bằng 1, ta tính đầu mút phải này cần mở rộng sang trái ít nhất bao xa để vẫn có số lần xuất hiện bằng 1. Khi đó mỗi tập right tương ứng với một đoạn và một giá trị lấy min, cùng với một tiền tố và một cấp số cộng có công sai -1 lấy min. Việc này có thể thực hiện bằng cách sắp xếp rồi mô phỏng đơn giản.

Độ phức tạp là $O(n \log n)$.

Ví dụ 4: Difference. *Nguồn bài: AHOI 2013.*

Tóm tắt đề. Cho một xâu độ dài n , tính tổng độ dài LCP của mọi cặp hậu tố của nó.

$$1 \leq n \leq 5 \times 10^5.$$

Phân tích. Trước hết đảo xâu này lại. Khi đó bài toán trở thành tính tổng độ dài hậu tố chung dài nhất của mọi cặp tiền tố.⁶

Tính chất 5. Hậu tố chung dài nhất của hai xâu nằm ở trạng thái LCA của hai trạng thái tương ứng với hai xâu đó trên cây parent.

Lý do là mọi tổ tiên của trạng thái tương ứng với một xâu đều là hậu tố của nó, và độ sâu càng lớn thì độ dài càng lớn.

⁴Phải có xâu rất đặc biệt mới đạt tới độ phức tạp này, và hằng số rất nhỏ.

⁵Ở đây chỉ vị trí trong xâu gốc.

⁶Ở đây đã trình bày đề bài sau khi biến đổi đơn giản từ đề gốc.

Vì vậy, ta tô đen các nút chứa từng tiền tố. Bài toán trở thành: mỗi nút là LCA của bao nhiêu cặp nút đen. Đây là một bài toán đơn giản; chỉ cần quy hoạch từ dưới lên trên trong cây parent.

Suffix automaton cũng có thể kết hợp với các cấu trúc dữ liệu và công cụ xử lý xâu khác để giải một số bài toán khó hơn.

Ví dụ 5: Jeweler. Nguồn bài: CTSC 2010.

Tóm tắt đề. Cho một cây gồm n đỉnh, mỗi đỉnh có một ký tự. Cho thêm một xâu S độ dài m . Với mỗi cặp đỉnh, các đỉnh trên đường đi giữa chúng tạo thành một xâu; tính tổng số lần xuất hiện trong S của tất cả các xâu như vậy.

$$1 \leq n, m \leq 5 \times 10^4.$$

Phân tích. Xét dùng centroid decomposition. Vì n, m cùng bậc, khi tính độ phức tạp ta thống nhất dùng n .

1. Nếu kích thước khối phân rã nhỏ hơn $\sqrt{n}$, ta DFS trực tiếp mọi đường đi và thống kê số lần xuất hiện trong suffix automaton của S . Vì các khối này rời nhau, chi phí tính một khối là $O(\text{size}^2)$, nên tổng độ phức tạp nhiều nhất là $O(n\sqrt{n})$.
2. Nếu kích thước khối phân rã lớn hơn $\sqrt{n}$, xét cách tính các đường đi qua trọng tâm của khối hiện tại; các đường đi khác được xử lý đệ quy bằng phân rã.

Xét các vị trí xuất hiện trong S của ký tự c ở trọng tâm x . Nếu một đường đi có dạng $a \rightarrow x \rightarrow b$, thì tập right của đoạn $a \rightarrow x$ trong S chắc chắn là tập con của các vị trí xuất hiện của c trong S . Tương tự, nếu đảo đoạn $x \rightarrow b$, thì tập right của nó trong xâu đảo của S cũng chắc chắn là tập con của các vị trí xuất hiện của c trong S .

Vậy ta bắt đầu DFS khối phân rã từ x . Mỗi lần, duy trì vị trí của xâu hiện tại trên suffix tree của xâu gốc và của xâu đảo. Sau đó ta đẩy thông tin trên hai suffix tree một lượt từ trên xuống dưới để tính số lượng khớp của mỗi hậu tố, tức mỗi vị trí. Cuối cùng nhân số lượng khớp ở hai vị trí tương ứng trên hai suffix tree, ta thu được tổng số lần xuất hiện trong xâu gốc của các đường đi qua trọng tâm.

Chú ý cách làm này đếm thừa trường hợp a, b nằm trong cùng một cây con của trọng tâm; vì vậy với từng cây con của trọng tâm, ta dùng cùng phương pháp tính riêng một lượt rồi trừ đi.

Chi phí tính một khối phân rã là $O(\text{size} + n)$. Số khối phân rã có kích thước lớn hơn $\sqrt{n}$ nhiều nhất là $O(\sqrt{n})$,⁷ nên độ phức tạp là $O(n\sqrt{n})$.

Tóm lại, ta thu được một thuật toán tốt với độ phức tạp $O(n\sqrt{n})$.

Ví dụ 6: str. Nguồn bài: Feyat Cup 1.5; tác giả đề: Huang Zhixiang.

Tóm tắt đề. Cho hai xâu có độ dài lần lượt là n, m . Định nghĩa hai xâu là khớp nếu chúng có nhiều nhất một vị trí khác nhau. Hãy tìm xâu con chung dài nhất của hai xâu theo định nghĩa khớp này.

$$1 \leq n, m \leq 10^5.$$

Phân tích. Xét xâu cuối cùng có thể khớp được: ở giữa nó có một vị trí không khớp, còn hai phía trái và phải của vị trí đó khớp tương ứng. Nếu đã biết xâu phía trái, ta dễ biết mọi vị trí xuất hiện của nó trong hai xâu, rồi lấy LCP⁸ của các xâu phía phải tương ứng theo từng cặp là được.

Ta nối hai xâu lại, ở giữa thêm một ký tự không thuộc bảng chữ cái gốc, rồi xây suffix automaton của xâu này. Xét một nút trong suffix automaton: trong tập right của nó, một phần xuất hiện bên trong xâu

⁷Ý tưởng chứng minh: sau mỗi lần phân rã, kích thước cây con giảm ít nhất một nửa; sau k lần phân rã, khối lớn nhất không vượt quá $n/2^k$ và số khối là 2^k . Đặt khối lớn nhất không vượt quá $\sqrt{n}$ là thu được kết luận.

⁸Longest common prefix.

thứ nhất, phần còn lại xuất hiện bên trong xâu thứ hai. Ví dụ có một giá trị right là a , nằm trong xâu thứ nhất, và một giá trị right khác là b , nằm trong xâu thứ hai. Khi đó LCP của $a + 2$ và $b + 2$ chính là độ dài khớp dài nhất tương ứng. Nếu ta tính được LCP tương ứng của mọi cặp (a, b) như vậy, lấy giá trị lớn nhất cộng với len của nút hiện tại, thì đó là đáp án của nút này. Lấy lớn nhất trên mọi nút là đáp án cuối cùng.

Tuy nhiên tính LCP cho mọi cặp (a, b) chắc chắn không đủ nhanh. Ta xét thứ tự mọi hậu tố. Với một nút, nếu trộn mọi a và b rồi sắp theo thứ tự hậu tố tương ứng, thì ta chỉ cần tính LCP của các cặp kề nhau sau khi sắp xếp mà xuất hiện trong hai xâu khác nhau. Với bài toán này, ta xét các nút theo thứ tự từ dưới lên trên trên cây parent, dùng cây cân bằng để duy trì toàn bộ dãy. Khi gộp cây cân bằng, duy trì thêm thông tin cần thiết, ta đạt độ phức tạp

$$O((n + m) \log(n + m)).$$

6.3 Suffix automaton của cây chữ cái

Ta có thể mở rộng thuật toán KMP lên cây chữ cái để tạo thành AC automaton. Tương tự, ta cũng có thể mở rộng suffix automaton lên cây chữ cái.

Trước hết xem một ví dụ.

Ví dụ 7: Xâu của pty. Tác giả đề: Peng Tianyi.

Tóm tắt đề. Cho một cây có gốc gồm n nút, mỗi cạnh có một ký tự. Định nghĩa một đường đi là bắt đầu từ một nút nào đó, đi xuống tới một nút nào đó; các ký tự trên cạnh tạo thành một xâu. Hai xâu khớp nếu chúng hoàn toàn giống nhau. Cho thêm một xâu độ dài m , hỏi có bao nhiêu cặp khớp giữa xâu con của nó và đường đi trên cây.

$$1 \leq n \leq 8 \times 10^5, \quad 1 \leq m \leq 8 \times 10^6.$$

Phân tích. Cách cơ bản là xây suffix automaton cho xâu và tính kích thước tập right, rồi DFS cây chữ cái.

Nhưng trong bài này, vì xâu quá dài, xây suffix automaton cho xâu không chấp nhận được về bộ nhớ.⁹ Cần tìm hướng khác.

Xét xây suffix automaton cho cây chữ cái đã cho. Khi chèn một nút, ta dùng trạng thái của tiền tố của cha nó trong suffix automaton làm trạng thái last để chèn.¹⁰ Nhưng có một vấn đề: một nút có thể có hai con mà ký tự trên cạnh nối tới chúng giống nhau.

Cách xử lý thực ra rất đơn giản: gộp các nút con có cùng ký tự dưới cùng một nút; kích thước right ban đầu của nút sau khi gộp là số nút được gộp.

Cuối cùng, đưa xâu đã cho chạy trên suffix automaton và thống kê đáp án.

Xem thêm một ví dụ khác.

Ví dụ 8: Vùng đất huyền tường được chư thần ưu ái. Nguồn bài: ZJOI 2015; tác giả đề: Chen Lijie.

Tóm tắt đề. Cho một cây gồm n nút, mỗi nút có một ký tự. Định nghĩa một đường đi là xâu tạo bởi các ký tự trên các nút giữa hai điểm nào đó. Hỏi trên cây có bao nhiêu đường đi đôi một khác nhau.

$$1 \leq n \leq 10^5,$$

bảng chữ cái có kích thước 10, và cây có nhiều nhất 20 nút lá.

⁹Có lẽ cách lưu cạnh bằng hash vẫn dùng được.

¹⁰Tiền tố ở đây là đường đi từ gốc tới nút đó.

Phân tích. Chú ý đặc điểm của đề: số nút lá rất ít. Vì vậy ta lần lượt lấy mỗi nút lá làm gốc, rồi hợp nhất các cây thu được để tạo thành một cây chữ cái.

Bài toán tiếp theo là trên cây chữ cái này có bao nhiêu đường đi từ trên xuống dưới khác nhau. Ta xây suffix automaton cho nó; với mỗi nút, cộng

$$\text{len} - \text{len}_{\text{fa}}$$

vào đáp án.

6.4 Tổng kết

Là một công cụ xử lý xâu mới nổi trong OI, suffix automaton có ưu điểm là chức năng toàn diện, mã nguồn ngắn gọn, độ phức tạp thấp và hằng số nhỏ. Tương ứng, trong quá trình ứng dụng, nó đòi hỏi ta phải suy nghĩ và nghiên cứu nhiều hơn để hiểu được vẻ đẹp của suffix automaton.

6.5 Lời cảm ơn

- Xin cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.
- Xin cảm ơn Chen Lijie đã giới thiệu suffix automaton.
- Xin cảm ơn bạn Shi Wenbin đã dạy tôi suffix automaton.
- Xin cảm ơn các bạn Chen Xiaobo và Lu Kaifeng đã giúp đỡ trong quá trình viết luận văn.
- Xin cảm ơn thầy huấn luyện Zhou Zusong đã hỗ trợ.

Tài liệu tham khảo của bài viết

- [1] Chen Lijie, *Suffix Automaton*, trao đổi học viên trại đông 2012.
- [2] Fan Haoqiang, *Suffix automaton và xây dựng suffix tree tuyến tính*.
- [3] WRIA, *Lời giải mới cho CTSC 2010 Jeweler*, bài tập đội tuyển quốc gia 2013.
- [4] Zhang Tianyang, *Lời giải APIO 2014 Palindrome*, bài tập đội tuyển quốc gia 2015.
- [5] Chen Lijie, *Lời giải ZJOI 2015 Day 1*.
- [6] #22334, *Lời giải Feyat Cup 1.5*.

7 Phép toán trên hàm sinh và bài toán đếm tổ hợp

Jin Ce

Trường Học Quân Hàng Châu

Tóm tắt

Bài viết giới thiệu một số thuật toán hiệu quả để xử lý chuỗi lũy thừa hình thức, rồi áp dụng chúng vào các phép toán trên hàm sinh, từ đó giải một loạt bài toán đếm tổ hợp.

7.1 Dẫn nhập

Bài toán đếm tổ hợp là một lớp bài toán thường gặp trong thi tin học, và hàm sinh thường là một công cụ quan trọng để xử lý lớp bài toán này. Những năm gần đây, trong các kỳ thi xuất hiện một loại bài đếm không chỉ yêu cầu thí sinh phân tích và suy diễn hàm sinh theo đề bài, mà còn cần dùng thuật toán hiệu quả để hoàn thành nhiều phép toán đa thức khác nhau, nhờ đó tối ưu độ phức tạp thời gian. Bài viết này phân tích và tổng kết thêm về loại bài toán đó.

Mục 2 và mục 3 nhắc lại một số kiến thức cơ bản cần dùng. Trong đó mục 2 nêu một vài thuật toán nhân đa thức quen thuộc và các điểm cần chú ý khi cài đặt; mục 3 đưa vào khái niệm hàm sinh của đối tượng tổ hợp, đồng thời phân tích ý nghĩa tổ hợp của phép cộng và phép nhân hàm sinh.

Mục 4 giới thiệu phương pháp Newton để tìm nghịch đảo nhân của chuỗi lũy thừa hình thức; đây là cơ sở cho nhiều thuật toán phía sau. Các ví dụ ở mục 5 áp dụng đơn giản thuật toán này.

Mục 6 giới thiệu thuật toán tính logarit và hàm mũ của chuỗi lũy thừa hình thức. Mục 7 và mục 8 phân tích cách đếm hai mô hình tổ hợp thường gặp là tập hợp và vòng, đồng thời áp dụng thuật toán ở mục 6. Mục 9 giới thiệu ngắn gọn các thuật toán và định lý liên quan tới phép hợp thành. Mục 10 trình bày kỹ thuật dùng hàm sinh hai biến.

7.2 Đa thức và chuỗi lũy thừa hình thức

7.2.1 Đa thức

Đa thức là một khái niệm toán học quen thuộc. Một đa thức theo biến x có thể viết thành

$$A(x) = \sum_{i=0}^{n-1} a_i x^i. \quad (2.1)$$

Các hệ số thuộc một vành R , và toàn bộ các đa thức như vậy tạo thành vành đa thức $R[x]$.¹¹

Bậc của đa thức khác không $A(x)$ được định nghĩa là bậc của hạng tử khác không cao nhất, ký hiệu $\deg A(x)$.

7.2.2 Các phép toán cơ bản trên đa thức

Giả sử bậc lớn nhất của các đa thức tham gia phép toán là n . Khi đó phép cộng và phép trừ đa thức hiển nhiên có thể tính trong $O(n)$ thời gian.

Điều ta quan tâm hơn là tích của hai đa thức. Cách tính thô cần $O(n^2)$ thời gian, chưa đủ tốt.

Một hướng tối ưu là phép nhân chia để trị, hay Karatsuba. Nguyên lý của nó là dùng

$$\begin{aligned} (Ax^m + B)(Cx^m + D) &= ACx^{2m} \\ &+ ((A+B)(C+D) - AC - BD)x^m + BD. \end{aligned}$$

Vì thế chỉ cần ba phép nhân kích thước một nửa, và có truy hồi

$$T(n) = 3T(n/2) + O(n) = O(n^{\log_2 3}).$$

Nhược điểm của phép nhân chia để trị là độ phức tạp vẫn còn khá cao, nhưng nó không dùng phép chia nên không đặt yêu cầu đặc biệt lên vành R .

Ngoài ra, khi phép toán được thực hiện theo modulo 2, ta có thể dùng phép toán bit để tăng tốc, làm hằng số giảm khoảng 32 lần.

¹¹Ở đây thường hiểu là vành giao hoán, chẳng hạn trường số phức $\mathbb{C}$, trường số thực $\mathbb{R}$, vành số nguyên $\mathbb{Z}$, hay vành lớp thặng dư $\mathbb{Z}/m\mathbb{Z}$.

Cách khác là dùng FFT.¹² FFT lợi dụng tính chất của căn đơn vị để chuyển nhanh giữa biểu diễn hệ số và biểu diễn giá trị điểm của đa thức. Độ phức tạp thời gian là $O(n \log n)$.

Để dùng FFT, vành cần có căn đơn vị bậc 2^k với $2^k > 2n$, đồng thời 2 cần có nghịch đảo nhân. Khi hệ số nằm trong trường số phức, căn đơn vị luôn tồn tại, nhưng khi tính toán dễ gặp lỗi độ chính xác.

Cần chú ý rằng các bài đếm trong thi tin học thường yêu cầu in đáp án sau khi lấy modulo một số nguyên tố lớn p , chẳng hạn $10^9 + 7$, hoặc $998244353 = 7 \times 17 \times 2^{23} + 1$. Cách này có các lợi ích: thời gian cho mỗi phép toán số học có thể coi là $O(1)$, không có vấn đề sai số, và trong phần lớn tình huống có thể hỗ trợ phép chia, miễn là trong phạm vi quy mô bài toán, số chia nhỏ hơn p và có nghịch đảo nhân. Vì vậy bài viết này mặc định mọi phép toán đều thực hiện trong $\mathbb{Z}_p$.

Khi làm FFT theo modulo p , nếu $2^k \mid \varphi(p) = p - 1$, ta có thể lấy một căn nguyên thủy g của p rồi dùng $g^{\varphi(p)/2^k}$ làm căn đơn vị. Nếu không, có thể xem hệ số như phân tử của $\mathbb{Z}$, nhân xong rồi lấy modulo p . Khi đó hệ số của tích không vượt quá cỡ np^2 ; chỉ cần chọn vài số nguyên tố lớn tiện cho FFT, tính riêng từng modulo, rồi dùng định lý phần dư Trung Hoa để khôi phục hệ số.

7.2.3 Chuỗi lũy thừa hình thức

Một đa thức chỉ có hữu hạn hạng tử khác không. Nếu bỏ hạn chế này, ta thu được chuỗi lũy thừa hình thức

$$A(x) = \sum_{i \geq 0} a_i x^i. \quad (2.2)$$

Các chuỗi này tạo thành vành chuỗi lũy thừa hình thức $R[[x]]$.

Trong định nghĩa này, x chỉ là một ký hiệu, không cần thay một giá trị cụ thể vào để tính. Vì thế ta không phải xét các vấn đề hội tụ hay phân kỳ của chuỗi.

Ta dùng $[x^n]A(x)$ để chỉ hệ số của hạng tử bậc n trong $A(x)$.

Phép cộng, trừ và nhân của chuỗi lũy thừa hình thức được định nghĩa tương tự đa thức. Với

$$A(x) = \sum_{i \geq 0} a_i x^i, \quad B(x) = \sum_{i \geq 0} b_i x^i, \quad (2.3)$$

ta có

$$A(x) \pm B(x) = \sum_{i \geq 0} (a_i \pm b_i) x^i, \quad (2.4)$$

$$A(x)B(x) = \sum_{k \geq 0} \left(\sum_{i+j=k} a_i b_j \right) x^k. \quad (2.5)$$

Khi tính toán thực tế, ta thường chỉ cần giữ các hạng tử bậc không quá $n - 1$ và bỏ các hạng tử dư, tức tính theo modulo x^n . Vì vậy cộng và trừ có độ phức tạp $O(n)$, còn nhân có thể đạt $O(n \log n)$.

7.3 Bài toán đếm tổ hợp

7.3.1 Đối tượng tổ hợp

Bài toán đếm tổ hợp là một lớp bài toán thường gặp. Thông thường đề bài định nghĩa một lớp đối tượng tổ hợp $\mathcal{A}$, có thể là tập các cây, đồ thị, xâu, v.v. thỏa một tính chất nào đó. Mỗi đối tượng $a \in \mathcal{A}$ có một kích thước $\text{size}(a) \leq N$, chẳng hạn số nút hay độ dài dãy. Với một n cố định, ta cần đếm số đối tượng $a \in \mathcal{A}$ thỏa $\text{size}(a) = n$.

Tùy yêu cầu đề bài, đối tượng tổ hợp có thể chia thành hai loại: không gắn nhãn và có gắn nhãn. Lấy đồ thị vô hướng làm ví dụ: trong đồ thị có nhãn trên n đỉnh, mỗi đỉnh được gán một nhãn duy nhất trong

¹²http://en.wikipedia.org/wiki/Discrete_Fourier_transform

$1, 2, \dots, n$; còn trong đồ thị không nhãn, các đỉnh không phân biệt vai trò. Khi $n = 3$, số đồ thị đơn vô hướng không nhãn là 4, còn số đồ thị đơn vô hướng có nhãn là 8.

7.3.2 Hàm sinh thường

Hàm sinh thường (ordinary generating function, OGF) của dãy $A_0, A_1, \dots$ là

$$A(x) = \sum_{i \geq 0} A_i x^i. \quad (3.1)$$

Nếu $\mathcal{A}$ là một lớp đối tượng không nhãn, thì OGF của $\mathcal{A}$ được định nghĩa là OGF của dãy $A_0, A_1, \dots$, trong đó A_n là số đối tượng $a \in \mathcal{A}$ thỏa $\text{size}(a) = n$.

Với hai lớp đối tượng không nhãn $\mathcal{A}, \mathcal{B}$, định nghĩa lớp đối tượng mới $\mathcal{C} = \mathcal{A} \cup \mathcal{B}$. Nếu $\mathcal{A}$ và $\mathcal{B}$ rời nhau thì hàm sinh của $\mathcal{C}$ là

$$C(x) = A(x) + B(x). \quad (3.2)$$

Ta cũng có thể định nghĩa tích ghép $\mathcal{D} = \mathcal{A} \times \mathcal{B}$, trong đó mỗi phần tử d là một cặp (a, b) , với $a \in \mathcal{A}, b \in \mathcal{B}$, và

$$\text{size}(d) = \text{size}(a) + \text{size}(b).$$

Khi đó

$$D_k = \sum_{i+j=k} A_i B_j, \quad (3.3)$$

nên OGF của $\mathcal{D}$ là

$$D(x) = A(x)B(x). \quad (3.4)$$

Phép toán này biểu diễn việc ghép một phần tử của $\mathcal{A}$ với một phần tử của $\mathcal{B}$.

7.3.3 Hàm sinh mũ

Hàm sinh mũ (exponential generating function, EGF) của dãy $A_0, A_1, \dots$ là

$$A(x) = \sum_{i \geq 0} A_i \frac{x^i}{i!}. \quad (3.5)$$

Hàm sinh mũ thuận tiện hơn khi xử lý các bài toán có nhãn. Với một lớp đối tượng có nhãn $\mathcal{A}$, EGF của $\mathcal{A}$ là EGF của dãy $A_0, A_1, \dots$, trong đó A_n là số đối tượng $a \in \mathcal{A}$ có $\text{size}(a) = n$.

Với hai lớp đối tượng có nhãn $\mathcal{A}, \mathcal{B}$, nếu xét hợp rời $\mathcal{C} = \mathcal{A} \cup \mathcal{B}$, ta vẫn có

$$C(x) = A(x) + B(x). \quad (3.6)$$

Khi ghép một đối tượng kích thước n của $\mathcal{A}$ với một đối tượng kích thước m của $\mathcal{B}$, cần phân phối lại các nhãn $\{1, 2, \dots, n+m\}$ cho hai phần, đồng thời giữ thứ tự tương đối ban đầu của các nhãn trong mỗi phần. Số cách phân phối là

$$\binom{n+m}{n} = \frac{(n+m)!}{n!m!}. \quad (3.7)$$

Vì thế nếu ghép hai lớp đối tượng có nhãn $\mathcal{A}, \mathcal{B}$ để được $\mathcal{D}$, thì

$$D_k = \sum_{i+j=k} A_i B_j \frac{k!}{i!j!}. \quad (3.8)$$

Suy ra EGF của $\mathcal{D}$ cũng có dạng

$$D(x) = A(x)B(x). \quad (3.9)$$

7.4 Nghịch đảo nhân

Khi $A(x)B(x) = 1$, ta nói $A(x)$ và $B(x)$ là nghịch đảo nhân của nhau, và có thể viết $A(x) = B(x)^{-1} = 1/B(x)$. Chia cho một chuỗi lũy thừa hình thức tương đương với nhân với nghịch đảo nhân của nó.

Ví dụ,

$$\frac{1}{1-x} = \sum_{i \geq 0} x^i. \quad (4.1)$$

Từ đẳng thức này, nếu dãy $a_0, a_1, \dots$ có OGF là $A(x)$, và $s_n = \sum_{i=0}^n a_i$, thì dãy $s_0, s_1, \dots$ có OGF là $A(x)/(1-x)$.

Kết hợp thêm tính chất truy hồi của hệ số nhị thức, ta có

$$\frac{1}{(1-x)^k} = \sum_{i \geq 0} \binom{k+i-1}{i} x^i. \quad (4.2)$$

Tiếp theo, xét cách tìm nghịch đảo nhân $B(x)$ của $A(x)$ khi đã biết $A(x)$.

Có thể chứng minh $A(x)$ có nghịch đảo nhân khi và chỉ khi hệ số hằng $[x^0]A(x)$ có nghịch đảo nhân. Tính cần thiết suy từ $[x^0]A(x)[x^0]B(x) = 1$. Khi hệ số hằng của $A(x)$ khả nghịch, theo định nghĩa phép nhân (2.5), ta có thể lần lượt tính các giá trị $[x^0]B(x), [x^1]B(x), [x^2]B(x), \dots$. Khi đó nghịch đảo nhân tồn tại và duy nhất. Đồng thời, ta thu được thuật toán thô $O(n^2)$ để tính n hạng tử đầu.

Dưới đây là thuật toán $O(n \log n)$, bản chất là phương pháp Newton.

Trước hết tính nghịch đảo p của hệ số hằng của $A(x)$, rồi lấy giá trị ban đầu của $B(x)$ là p .

Giả sử đã tính được $B(x)$ thỏa

$$A(x)B(x) \equiv 1 \pmod{x^n}. \quad (4.3)$$

Khi đó

$$\begin{aligned} A(x)B(x) - 1 &\equiv 0 \pmod{x^n}, \\ (A(x)B(x) - 1)^2 &\equiv 0 \pmod{x^{2n}}, \end{aligned}$$

và suy ra

$$A(x)(2B(x) - B(x)^2 A(x)) \equiv 1 \pmod{x^{2n}}. \quad (4.4)$$

Ta dùng $O(n \log n)$ thời gian để tính $2B(x) - B(x)^2 A(x)$, rồi gán lại cho $B(x)$ để bước sang vòng lặp kế tiếp. Mỗi lần lặp, số hạng tử đúng của $B(x)$ tăng gấp đôi, nên độ phức tạp là

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

7.5 Ứng dụng của nghịch đảo nhân

Ví dụ 1: Đếm dãy. Ta có một số loại quân domino khác màu nhau; trong đó domino kích thước $1 \times i$ có a_i loại. Mỗi loại domino có thể dùng vô hạn lần. Hỏi có bao nhiêu cách lát kín một dải $1 \times n$ mà không chồng lấn?

$$a_i, n \leq 10^5.$$

Ta liệt kê số quân domino được dùng. Đặt

$$A(x) = \sum_{i=1}^n a_i x^i.$$

Theo ý nghĩa của phép nhân hàm sinh, khi dùng đúng k quân, số cách là $[x^k]A(x)^k$. Do đó tổng số cách là

$$\sum_{k \geq 0} [x^k]A(x)^k = [x^n] \frac{1}{1-A(x)}. \quad (5.1)$$

Vì vậy chỉ cần tính nghịch đảo nhân của $1 - A(x)$, độ phức tạp $O(n \log n)$.

Tổng quát hơn, với một lớp đối tượng tổ hợp $\mathcal{A}$, nếu lấy các dãy gồm các phần tử của $\mathcal{A}$ để định nghĩa lớp mới $\mathcal{B}$, thì hàm sinh của $\mathcal{B}$ là

$$B(x) = \frac{1}{1 - A(x)}. \quad (5.2)$$

Kết luận này đúng cho cả trường hợp không nhãn (OGF) lẫn có nhãn (EGF).

Ví dụ 2: Tiền xử lý số Bernoulli. Với mọi $0 \leq i \leq n - 1$, hãy tính B_i , trong đó $n \leq 10^5$.

EGF của các số Bernoulli $B_0, B_1, \dots$ là

$$B(x) = \sum_{i \geq 0} B_i \frac{x^i}{i!} = \frac{x}{e^x - 1} = \left(\sum_{i \geq 0} \frac{x^i}{(i+1)!} \right)^{-1}. \quad (5.3)$$

Như vậy chỉ cần tính một lần nghịch đảo nhân.

Ví dụ 3. Bảng chữ cái có kích thước m . Cho một xâu s độ dài k . Hỏi trong mọi xâu độ dài n , có bao nhiêu xâu không chứa s làm xâu con?

$$n, m, k \leq 10^5.$$

Giả sử chỉ số của xâu đều bắt đầu từ 1. Xét quy hoạch động: dùng f_i là số cách điền i ký tự đầu sao cho lần đầu khớp với s nằm ở đoạn $[i - k + 1, i]$. Khi $i < k$, $f_i = 0$; khi $i \geq k$,

$$f_i = m^{i-k} - \sum_{0 \leq j \leq i-k} f_j m^{i-k-j} - \sum_{\substack{d > 0 \\ d \in C}} f_{i-d}, \quad (5.4)$$

trong đó

$$C = \{d \geq 0 \mid s[d+1, k] = s[1, k-d]\}.$$

Ý nghĩa của công thức là: k ký tự cuối giống s , còn $i - k$ ký tự trước đó có thể chọn tùy ý; để đảm bảo vị trí $i - k + 1$ là lần khớp đầu tiên, ta phải trừ các trường hợp đã khớp trước đó. Tổng thứ nhất trừ những lần khớp trước không chồng lên lần này, tổng thứ hai trừ những lần khớp trước có chồng lấn. Tập C có thể tìm trước bằng thuật toán KMP.

Để tối ưu DP, xét hàm sinh

$$f(x) = \sum_{i \geq 0} f_i x^i, \quad c(x) = \sum_{d \in C} x^d.$$

Từ phương trình DP suy ra

$$f(x) = \frac{x^k}{1 - mx} - \frac{x^k f(x)}{1 - mx} - f(x)(c(x) - 1),$$

tức

$$f(x) = \frac{x^k}{x^k + (1 - mx)c(x)}.$$

Đáp án cuối cùng là

$$g_n = m^n - \sum_{i \geq 0} f_i m^{n-i},$$

và hàm sinh tương ứng là

$$g(x) = \frac{c(x)}{x^k + (1 - mx)c(x)}. \quad (5.5)$$

Độ phức tạp thời gian là $O(n \log n)$.

7.6 Logarit và hàm mũ

7.6.1 Phép hợp thành

Trước hết đưa vào phép hợp thành của chuỗi lũy thừa hình thức. Đặt

$$A(w) = \sum_{i \geq 0} a_i w^i, \quad B(x) = \sum_{i \geq 1} b_i x^i.$$

Hợp thành của $B(x)$ với $A(w)$ là

$$C(x) = A \circ B(x) = A(B(x)) = \sum_{i \geq 0} a_i (B(x))^i, \quad (6.1)$$

và có thể thu gọn về dạng $C(x) = \sum_{i \geq 0} c_i x^i$.

Vì đã giả sử $B(x)$ không có hệ số hằng, nên ngay cả khi A có vô hạn hạng tử khác không, hợp thành $A(B(x))$ vẫn được định nghĩa.

7.6.2 Đạo hàm hình thức

Với

$$A(x) = \sum_{i \geq 0} a_i x^i,$$

định nghĩa đạo hàm hình thức của $A(x)$ là

$$A'(x) = \sum_{i \geq 1} i a_i x^{i-1}. \quad (6.2)$$

Ta dễ kiểm chứng các quy tắc đạo hàm cơ bản vẫn đúng:

$$(cA(x))' = cA'(x), \quad (6.3)$$

$$(A(x) \pm B(x))' = A'(x) \pm B'(x), \quad (6.4)$$

$$(A(x)B(x))' = A'(x)B(x) + A(x)B'(x), \quad (6.5)$$

$$\left(\frac{1}{A(x)} \right)' = -\frac{A'(x)}{A(x)^2}, \quad (6.6)$$

$$(A(B(x)))' = A'(B(x))B'(x). \quad (6.7)$$

7.6.3 Hàm logarit và hàm mũ

Ta có thể định nghĩa logarit và hàm mũ của chuỗi lũy thừa hình thức. Điều này tương đương hợp thành chuỗi đã cho với chuỗi Maclaurin tương ứng:

$$\ln(1 - A(x)) = - \sum_{i \geq 1} \frac{A(x)^i}{i}, \quad (6.8)$$

$$\exp(A(x)) = \sum_{i \geq 0} \frac{A(x)^i}{i!}. \quad (6.9)$$

Dễ kiểm chứng phần lớn tính chất của logarit và hàm mũ vẫn còn đúng trong ngữ cảnh này.

Tính logarit. Cho

$$A(x) = 1 + \sum_{i \geq 1} a_i x^i, \quad B(x) = \ln A(x).$$

Khi đó

$$B'(x) = \frac{A'(x)}{A(x)}. \quad (6.10)$$

Vì vậy chỉ cần tính nghịch đảo nhân của $A(x)$, rồi tích phân hình thức, là có thể tìm $\ln A(x)$ trong $O(n \log n)$ thời gian.

Tính hàm mũ. Cho

$$A(x) = \sum_{i \geq 1} a_i x^i, \quad B(x) = \exp(A(x)) = \sum_{i \geq 0} b_i x^i.$$

Khi đó

$$B'(x) = B(x)A'(x). \quad (6.11)$$

So sánh hệ số, ta được

$$b_0 = 1, \quad b_i = \frac{1}{i} \sum_{j=1}^i j a_j b_{i-j} \quad (i \geq 1). \quad (6.12)$$

Có thể dùng chia để trị để tính toàn bộ b_i ; mỗi lần dùng FFT tính đóng góp của nửa trái lên nửa phải, thời gian $O(n \log^2 n)$.

Dưới đây là một thuật toán $O(n \log n)$ để tính $\exp(A(x))$. Đặt

$$f(x) = \exp(A(x)).$$

Khi đó $f(x)$ là nghiệm của phương trình

$$g(f(x)) = \ln(f(x)) - A(x) = 0. \quad (6.13)$$

Giả sử đã biết $f_0(x)$, tức $f(x)$ theo modulo x^n :

$$f(x) \equiv f_0(x) \pmod{x^n}. \quad (6.14)$$

Khai triển Taylor,

$$\begin{aligned} 0 &= g(f(x)) \\ &\equiv g(f_0(x)) + g'(f_0(x))(f(x) - f_0(x)) \pmod{x^{2n}}. \end{aligned}$$

Suy ra

$$f(x) \equiv f_0(x) - \frac{g(f_0(x))}{g'(f_0(x))} \pmod{x^{2n}}. \quad (6.15)$$

Thay (6.13) vào, ta được

$$f(x) = f_0(x)(1 - \ln(f_0(x)) + A(x)). \quad (6.16)$$

Lặp theo công thức này, độ phức tạp là

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

7.6.4 Tính lũy thừa

Ví dụ 4. Cho đa thức $f(x)$ và số nguyên dương k , hãy tính n hệ số đầu của $f(x)^k$.

$$n, k \leq 10^5.$$

Thuật toán lũy thừa nhanh thông thường cần $O(\log k)$ phép nhân đa thức, nên thời gian là

$$O(n \log n \log k).$$

Theo tính chất của hàm mũ và logarit, khi hệ số hằng của $f(x)$ bằng 1, ta có

$$f(x)^k = \exp(k \ln f(x)). \quad (6.17)$$

Tính theo công thức này mất $O(n \log n)$ thời gian.

Nếu hệ số hằng của $f(x)$ không bằng 1, giả sử hạng tử bậc thấp nhất của $f(x)$ là bx^d . Khi đó

$$f(x)^k = b^k x^{dk} \left(\frac{f(x)}{bx^d} \right)^k,$$

và ta đưa về trường hợp hệ số hằng bằng 1 rồi dùng (6.17).

Nếu $f(x)$ có hệ số hằng bằng 0 và $dk \geq n$, thì mọi hệ số cần tìm đều bằng 0; nếu không, chỉ cần dịch bậc tương ứng sau khi tính phần còn lại.

7.7 Đếm tập hợp

7.7.1 Đếm tập hợp có nhãn

Trong ví dụ 1, ta đã bàn về số dãy tạo bởi các đối tượng không nhãn, và kết luận cũng đúng với đối tượng có nhãn. Bây giờ xét số tập hợp tạo bởi các đối tượng có nhãn.

Khác biệt giữa tập hợp và dãy là các phần tử trong tập không có thứ tự, nên cần loại bỏ thống kê trùng. Vì vậy EGF của tập hợp các phần tử thuộc $\mathcal{A}$ là

$$B(x) = \sum_{i \geq 0} \frac{A(x)^i}{i!} = e^{A(x)}. \quad (7.1)$$

Ví dụ 5: Đếm đồ thị liên thông. Tính số đồ thị vô hướng liên thông trên n đỉnh. Đỉnh có nhãn, không cho phép cạnh bội và khuyên.

$$n \leq 10^5.$$

Gọi $\mathcal{G}$ là lớp mọi đồ thị đơn vô hướng. EGF của $\mathcal{G}$ là

$$G(x) = \sum_{n \geq 0} 2^{\binom{n}{2}} \frac{x^n}{n!}. \quad (7.2)$$

Gọi $\mathcal{C}$ là lớp mọi đồ thị đơn liên thông. Vì một đồ thị đơn vô hướng có thể xem là một tập hợp các thành phần liên thông, nên

$$G(x) = e^{C(x)}.$$

Do đó

$$C(x) = \ln G(x). \quad (7.3)$$

Chỉ cần tính logarit của $G(x)$ là thu được đáp án, độ phức tạp $O(n \log n)$.

Ví dụ 6: Đếm hoán vị có hạn chế. Cho tập S và số nguyên dương n . Tính số hoán vị bậc n sao cho mọi chu trình trong phân rã chu trình của nó đều có kích thước thuộc S .

$$n \leq 10^5.$$

Một hoán vị là một tập hợp các chu trình, mỗi chu trình tương đương một vòng có nhãn. Theo công thức hoán vị vòng, số chu trình kích thước k là $(k-1)!$, EGF tương ứng là

$$\frac{(k-1)!x^k}{k!} = \frac{x^k}{k}. \quad (7.4)$$

Vì vậy EGF của đáp án là

$$\exp\left(\sum_{k \in S} \frac{x^k}{k}\right). \quad (7.5)$$

7.7.2 Đếm tập hợp không nhãn

Ví dụ 7: Đếm balo đầy đủ. Ta có một số loại vật phẩm khác nhau; trong đó vật phẩm thể tích i có a_i loại. Mỗi loại vật phẩm có vô hạn bản sao. Hỏi có bao nhiêu cách chọn vật phẩm để tổng thể tích đúng bằng n ?

$$a_i, n \leq 10^5.$$

Trong bài này, tập hợp không nhãn có thể chứa các phần tử lặp lại, nên không thể chỉ chia cho $i!$ để khử trùng.

Đặt $A(x) = \sum_i a_i x^i$. Xét riêng từng thể tích i , viết hàm sinh rồi nhân lại, sau đó dùng logarit để biến tích thành tổng, ta thu được OGF:

$$\begin{aligned} \prod_{1 \leq i \leq n} (1 + x^i + x^{2i} + \dots)^{a_i} &= \prod_{1 \leq i \leq n} \left(\frac{1}{1 - x^i} \right)^{a_i} \\ &= \exp\left(-\sum_{1 \leq i \leq n} a_i \ln(1 - x^i)\right) \\ &= \exp\left(\sum_{j \geq 1} \frac{1}{j} A(x^j)\right). \end{aligned} \quad (7.6)$$

Chú ý rằng trong $A(x^j)$ chỉ có $\lfloor n/j \rfloor$ hạng tử hữu ích, nên có thể tính tổng mọi $A(x^j)$ trong thời gian

$$n + \frac{n}{2} + \frac{n}{3} + \dots + 1 = O(n \log n).$$

Cuối cùng tính thêm một lần exp, tổng độ phức tạp là $O(n \log n)$.

Ví dụ 8: Đếm balo 0-1. Ta có một số loại vật phẩm khác nhau; trong đó vật phẩm thể tích i có a_i loại. Mỗi vật phẩm chỉ có một bản. Hỏi có bao nhiêu cách chọn vật phẩm để tổng thể tích đúng bằng n ?

$$a_i, n \leq 10^5.$$

Cách xử lý giống ví dụ trước: OGF là

$$\prod_i (1 + x^i)^{a_i} = \exp\left(\sum_{j \geq 1} \frac{(-1)^{j-1}}{j} A(x^j)\right).$$

Vì vậy không nhắc lại chi tiết.

7.8 Đếm vòng

Mục này xét đếm vòng. Điểm đặc biệt của vòng là nó có thể quay: hai vòng giống nhau sau một phép quay được coi là cùng một vòng.

7.8.1 Đếm vòng có nhãn

Trường hợp dễ xử lý hơn là vòng có nhãn. Theo công thức hoán vị vòng, số vòng gồm k phần tử là $(k-1)!$. Vì vậy với một lớp đối tượng $\mathcal{A}$, EGF của các vòng tạo bởi phần tử của $\mathcal{A}$ (được quay nhưng không được lật) là

$$\sum_{k \geq 1} \frac{A(x)^k}{k} = -\ln(1 - A(x)). \quad (8.1)$$

Ví dụ 9. Tính số đồ thị vô hướng liên thông có n đỉnh và n cạnh. Đỉnh có nhãn, không cho phép cạnh bội và khuyên.

$$n \leq 10^5.$$

Theo kiến thức đồ thị cơ bản, đồ thị như vậy chứa đúng một vòng độ dài không nhỏ hơn 3, và mỗi đỉnh trên vòng là gốc của một cây có gốc.

Trước hết xét bài toán cây. Ở đây cây là cây có gốc có nhãn, và các con của mỗi nút không có thứ tự. Gọi EGF của cây có gốc là $T(x)$. Theo định lý Cayley, số cây có gốc trên n đỉnh là n^{n-1} , nên

$$T(x) = \sum_{n \geq 1} \frac{n^{n-1} x^n}{n!}. \quad (8.2)$$

Ta cũng có thể từ định nghĩa cây suy ra $T(x) = xe^{T(x)}$, rồi dùng phản diễn Lagrange để thu được kết luận này.

Tiếp theo xét vòng. Cần chú ý rằng vòng trong bài này không có hướng, tức có thể lật. Vì vậy EGF của các vòng gồm cây, có độ dài ít nhất 3, là

$$\frac{1}{2} \sum_{k \geq 3} \frac{T(x)^k}{k} = -\frac{1}{2} \ln(1 - T(x)) - \frac{T(x)}{2} - \frac{T(x)^2}{4}. \quad (8.3)$$

7.8.2 Đếm vòng không nhãn

Ví dụ 10: Đếm vòng không nhãn. Ta có một số loại hạt khác màu; trong đó hạt trọng lượng i có a_i loại. Mỗi loại hạt có thể dùng vô hạn lần. Hỏi có bao nhiêu chuỗi hạt tổng trọng lượng n ? Chuỗi hạt được phép quay nhưng không được lật.

$$a_i, n \leq 10^5.$$

Vòng không nhãn phức tạp hơn một chút, vì trong vòng có thể xuất hiện các phần tử giống nhau, tạo ra chu kỳ. Ta tự nhiên dùng công thức đếm Pólya để xử lý.

Với lớp đối tượng $\mathcal{A}$, OGF của vòng gồm đúng k phần tử thuộc $\mathcal{A}$ là

$$\frac{1}{k} \sum_{i=0}^{k-1} A(x^{k/\gcd(i,k)})^{\gcd(i,k)} = \sum_{d|k} \frac{\varphi(d)}{k} A(x^d)^{k/d}.$$

Suy ra OGF của mọi vòng tạo bởi phần tử của $\mathcal{A}$ là

$$\begin{aligned} \sum_{k \geq 1} \sum_{d|k} \frac{\varphi(d)}{k} A(x^d)^{k/d} &= \sum_{d \geq 1} \frac{\varphi(d)}{d} \sum_{r \geq 1} \frac{A(x^d)^r}{r} \\ &= - \sum_{d \geq 1} \frac{\varphi(d)}{d} \ln(1 - A(x^d)). \end{aligned} \quad (8.4)$$

Tương tự ví dụ 7, trong $\ln(1 - A(x^d))$ chỉ có $\lfloor n/d \rfloor$ hạng tử hữu ích. Do đó chỉ cần tính $\ln(1 - A(x))$ một lần, rồi dùng $O(n \log n)$ thời gian cộng các đóng góp. Tổng thời gian là $O(n \log n)$.

7.9 Phép hợp thành

7.9.1 Hợp thành và nghịch đảo hợp thành

Ở mục 6.1, ta đã giới thiệu phép hợp thành của hai chuỗi lũy thừa hình thức.

Giả sử

$$f(x) = \sum_{i \geq 1} f_i x^i, \quad g(x) = \sum_{i \geq 1} g_i x^i.$$

Nếu $f(g(x)) = x$, ta gọi $f(x)$ và $g(x)$ là nghịch đảo hợp thành, hay hàm ngược của nhau. Đồng thời có $f_1 g_1 = 1$ và $g(f(x)) = x$.

Giả sử $f(x)$ và $g(x)$ là nghịch đảo hợp thành. Nếu tồn tại một thuật toán có thể tính nhanh $f(p(x))$ với mọi $p(x)$, thì có thể dùng phương pháp Newton ở mục 6.4 để tính $g(q(x))$ với mọi $q(x)$. Lấy $q(x) = x$ sẽ thu được $g(x)$.

Với $f(x), g(x)$ đã biết, nếu trực tiếp dùng FFT theo định nghĩa để tính n hệ số đầu của $f(g(x))$, độ phức tạp là $O(n^2 \log n)$. Dưới đây là một thuật toán $O(n^2)$, cốt lõi là tư tưởng chia căn bậc hai.

Đặt $d = \lceil \sqrt{n} \rceil$. Trước hết tính

$$g(x)^2, g(x)^3, \dots, g(x)^d$$

trong $O(dn \log n)$ thời gian. Sau đó dùng kết quả này để tính tiếp

$$g(x)^d, g(x)^{2d}, \dots, g(x)^{d^2},$$

cũng trong $O(dn \log n)$ thời gian.

Tiếp theo, theo

$$f(g(x)) = \sum_{0 \leq i < d} g(x)^{id} \sum_{0 \leq j < d} f_{id+j} g(x)^j, \quad (9.1)$$

thay các kết quả đã tính vào là được. Tổng thời gian là

$$O(n\sqrt{n} \log n + n^2) = O(n^2).$$

Tài liệu tham khảo [1] có một thuật toán $O((n \log n)^2)$ để tính $f(g(x))$; độc giả quan tâm có thể tự xem thêm.

7.9.2 Phản diễn Lagrange

Công thức phản diễn Lagrange: nếu $f(w)$ và $g(x)$ là hai chuỗi nghịch đảo hợp thành, thì

$$[x^n]g(x) = \frac{1}{n} [w^{n-1}] \left(\frac{w}{f(w)} \right)^n, \quad (9.2)$$

và tổng quát hơn,

$$[x^n]h(g(x)) = \frac{1}{n} [w^{n-1}] h'(w) \left(\frac{w}{f(w)} \right)^n. \quad (9.3)$$

Lấy $h(w) = w$ chính là (9.2).

Tính toàn bộ nghịch đảo hợp thành của $g(x)$ khá tốn kém, nhưng theo (9.2) và phương pháp trong ví dụ 4, ta có thể tính một hệ số cụ thể của nghịch đảo hợp thành trong $O(n \log n)$ thời gian.

Ví dụ 11. Cho số nguyên n và tập S . Tính số cây có gốc nhiều nhánh gồm n nút, trong đó mỗi nút không phải lá v có số con $\deg(v) \in S$. Các nút không có nhãn, nhưng các con của một nút có thứ tự.

$$n \leq 10^5.$$

Gọi OGF của các cây này là $T(x)$. Theo định nghĩa, một cây hoặc chỉ gồm một nút lá, hoặc gồm một nút không phải lá ghép với một dãy s cây con, trong đó $s \in S$. Vì vậy

$$T(x) = x + \sum_{s \in S} T(x)^s. \quad (9.4)$$

Có thể viết phương trình này thành $x = f(T(x))$, trong đó $f(w)$ là một đa thức có hệ số bậc nhất bằng 1. Khi đó $T(x)$ chính là nghịch đảo hợp thành của $f(w)$, nên dùng phản diễn Lagrange để tính hệ số bậc n .

Ví dụ 12: Đếm đồ thị cạnh-song-liên-thông có nhãn. Tính số đồ thị vô hướng cạnh-song-liên-thông bậc n . Đỉnh có nhãn, không cho phép cạnh bội và khuyên.¹³

$$n \leq 10^5.$$

Gọi $C(x)$ là EGF của mọi đồ thị vô hướng liên thông, và $B(x)$ là EGF của mọi đồ thị cạnh-song-liên-thông trong số đó. Ví dụ 5 đã tính được $C(x)$; bài toán hiện tại là tính hệ số bậc n của $B(x)$.

Xét cặp (G, v) , trong đó G là một đồ thị, còn v là một đỉnh đặc biệt của G ; gọi đó là đồ thị có gốc. Một đồ thị G_0 sinh ra $|G_0|$ đồ thị có gốc, nên EGF của mọi đồ thị liên thông có gốc là $xC'(x)$.

Với đồ thị có gốc (G_C, v) , co G_C thành cây BCC T , mỗi nút của cây biểu diễn một BCC, tức một thành phần cạnh-song-liên-thông của đồ thị gốc. Gọi BCC chứa v là B_v và chọn nó làm gốc của cây T . Nếu B_v có n đỉnh, thì EGF của đồ thị có gốc (B_v, v) là $nb_n x^n / n!$.

Trong T , gốc B_v có thể có một số cây con, hoặc không có cây con nào. Mỗi cây con cũng đến từ cây BCC của một đồ thị liên thông có gốc (G_1, u) , và đỉnh u nối với một đỉnh w trong B_v . EGF của đồ thị có gốc (G_1, u) là $xC'(x)$; vì w có $|B_v| = n$ cách chọn, EGF của một cây con là $nx C'(x)$, còn EGF của một tập các cây con là $e^{nx C'(x)}$.

Liệt kê kích thước n của B_v , ta có

$$xC'(x) = \sum_{n \geq 1} \frac{b_n}{(n-1)!} x^n e^{nx C'(x)}. \quad (9.5)$$

Viết gọn

$$C_1(x) = xC'(x), \quad B_1(w) = wB'(w),$$

thì

$$C_1(x) = B_1(xe^{C_1(x)}). \quad (9.6)$$

Trong đó $C_1(x)$ và $xe^{C_1(x)}$ đều dễ tính. Ta cần một hệ số của $B_1(w)$. Bài toán trở thành: biết $P(x), Q(x)$, hãy tìm hệ số bậc n của $R(w)$ thỏa $P(x) = R(Q(x))$. Dùng $Q^{(-1)}(w)$ để chỉ nghịch đảo hợp thành của $Q(x)$, ta có $R(w) = P(Q^{(-1)}(w))$. Theo dạng mở rộng của phản diễn Lagrange,

$$[w^n]R(w) = \frac{1}{n} [x^{n-1}]P'(x) \left(\frac{x}{Q(x)} \right)^n.$$

Vì vậy có thể tính đáp án trong $O(n \log n)$ thời gian.

¹³Ở đây cạnh-song-liên-thông nghĩa là sau khi xóa bất kỳ một cạnh nào, đồ thị vẫn liên thông.

7.10 Hàm sinh hai biến

Đôi khi ta đưa thêm biến thứ hai vào hàm sinh để làm một số dấu đánh dấu phụ.

Ví dụ 13: Đếm đồ thị liên thông II. Tính số đồ thị vô hướng liên thông có n đỉnh và m cạnh. Đỉnh có nhãn, không cho phép cạnh bội và khuyên.

$$n, m \leq 1000.$$

Đưa chữ cái y vào để đánh dấu cạnh, tức dùng $x^n y^m / n!$ để biểu diễn EGF của đồ thị có n đỉnh và m cạnh. Khi đó EGF của mọi đồ thị vô hướng là

$$G(x, y) = \sum_{n \geq 0} \frac{x^n}{n!} (1 + y)^{\binom{n}{2}}. \quad (10.1)$$

Tương tự ví dụ 5, cần tính logarit của $G(x, y)$. Ở đây phát sinh phép nhân và nghịch đảo nhân của đa thức hai biến.

Để nhân hai đa thức hai biến, ta có thể xếp các hệ số thành ma trận, rồi làm DFT trên từng hàng và từng cột. Như vậy thu được biểu diễn giá trị điểm của đa thức hai biến; sau đó nhân các giá trị điểm và biến đổi ngược về hệ số. Quá trình tìm nghịch đảo nhân gần như giống trường hợp một biến. Tổng độ phức tạp là

$$O(nm(\log n + \log m)).$$

Ví dụ 14. Cho một đa thức $f(x)$ và một số k . Với mọi $1 \leq i \leq n$, hãy tính $[x^k]f(x)^i$.

$$n, k \leq 3000.$$

Giả sử $k = O(n)$. Dùng một thuật toán chia căn bậc hai tương tự mục 9.1 có thể đạt độ phức tạp $O(n^2)$.

Bây giờ xét từ một góc khác. Đưa vào chữ cái u , khi đó dãy đáp án cần tìm là hệ số bậc x^k trong

$$1 + uf(x) + u^2 f(x)^2 + \dots = \frac{1}{1 - uf(x)}. \quad (10.2)$$

Đây là một chuỗi lũy thừa theo u .

Để trích hệ số bậc x^k , xét phản diễn Lagrange. Trường hợp đơn giản nhất là $[x^0]f(x) = 0$ và $[x^1]f(x) \neq 0$; khi đó $f(x)$ có nghịch đảo hợp thành $g(w)$, và

$$[x^k] \frac{1}{1 - uf(x)} = \frac{1}{k} [w^{k-1}] \frac{u}{(1 - uw)^2} \left(\frac{w}{g(w)} \right)^k. \quad (10.3)$$

Mẫu số có thể khai triển bằng (4.2). Do đó bài toán này và bài toán tìm nghịch đảo hợp thành của $f(x)$ có thể chuyển hóa lẫn nhau trong $O(n \log n)$ thời gian.

Nếu $f(x)$ không có nghịch đảo hợp thành, còn cần xử lý thêm:

- Nếu hệ số hằng của $f(x)$ bằng 0, nhưng hạng tử bậc thấp nhất có bậc $d \geq 2$, ta tách phần x^d ra để đưa về trường hợp hạng tử thấp nhất có bậc 1, rồi chỉ giữ các bậc phù hợp.
- Nếu hệ số hằng của $f(x)$ là $c \neq 0$, viết $f(x) = c + q(x)$. Khi đó trước hết tính mọi $[x^k]q(x)^j$, rồi dùng

$$f(x)^i = \sum_{j=0}^i \binom{i}{j} c^{i-j} q(x)^j$$

và một phép chập FFT để thu được dãy đáp án.

Miễn là có thể tìm nghịch đảo hợp thành của $f(x)$, bài này có thể giải tiếp trong $O(n \log n)$. Nói chung, tìm nghịch đảo hợp thành rất khó đạt độ phức tạp tốt, nhưng trong một số trường hợp đặc biệt, nghịch đảo hợp thành của $f(x)$ dễ tìm; khi đó có thể dùng phương pháp này.

Ví dụ 15: Tiên xử lý số Stirling loại một. Với n cho trước, hãy tính mọi

$$\begin{bmatrix} n \\ k \end{bmatrix} \quad (0 \leq k \leq n),$$

trong đó $n \leq 10^5$.

Cách truyền thống là dùng

$$\sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} x^k = x(x+1)(x+2) \cdots (x+n-1) \quad (10.4)$$

và chia để trị: mỗi lần chia đều các nhân tử thành hai nửa, tính riêng rồi dùng FFT nhân lại. Độ phức tạp là $O(n \log^2 n)$.

Một cách khác dùng công thức

$$\begin{bmatrix} n \\ k \end{bmatrix} = \frac{n!}{k!} [x^n] (-\ln(1-x))^k. \quad (10.5)$$

Áp dụng phương pháp vừa nêu cho công thức này sẽ đạt độ phức tạp $O(n \log n)$, tuy nhiên hằng số của thuật toán hơi lớn.

7.11 Kết luận

Bài viết đã giới thiệu các thuật toán $O(n \log n)$ cho nghịch đảo nhân, logarit, hàm mũ và một số phép toán khác trên chuỗi lũy thừa hình thức; đồng thời lấy các mô hình tổ hợp như sâu, cây, đồ thị và tập hợp làm ví dụ ứng dụng trong bài toán đếm. Với nghịch đảo hợp thành, bài viết cũng giới thiệu cách tính một hệ số của nó trong $O(n \log n)$ thời gian.

Một số bài toán đếm vốn cần $O(n^2)$ hoặc $O(n \log^2 n)$ có thể được tối ưu thành $O(n \log n)$ bằng các phương pháp trong bài viết, nhưng đôi khi hằng số thuật toán khá lớn, cần chú ý khi cài đặt.

Có thể dự đoán rằng loại bài toán đếm tổ hợp này sẽ tiếp tục xuất hiện trong các kỳ thi tin học sau này. Tuy vậy, vẫn còn rất nhiều vấn đề thú vị chưa được đề cập trong bài viết, cần tiếp tục nghiên cứu và khám phá.

7.12 Lời cảm ơn

- Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Xin cảm ơn thầy Xu Xianyou đã hướng dẫn tôi.
- Xin cảm ơn các bạn Peng Yuxiang, Lu Kaifeng, Mao Xiao và nhiều bạn khác đã gợi mở cho tôi.
- Xin cảm ơn Xu Yinzhan đã đọc kiểm tra bản thảo bài viết.

Tài liệu tham khảo của bài viết

- [1] Brent R. P., Kung H. T. *Fast Algorithms for Manipulating Formal Power Series*. Journal of the ACM, 1978, 25(4): 581–595.

- [2] Philippe Flajolet, Robert Sedgewick. *Analytic Combinatorics*. Cambridge University Press, 2009.
- [3] Peng Yuxiang, *Inverse Element of Polynomial*.
- [4] #92h, *Fast Fourier Transform Modulo Prime*.
- [5] Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.

8 Báo cáo ra đề ydc's bonus

Liu Jiancheng
Trường Yali, Trường Sa

8.1 Ý chính đề bài

Có một cuộc thi. Nếu bạn đạt hạng i trong cuộc thi này thì bạn nhận được tiền thưởng p_i .

Hiện bạn đã biết trước năng lực của $n - 1$ thí sinh khác và của chính mình, từ đó tính được phân bố xác suất điểm số của từng người. Bài thi có điểm tối đa là 1, điểm tối thiểu là 0, và điểm số có thể là một số thực bất kỳ. Với người thứ i , xác suất để người đó đạt điểm x tỉ lệ thuận với một hàm đa thức, ký hiệu $f_i(x)$.

Nếu hàm của một người là $f(x) = 2$, thì mọi điểm số có xác suất như nhau. Nếu hàm của một người là $f(x) = 2 - x$, thì điểm càng cao càng ít có khả năng xảy ra, và xác suất đạt 0 điểm gấp đôi xác suất đạt 1 điểm.

Cần tính kỳ vọng số tiền thưởng mà bạn nhận được trong cuộc thi này. Vì điểm số có thể là số thực nên không cần xét trường hợp đồng hạng.

Đáp án lấy modulo

$$998244353 = 7 \cdot 17 \cdot 2^{23} + 1,$$

là một số nguyên tố.

Giới hạn thời gian của nguồn: Tsinghua 8 giây, UOJ 6 giây. Giới hạn bộ nhớ: 64 MB.

8.2 Định dạng vào

Dữ liệu vào có $n + 2$ dòng.

Dòng đầu chứa số nguyên n .

Dòng thứ hai chứa n số nguyên, số thứ i là p_i .

Trong $n - 1$ dòng tiếp theo, mỗi dòng mô tả một thí sinh khác. Số đầu tiên là t_i , nghĩa là hàm của người thứ i là đa thức bậc $t_i - 1$; tiếp theo là t_i số thực, số thứ j là hệ số của hạng tử x^{j-1} .

Dòng cuối mô tả hàm của bạn theo cùng cách: số đầu tiên là t_n , nghĩa là hàm của bạn là đa thức bậc $t_n - 1$; tiếp theo là t_n số thực, số thứ j là hệ số của x^{j-1} .

8.3 Định dạng ra

In một dòng gồm một số nguyên, biểu diễn kỳ vọng tiền thưởng mà bạn nhận được.

8.4 Phạm vi dữ liệu

Gọi

$$S = \sum_i t_i.$$

Các nhóm dữ liệu trong nguồn được tóm tắt như sau.

Nhóm	n	S	Ràng buộc đặc biệt
1	2	5	Không có
2	2	10	Không có
3	10	40	Không có
4	10	60	Không có
5	10	80	Không có
6	10	100	Không có
7	100	400	Không có
8	100	600	Không có
9	100	800	Không có
10	100	1000	Không có
11	500	4000	Tiền thưởng của mọi thứ hạng bằng nhau
12	500	4000	Hàm của mọi người giống nhau
13	500	4000	Trừ hạng nhất và hạng cuối, tiền thưởng của các hạng còn lại bằng nhau
14	500	4000	Trừ hạng nhất và hạng cuối, tiền thưởng của các hạng còn lại bằng nhau
15	500	1500	Không có
16	500	2000	Không có
17	500	2500	Không có
18	500	3000	Không có
19	500	3500	Không có
20	500	4000	Không có

Bảo đảm hàm của mọi người không âm trên đoạn $[0, 1]$. Mọi số thực trong đầu vào chỉ có hai chữ số thập phân; ngoài hệ số tự do, trị tuyệt đối của mọi hệ số đều nhỏ hơn 5, còn trị tuyệt đối của hệ số tự do nhỏ hơn 30. Mọi dữ liệu đều được sinh ngẫu nhiên. Các p_i nằm trong khoảng từ 0 đến 10000.

8.5 Giới thiệu thuật toán

8.5.1 Thuật toán 1

Trong 10% dữ liệu đầu chỉ có 2 người, tức ta chỉ cần tính xác suất để người thứ nhất có điểm thấp hơn người thứ hai, rồi nhân với tiền thưởng tương ứng.

Với người thứ i , xác suất điểm của người đó rơi vào đoạn $[l, r]$, ký hiệu $P_i(l, r)$, là

$$P_i(l, r) = \frac{\int_l^r f_i(x) dx}{\int_0^1 f_i(x) dx}. \quad (1)$$

Trực giác hình học là diện tích dưới đồ thị của hàm trên đoạn $[l, r]$ chia cho diện tích dưới đồ thị của hàm trên đoạn $[0, 1]$. Hình trong nguồn minh họa hai hàm $f(x) = 1$ và $g(x) = 2x$; phần tô màu chính là phần diện tích ứng với khoảng điểm đang xét.

Vì nhân một hàm với một hằng số không làm thay đổi đáp án, ta có thể chuẩn hóa mỗi hàm bằng cách chia nó cho diện tích dưới đồ thị trên $[0, 1]$. Khi đó

$$P_i(l, r) = \int_l^r f_i(x) dx. \quad (2)$$

Trước hết giả sử đáp án không cần lấy modulo 998244353. Xét một cách làm chưa hoàn hảo: chọn một độ dài rất nhỏ ε , chia đoạn điểm thành các khoảng độ dài ε , rồi liệt kê khoảng chứa điểm của bạn. Bằng cách tính xác suất điểm của người còn lại nằm dưới điểm của bạn và cộng lại, ta thu được xác suất bạn vượt người đó:

$$P \approx \sum_{k\varepsilon < 1} P_1(0, k\varepsilon - \varepsilon) P_2(k\varepsilon - \varepsilon, k\varepsilon). \quad (3)$$

Theo dạng tích phân,

$$P \approx \sum_{k\varepsilon < 1} \left(\int_0^{k\varepsilon - \varepsilon} f_1(x) dx \right) \left(\int_{k\varepsilon - \varepsilon}^{k\varepsilon} f_2(x) dx \right). \quad (4)$$

Khi ε tiến tới 0, phần $\int_{k\varepsilon - \varepsilon}^{k\varepsilon} f_2(x) dx$ tương ứng với đạo hàm của nguyên hàm tại $k\varepsilon$, tức $f_2(k\varepsilon)$. Gọi $F_i(x)$ là nguyên hàm chuẩn hóa của $f_i(x)$, ta có

$$P = \int_0^1 F_1(x) f_2(x) dx. \quad (5)$$

Vì trong bài này mọi hàm đều là đa thức, nếu

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m,$$

thì

$$F(x) = a_0x + \frac{a_1}{2}x^2 + \frac{a_2}{3}x^3 + \dots + \frac{a_m}{m+1}x^{m+1}.$$

Do đó ta chỉ cần nhân $F_1(x)$ với $f_2(x)$, rồi lấy tích phân xác định một lần nữa.

Độ phức tạp thời gian là $O(S^2)$, độ phức tạp bộ nhớ là $O(S)$.

8.5.2 Thuật toán 2

Với 20% dữ liệu tiếp theo, $n = 10$. Khi đó có thể liệt kê tập những người có điểm thấp hơn bạn và tập những người có điểm cao hơn bạn.

Gọi $D = \{1, 2, \dots, n-1\}$. Giả sử tập những người bị bạn vượt qua là A , thì các người còn lại là $D \setminus A$. Áp dụng cách tính của thuật toán 1, xác suất của cầu hình A là

$$P_A = \int_0^1 f_n(x) \prod_{i \in A} F_i(x) \prod_{i \in D \setminus A} (1 - F_i(x)) dx. \quad (6)$$

Sau đó chỉ cần đếm số người bạn vượt qua, đổi ra thứ hạng tương ứng và cộng vào kỳ vọng.

Độ phức tạp thời gian là $O(2^n S^2)$, độ phức tạp bộ nhớ là $O(S)$.

8.5.3 Thuật toán 3

Quan sát tổng xác suất khi số người bị bạn vượt qua bằng j :

$$P_j = \sum_{\substack{A \subseteq \{1, 2, \dots, n-1\} \\ |A|=j}} \int_0^1 f_n(x) \prod_{i \in A} F_i(x) \prod_{i \in D \setminus A} (1 - F_i(x)) dx. \quad (7)$$

Đưa tích phân ra ngoài, trước hết cần tính

$$\sum_{\substack{A \subseteq \{1, 2, \dots, n-1\} \\ |A|=j}} \prod_{i \in A} F_i(x) \prod_{i \in D \setminus A} (1 - F_i(x)). \quad (8)$$

Sau đó nhân với $f_n(x)$ rồi lấy tích phân là được.

Dễ thấy mỗi $F_i(x)$ và $1 - F_i(x)$ đều là đa thức. Đặt trạng thái $g_{i,j}(x)$ là đa thức sau khi đã xét tới người thứ i , trong đó có j người có điểm thấp hơn bạn. Khi đó có truy hồi

$$g_{i,j}(x) = F_i(x)g_{i-1,j-1}(x) + (1 - F_i(x))g_{i-1,j}(x). \quad (9)$$

Tiếp theo chỉ cần nhân đa thức trực tiếp để hoàn thành quy hoạch động. Dùng mảng cuộn có thể tối ưu bộ nhớ xuống $O(nS)$.

Độ phức tạp thời gian là $O(nS^2)$, độ phức tạp bộ nhớ là $O(nS)$.

8.6 Dữ liệu bonus

8.6.1 Dữ liệu 11

Khi tiền thưởng của mọi thứ hạng bằng nhau, dù đạt thứ hạng nào bạn cũng chắc chắn nhận đúng số tiền đó, nên có thể in trực tiếp giá trị tiền thưởng.

Độ phức tạp thời gian và bộ nhớ đều là $O(1)$.

8.6.2 Dữ liệu 12

Khi hàm xác suất của mọi người giống nhau, xác suất để mỗi người đạt hạng i là như nhau. Vì vậy ta chỉ cần lấy trung bình các mức thưởng.

Độ phức tạp thời gian là $O(n)$, độ phức tạp bộ nhớ là $O(1)$.

8.6.3 Dữ liệu 13–14

Với hai nhóm dữ liệu 13 và 14, chỉ cần tính xác suất đạt hạng nhất và hạng cuối, rồi nhân với chênh lệch giữa tiền thưởng của chúng và tiền thưởng chung của các hạng 2 đến $n - 1$.

Xác suất đạt hạng nhất là

$$P_{\text{first}} = \int_0^1 \prod_{i=1}^{n-1} F_i(x) f_n(x) dx. \quad (10)$$

Xác suất đạt hạng cuối là

$$P_{\text{last}} = \int_0^1 \prod_{i=1}^{n-1} (1 - F_i(x)) f_n(x) dx. \quad (11)$$

Ta chỉ cần nhân đa thức trực tiếp.

Độ phức tạp thời gian là $O(nS^2)$, độ phức tạp bộ nhớ là $O(S)$.

8.6.4 Một cách tốt hơn

Khi tính xác suất hạng nhất và hạng cuối, việc chính cần làm là tính tích của n đa thức. Vì modulo của bài có dạng thuận lợi, rất tự nhiên nghĩ tới việc dùng biến đổi Fourier nhanh trên trường hữu hạn để tối ưu phép nhân đa thức.

Lấy k sao cho $2^k < n$, thực hiện DFT trên $f_i(x)$, tức dùng một mảng để ghi giá trị của hàm tại các điểm $w^0, w^1, w^2, \dots, w^{2^k-1}$.

Nếu $C(x) = A(x) + B(x)$, sau DFT ta có $C_i = A_i + B_i$. Nếu $C(x) = A(x)B(x)$, sau DFT ta có $C_i = A_i B_i$. Vì vậy khi cần tính toán trên đa thức, có thể DFT hai đa thức, thực hiện phép toán theo từng điểm, rồi dùng DFT ngược để chuyển về hệ số.

Thời gian biến đổi là $O(S \log S)$, thời gian tính theo từng điểm là $O(S)$. Nếu dùng FFT trực tiếp cho từng lần nhân, cần biến đổi $2n$ lần và tính n lần, độ phức tạp trở thành $O(nS \log S + nS)$, chưa tốt.

Chú ý rằng nếu cứ nhân các đa thức theo thứ tự, cuối cùng sẽ có một đa thức độ dài $O(S)$ nhân với một đa thức độ dài $O(1)$, không tối ưu. Ta có thể mỗi lần chia các đa thức thành hai nửa, nhân tích ở hai nửa rồi mới nhân hai kết quả. Khi đó

$$T(S) = 2T\left(\frac{S}{2}\right) + O(S \log S), \quad (12)$$

nên tổng độ phức tạp là $O(S \log^2 S)$.

Độ phức tạp thời gian là $O(S \log^2 S)$, độ phức tạp bộ nhớ là $O(S)$.

8.7 Thuật toán 4

Tương tự, khi truy hồi để tính các $g_{i,j}$, ta cũng có thể dùng FFT để tối ưu.

Sau khi DFT các F_i , phương trình chuyển (9) trở thành

$$\hat{g}_{i,j,k} = \hat{F}_{i,k} \hat{g}_{i-1,j-1,k} + (1 - \hat{F}_{i,k}) \hat{g}_{i-1,j,k}. \quad (13)$$

Bây giờ cố định k , tức hoán đổi chiều thứ hai và thứ ba của toàn bộ g . Khi đó, nếu $G_{i,k}(y)$ là hàm sinh theo chỉ số j , truy hồi trở thành

$$G_{i,k}(y) = (\hat{F}_{i,k}y + 1 - \hat{F}_{i,k})G_{i-1,k}(y). \quad (14)$$

Với $G_{0,k}(y) = 1$, suy ra

$$G_{n-1,k}(y) = \prod_{i=1}^{n-1} (\hat{F}_{i,k}y + 1 - \hat{F}_{i,k}). \quad (15)$$

Quan sát công thức trên: với mỗi k , nếu biết các giá trị $\hat{F}_{i,k}$, ta có thể dùng chia để trị kèm FFT như trong mục trước để tính $G_{n-1,k}(y)$ trong $O(n \log^2 n)$ thời gian. Các $\hat{F}_{i,k}$ lại có thể được tính trực tiếp bằng FFT trong $O(nS \log S)$ thời gian.

Sau khi tính được mọi $\hat{g}_{n-1,j,k}$, dùng DFT ngược để chuyển về các đa thức $g_{n-1,j}(x)$.

Độ phức tạp thời gian là $O(nS \log^2 n)$, độ phức tạp bộ nhớ là $O(nS)$.

8.8 Thuật toán 5

Nhìn từ góc độ khác, nếu trực tiếp áp dụng hàm sinh cho truy hồi (9), thì với hàm sinh hai biến $G_i(x, y)$ của các $g_{i,j}(x)$ ta có

$$G_i(x, y) = (F_i(x)y + 1 - F_i(x))G_{i-1}(x, y). \quad (16)$$

Vì $G_0(x, y) = 1$, suy ra

$$G_{n-1}(x, y) = \prod_{i=1}^{n-1} (F_i(x)y + 1 - F_i(x)). \quad (17)$$

Như vậy, cuối cùng ta cần tính tích của một số đa thức hai biến.

DFT của đa thức hai biến tương đương với việc liệt kê các giá trị khác nhau của cả x và y . Ta có thể trước hết cố định y , thực hiện FFT theo biến x , rồi với từng giá trị của x , tiếp tục thực hiện FFT theo biến y . Với một đa thức hai biến kích thước $N \times M$, thời gian FFT là

$$O(NM \log NM).$$

Sau khi có DFT hai biến, ta tiếp tục dùng cùng ý tưởng chia để trị kèm FFT để giải quyết bài toán hiện tại.

Cần chú ý rằng nếu quy mô bài toán là $n \times S$, do dữ liệu được sinh ngẫu nhiên, khi chia n đa thức thành hai nửa có kích thước bằng nhau, số hạng của hai tích thu được cũng xấp xỉ nhau. Vì vậy thời gian thỏa

$$T(n, S) = 2T\left(\frac{n}{2}, \frac{S}{2}\right) + O(nS \log nS). \quad (18)$$

Suy ra độ phức tạp của chia để trị kèm FFT là $O(nS \log nS)$.

Cuối cùng, sau khi có các đa thức $g_{n-1,j}(x)$, chỉ cần nhân từng đa thức với $f_n(x)$, lấy tích phân và cộng theo tiền thường tương ứng.

Tới đây bài toán được giải quyết hoàn chỉnh.

Độ phức tạp thời gian là $O(nS \log nS)$, độ phức tạp bộ nhớ là $O(nS)$.

8.9 Lời cảm ơn

- Xin cảm ơn thầy Qu Yunhua đã quan tâm và chăm sóc tôi trong học tập cũng như sinh hoạt.
- Xin cảm ơn cha mẹ vì công ơn nuôi dưỡng.
- Xin cảm ơn các thầy Zhu Quanmin, Liao Xiaogang và Wang Xingming vì sự chỉ dạy.
- Xin cảm ơn CCF đã cung cấp nền tảng và cơ hội.
- Xin cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự tận tâm.
- Xin cảm ơn anh Huang Zhi'ao của Đại học Thanh Hoa đã giúp đỡ tôi.
- Xin cảm ơn các bạn Kuang Zhengfei, Yang Dingcheng và Liu Yanyi đã đọc kiểm tra bản thảo.
- Xin cảm ơn các bạn Yu Jiping, Lyu Kaifeng và Du Yuhao đã góp ý trong quá trình ra đề.
- Xin cảm ơn tất cả các bạn học đã từng giúp đỡ tôi.

9 Bàn về ứng dụng chia khối trong một lớp bài toán trực tuyến

Zou Daoyao

Trường Trung học Trần Hải, Ninh Ba

Tóm tắt

Chia khối là một phương pháp có tính thích dụng cao, dễ cài đặt và độ khó tư duy thấp. Nó có thể dùng để xử lý một số bài mà thông tin không thể hợp nhất nhanh. Bài viết giới thiệu vài cách chia khối thường gặp, rồi thông qua một số ví dụ trình bày sơ lược ứng dụng của chia khối trong các bài toán cần duy trì hoặc truy vấn trực tuyến.

9.1 Dẫn nhập

Trong thi đấu và thực hành thường xuất hiện một lớp bài yêu cầu duy trì dãy hoặc cấu trúc cây, hơn nữa thường có thao tác sửa đổi. Có rất nhiều cấu trúc dữ liệu có thể duy trì các trường hợp đơn giản với độ phức tạp cấp log. Tuy nhiên khi gặp yêu cầu phức tạp hơn, các cấu trúc dữ liệu này thường vướng khó khăn: cần lồng ghép nhiều cấu trúc, cần khai thác tính chất sâu của thông tin cần duy trì, thậm chí có lúc hoàn toàn không xử lý được.

Khi gặp lớp bài như vậy, ưu thế của chia khối thể hiện rất rõ: không cần phân tích bài toán quá sâu, nhưng một phương pháp có thể duy trì rất nhiều yêu cầu khác nhau. Dù độ phức tạp của chia khối thường ở cấp căn hoặc cao hơn, độ phức tạp tư duy và độ phức tạp mã nguồn của nó thường thấp hơn nhiều so với các cấu trúc dữ liệu có độ phức tạp lý thuyết tốt hơn, nên trong thực tế rất hữu dụng.

Dĩ nhiên lớp bài này cũng có một số thuật toán offline tốt, chẳng hạn thuật toán Mo, chia để trị CDQ, v.v. Nhưng nếu đề bài bắt buộc trực tuyến thì những thuật toán đó không còn phát huy được.

Bài viết này giới thiệu ngắn gọn vài cách thường dùng để duy trì cấu trúc chia khối trực tuyến hoặc tiền xử lý cấu trúc chia khối để trả lời truy vấn trực tuyến. Mục 1 chủ yếu nói về cách trực tiếp chia dãy hoặc cây thành nhiều khối để duy trì. Mục 2 chủ yếu nói về cách tách thông tin hoặc tách truy vấn theo kích thước để xử lý riêng. Mục 3 chủ yếu nói về ý tưởng xây dựng lại định kỳ.

Nếu không nói riêng, các bài toán dưới đây đều yêu cầu thuật toán trực tuyến.

9.2 Chia khối trực tiếp

9.2.1 Chia khối trên dãy

Chia khối trên dãy là một trong những loại bài chia khối thường gặp nhất và dễ cài đặt nhất trong thi thuật toán.

Ý tưởng cơ bản là trên dãy, cứ cách $H(N)$ điểm chọn một điểm mốc. Khi đó một truy vấn đoạn bất kỳ có thể bị tách thành $O(N/H(N))$ đoạn và $O(H(N))$ điểm lẻ. Lúc truy vấn không cần quét toàn bộ mảng, và lúc sửa đổi cũng không cần sửa toàn bộ mảng.

Ví dụ 1: Bouncing Sheep. Cho N số a_i , thỏa $i < a_i \leq N + 1$, và N thao tác. Mỗi thao tác có thể:

- sửa một số, sau khi sửa vẫn thỏa điều kiện trên;
- hỏi nếu bắt đầu từ $x = i$, mỗi lần đổi x thành a_x , thì cần bao nhiêu lần đổi để tới $x = N + 1$.

Bài này có thể dùng link-cut tree để giải trong $O(N \log N)$, nhưng điều đó không liên quan nhiều tới bài viết này nên bỏ qua.

Trước hết xét hai cách vét cạn. Một là sửa thì chỉ sửa vị trí đó, hỏi thì mô phỏng $O(N)$. Hai là lúc sửa tính lại đáp án cho mọi vị trí trong $O(N)$, hỏi thì trả lời $O(1)$.

Ta có thể dùng chia khối để cân bằng hai độ phức tạp này. Trước hết chia dãy thành các khối liên tiếp, mỗi khối có $H(N)$ phần tử; chẳng hạn khi $N = 18$ và $H(N) = 4$, các phần tử cùng màu trong hình nguồn thuộc cùng một khối.

Với mỗi khối, dùng cách vét cạn thứ hai để tính đáp án cho mọi vị trí trong khối. Khi sửa, chỉ cần tính lại toàn bộ khối đó, mất $O(H(N))$. Khi hỏi, dùng cách vét cạn thứ nhất; nhờ thông tin tiền xử lý, mỗi lần ít nhất nhảy qua được một đoạn, nên thời gian là $O(N/H(N))$.

Chú ý rằng trong tiền xử lý chỉ cần tính với mỗi vị trí: cần bao nhiêu bước để nhảy ra khỏi khối chứa nó, và sau khi nhảy ra thì rơi vào vị trí nào. Khi đó lúc truy vấn, từ điểm hiện tại có thể nhảy qua cả đoạn trong $O(1)$.

Để cân bằng sửa đổi và truy vấn, chọn $H(N) = \sqrt{N}$. Khi đó bài toán được giải trong $O(N\sqrt{N})$.¹⁴

Ví dụ 2: Chef and Churu. Cho N số a_i và N hàm f_i . Mỗi hàm biểu diễn tổng của a_i trên một đoạn, tức

$$f_i = \sum_{x=l_i}^{r_i} a_x,$$

và giá trị của f_i thay đổi theo các giá trị a_i . Có N thao tác, mỗi thao tác có thể:

- sửa một số a_i ;
- hỏi $\sum_{x=l_i}^{r_i} f_x$.

Nếu duy trì một mảng cây, ta có thể tính một giá trị f_i trong $O(\log N)$, nên có thể cân nhắc chia khối theo f_i .

Tương tự bài trước, cứ cách $H(N)$ hàm f_i chia thành $N/H(N)$ khối. Với mỗi khối, tiền xử lý số lần mỗi a_i xuất hiện trong khối đó, cùng tổng các f_i của khối ở trạng thái ban đầu. Thời gian tiền xử lý là $O(NH(N))$.

Với mỗi sửa đổi, cập nhật tổng f_i của từng đoạn là được; vì đã tiền xử lý số lần xuất hiện của a_i , điều này rất dễ làm. Thời gian là $O(N/H(N))$. Với mỗi truy vấn, tách đoạn hỏi thành $O(H(N))$ điểm lẻ và $O(N/H(N))$ đoạn nguyên khối. Mỗi điểm lẻ có thể giải trong $O(\log N)$ bằng mảng cây, mỗi đoạn nguyên khối giải trong $O(1)$.

Tuy nhiên duy trì mảng cây là $O(\log N) - O(\log N)$, trong khi lúc sửa đổi ta hoàn toàn có thể chấp nhận độ phức tạp cao hơn mà không ảnh hưởng độ phức tạp tổng thể. Vì vậy có thể dùng một cấu trúc $O(\sqrt{N}) - O(1)$ hỗ trợ sửa điểm và hỏi tổng đoạn để duy trì a_i .

Do chỉ cần hỏi tổng đoạn, tính chất trừ được cho phép đổi tổng đoạn thành hiệu hai tổng tiền tố. Sửa một điểm sẽ ảnh hưởng tới các truy vấn tiền tố tạo thành một hậu tố, nên bài toán sửa điểm, hỏi tiền tố có thể đổi thành sửa hậu tố, hỏi điểm. Cụ thể, chia a_i thành $O(\sqrt{N})$ khối, mỗi khối $O(\sqrt{N})$ phần tử. Mỗi lần sửa chỉ cần sửa $O(\sqrt{N})$ khối và $O(\sqrt{N})$ điểm lẻ; mỗi lần hỏi chỉ cần cộng đáp án trong khối với đáp án điểm lẻ.

Như vậy truy vấn và sửa đổi đều đạt

$$O\left(\frac{N}{H(N)} + H(N)\right).$$

Khi $H(N) = \sqrt{N}$, thời gian là $O(N\sqrt{N})$, bộ nhớ là $O(N\sqrt{N})$.¹⁵

Ví dụ 3: Chef and Problems. Cho N số a_i và N truy vấn. Mỗi lần cho l, r , hỏi giá trị lớn nhất của $j - i$ sao cho $a_i = a_j$ và $l \leq i, j \leq r$.

Trước hết xét cách vét cạn $O(N)$ cho mỗi truy vấn: quét đoạn hỏi, dùng mỗi số và vị trí xuất hiện đầu tiên của số đó trong đoạn để cập nhật đáp án. Dùng dấu thời gian khi cập nhật sẽ tránh được việc mỗi lần phải quét cả mảng, chỉ cần quét độ dài đoạn hỏi.

Ta có thể làm như bài trước: cứ mỗi $H(N)$ phần tử của a_i chia thành $N/H(N)$ khối, rồi dùng vét cạn tính đáp án bên trong từng khối. Thời gian là $O(N)$.

Nhưng bài này không thể trực tiếp hợp nhất thông tin trong khối như bài trước; còn cần tính đáp án cho một đoạn gồm nhiều khối. Trước hết tiền xử lý với mỗi a_i , vị trí nhỏ nhất j có cùng giá trị và $j > kH(N)$; tương tự tính vị trí lớn nhất j có cùng giá trị và $j \leq kH(N)$. Thời gian là $O(N \cdot N/H(N))$.

¹⁴Nguồn bài: HNOI 2010.

¹⁵Nguồn bài: CodeChef November 2014 Challenge.

Từ đó có thể tính đáp án lớn nhất khi i nằm ở khối thứ x , j nằm trong các khối từ x đến y ; ký hiệu là $\text{calc}(x, y)$, mất $O(NH(N))$.

Đáp án từ khối thứ x đến khối thứ y còn có thể có hai đầu mút không nằm ở khối x hoặc khối y , nên cần dùng một DP theo đoạn để tính phần này. Gọi $f(i, j)$ là đáp án giữa khối i và khối j . Khi đó

$$f(i, j) = \max\{f(i + 1, j), f(i, j - 1), \text{calc}(i, j)\}.$$

Sau đó xử lý truy vấn trở nên thuận tiện hơn. Mỗi đoạn có thể tách thành một đoạn gồm các khối đã tính đáp án và không quá $2H(N)$ điểm lẻ. Quét các điểm thừa trong $O(H(N))$ để tính đáp án có cả hai đầu ở bên ngoài; rồi dùng thông tin tiền xử lý để tính đáp án lớn nhất giữa mỗi điểm ngoài và điểm trong đoạn khối. Lấy lớn nhất của ba phần là được. Thời gian mỗi truy vấn là $O(H(N))$.

Tổng thời gian là

$$O\left(N\left(H(N) + \frac{N}{H(N)}\right)\right) - O(H(N)),$$

bộ nhớ là $O(N \cdot N/H(N))$. Khi $H(N) = \sqrt{N}$, thời gian là $O(N\sqrt{N})$. Tuy nhiên để tiết kiệm bộ nhớ, có thể chọn $H(N)$ lớn hơn một chút.

Những bài như ví dụ 3 không thuận tiện để thêm thao tác sửa, vì mỗi lần sửa có thể làm thay đổi $O((N/H(N))^2)$ giá trị tiền xử lý. Nhưng nếu thêm sửa đổi, có thể chọn $H(N) = O(N^{3/5})$ để đạt độ phức tạp $O(N^{7/5})$.¹⁶

9.2.2 Chia khối trên cây

Nhiều bài rất dễ cài đặt khi chia khối trên đường thẳng, nhưng trên cây thì phiền hơn. Mục này giới thiệu một cách chia khối trên cây rất dễ cài đặt, có thể xử lý một loạt bài chỉ truy vấn thông tin trên đường đi hoặc quan hệ tổ tiên.

Giải truy vấn thông tin trên đường đi. Ví dụ 4: *Children Trips*. Cho một cây N đỉnh, mỗi cạnh có độ dài 1 hoặc 2. Có N truy vấn; mỗi lần cho x, y, z , yêu cầu tìm ít điểm nhất $a_1, \dots, a_n$ sao cho

$$\text{dis}(x, a_1), \text{dis}(a_1, a_2), \dots, \text{dis}(a_n, y)$$

đều không vượt quá z . Ở đây $\text{dis}(i, j)$ là khoảng cách giữa đỉnh i và đỉnh j trên cây.¹⁷

Thông tin của bài này rất khó hợp nhất, vì phép hợp nhất không có tính kết hợp, chỉ có thể hợp nhất từ trước ra sau. Nhưng có thể quan sát hai tính chất sau.

Tính chất 1. Nếu chọn a_i theo thứ tự tăng dần của i , mỗi lần chọn điểm xa nhất trên đường từ a_{i-1} tới y thỏa điều kiện làm a_i (đặt $x = a_0, y = a_{n+1}$), thì số điểm chọn được là nhỏ nhất. Nói cách khác, mỗi lần đi bước dài nhất có thể không làm đáp án xấu đi.

Tính chất 2. Nếu một số điểm đầu được chọn xa nhất theo thứ tự tăng của i đến a_x , còn một số điểm cuối được chọn xa nhất theo thứ tự giảm của i đến a_x , thì cách chọn này cũng cho số điểm nhỏ nhất. Nói cách khác, đồng thời đi từ hai đầu vào trong không làm đáp án xấu đi.

Xét cách chọn một số điểm mốc trên cây giống chia khối trên dãy. Trước hết tính độ sâu dep_i của mỗi đỉnh (gốc có độ sâu 0) và kích thước cây con size_i , rồi chọn các đỉnh thỏa $\text{dep}_i \bmod H(N) = 0$.

¹⁶Nguồn bài: CodeChef March 2015 Challenge.

¹⁷Nguồn bài: CodeChef October 2014 Challenge.

Tuy nhiên cách này có thể chọn ra quá nhiều điểm, chẳng hạn một đường dài $H(N) - 1$ lại mọc thêm $O(N)$ điểm ở một đoạn của đường. Vì thế cần loại bỏ các điểm có kích thước cây con không quá $H(N)$. Hình nguồn minh họa các điểm mốc còn lại.

Hình b. Bản gốc vẽ cây theo tầng sâu; các điểm được chọn làm điểm mốc nằm ở những độ sâu chia hết cho $H(N)$ và có cây con đủ lớn.

Sau đó lấy toàn bộ điểm mốc dựng thành một cây, gọi là “cây mốc”. Trong cây mốc, cha của một điểm là điểm mốc đầu tiên gặp được khi đi lên trên cây gốc; mỗi cạnh tương ứng với một đường đi giữa hai điểm mốc trong cây gốc. Đồng thời mỗi điểm mốc đại diện cho một tập điểm: tập này chứa mọi điểm mà khi đi về gốc, điểm mốc đầu tiên gặp được là nó. Hình nguồn phân loại các tập điểm; điểm đặc là điểm mốc, các điểm cùng màu thuộc cùng một tập.

Ta phân tích các tính chất của cấu trúc này.

Tính chất 3. Hai điểm mốc kề nhau bất kỳ trên cây mốc cách nhau $H(N)$ trên cây gốc.

Tính chất 4. Tập điểm do mỗi điểm mốc đại diện có kích thước không nhỏ hơn $H(N)$.

Chứng minh. Nếu điểm mốc này là lá trong cây mốc, thì mọi con cháu của nó trong cây gốc đều thuộc tập điểm này, nên theo định nghĩa tính chất đúng. Nếu nó không phải lá trong cây mốc, thì đường từ nó tới một con của nó trong cây mốc có $H(N)$ điểm, và các điểm đó hiển nhiên thuộc tập mà nó đại diện.

Tính chất 5. Số điểm mốc được chọn nhiều nhất là $O(N/H(N))$.

Tính chất 6. Mỗi điểm đi lên nhiều nhất $2H(N)$ bước sẽ gặp một điểm mốc.

Chứng minh. Giả sử đi $2H(N)$ bước vẫn chưa gặp điểm mốc. Khi đó trong các điểm đã đi qua có ít nhất hai điểm có độ sâu chia hết cho $H(N)$. Điểm gần gốc hơn trong hai điểm đó rõ ràng có cây con lớn hơn $H(N)$, nên chắc chắn được chọn làm điểm mốc, mâu thuẫn.

Tính chất 7. Đường đi giữa hai điểm bất kỳ có thể tách thành $O(H(N))$ cạnh của cây gốc và $O(N/H(N))$ cạnh của cây mốc.

Chứng minh. Không thể có một điểm đi liên tiếp lên cha $2H(N)$ bước mà chưa gặp điểm mốc, nên phần cạnh cây gốc nhiều nhất chỉ có $4H(N)$ cạnh. Còn tổng số cạnh trên cây mốc chỉ là $O(N/H(N))$, nên không thể nhiều hơn mức này.

Nếu đã tiền xử lý đáp án giữa hai điểm mốc kề nhau trên cây mốc, truy vấn sẽ dễ xử lý. Chú ý rằng ở đây chỉ cần duy trì đáp án khi $z = 1, \dots, 2H(N)$. Nếu $z > 2H(N)$, đoạn đó sẽ bị nhảy qua trực tiếp nên không cần tính. Trước hết, với mỗi điểm và mỗi bước z , tính điểm đầu tiên đạt được khi đi lên với độ dài bước là z , cần bao nhiêu bước để nhảy tới điểm mốc gần nhất, và sau khi nhảy tới đó thì bước cuối còn dư bao nhiêu. Độ phức tạp tiền xử lý là $O(NH(N))$.

Khi truy vấn, chỉ cần dùng tính chất 2 và làm tương tự thuật toán nhảy bội để tìm LCA, liên tục đưa hai điểm về phía gốc. Chú ý LCA không nhất thiết là điểm mốc, nên nếu nhảy thêm một bước sẽ vượt quá LCA thì dùng vét cạn để nhảy nốt đoạn cuối. Trong quá trình di chuyển, duy trì số bước đã đi và độ dài còn lại của bước cuối. Khi di chuyển một bước lớn, trước hết dùng hết phần độ dài còn dư, rồi dùng thông tin tiền xử lý để đi một bước tới điểm mốc phía trên. Thời gian là $O(N/H(N))$.

Khi $H(N) = \sqrt{N}$, thời gian là $O(N\sqrt{N})$, bộ nhớ là $O(N\sqrt{N})$.

Giải truy vấn quan hệ tổ tiên. Ví dụ 5: *Regions*. Cho một cây N đỉnh, mỗi đỉnh có một màu. Mỗi lần cho x, y , hỏi có bao nhiêu cặp i, j thỏa màu của i là x , màu của j là y , và i là tổ tiên của j .¹⁸

Ta dựng một cây mốc như bài trước, nhưng cây mốc này chưa có những tính chất để phân hoạch theo quan hệ tổ tiên, nên cần mở rộng nó.

Trước hết, như bài trước, chọn các điểm mốc “ban đầu” thỏa $\text{dep}_i \bmod H(N) = 0$ và $\text{size}_i \geq H(N)$. Sau đó chọn thêm mọi điểm là LCA của hai điểm mốc ban đầu, đưa vào tập điểm mốc cuối cùng.

Tính chất 1. Số điểm mốc tăng nhiều nhất gấp đôi.

Chứng minh. Sắp xếp các điểm mốc ban đầu theo thứ tự DFS, rồi lần lượt xét từng điểm mốc ban đầu. Với mỗi điểm, liên tục đi lên phía gốc cho tới khi gặp một điểm mốc ban đầu khác hoặc một điểm đã được thăm. Nếu dừng ở một điểm mốc ban đầu, thì phần đi lên tiếp theo hoàn toàn giống với trường hợp của điểm đó, không cần đi nữa. Nếu dừng ở một điểm đã thăm, thêm điểm đó vào tập điểm mốc; phần đi lên tiếp theo hoàn toàn giống với trường hợp mà điểm lần đầu ghé tới nó đã gặp, cũng không cần đi nữa. Vì vậy mỗi điểm nhiều nhất chỉ đóng góp một điểm mới.

Thực ra cách làm này chính là dựng một “cây ảo”. Khi cài đặt có thể thêm trực tiếp LCA của mọi cặp điểm kề nhau theo thứ tự DFS rồi khử trùng. Thảo luận chi tiết về cây ảo có thể xem trong luận văn đội tuyển quốc gia năm 2014 của Xu Yinzhan.

Sau khi mở rộng tập điểm mốc, định nghĩa cây mốc vẫn như trước: cha của mỗi điểm là tổ tiên mốc gần nhất của nó trên cây gốc; cạnh trong cây mốc đại diện cho tập điểm nằm trên một đường trong cây gốc. Lúc này cây thỏa thêm các tính chất sau.

Tính chất 2. Hai cạnh bất kỳ của cây mốc, khi xét các đường mà chúng đại diện trong cây gốc, chỉ giao nhau ở đầu nút.

Tính chất 3. Đường đi giữa hai điểm bất kỳ trong cây có thể chia thành $O(H(N))$ cạnh và $O(N/H(N))$ đường giữa hai điểm mốc kề nhau.

Tính chất 4. Độ dài đường giữa hai điểm mốc kề nhau bất kỳ không vượt quá $H(N)$, và số điểm mốc không vượt quá $2N/H(N)$.

Hình c. Bản gốc minh họa các điểm mốc mới: màu hồng là điểm mốc ban đầu, màu xanh là điểm mốc thêm vào, màu nâu là các điểm nằm trên cạnh của cây mốc.

Nhờ các tính chất này, bài toán trở nên dễ xử lý. Chia mọi điểm thành hai loại: điểm nằm trên cạnh của cây mốc, kể cả điểm mốc, gọi là “điểm trong”; các điểm còn lại gọi là “điểm ngoài”. Khi đó các trường hợp cần thống kê được chia thành ba loại:

- cả hai điểm đều là điểm trong;
- điểm tổ tiên là điểm trong, điểm con cháu là điểm ngoài, nhưng điểm tổ tiên không nằm trên đường từ điểm con cháu đi lên điểm mốc đầu tiên;

¹⁸Nguồn bài: IOI 2009.

- điểm tổ tiên là điểm trong, điểm con cháu là điểm ngoài, và điểm tổ tiên nằm trên đường từ điểm con cháu đi lên điểm mốc đầu tiên.

Trường hợp thứ ba rất dễ xử lý: vì mỗi điểm đi lên nhiều nhất $2H(N)$ bước sẽ gặp một điểm mốc, chỉ cần vết cạn là tiền xử lý được đáp án cho trường hợp này.

Trên cây mốc, gom mỗi điểm mốc và các điểm trên đường từ nó tới cha của nó thành một khối; khối của gốc chỉ chứa chính gốc. Với mỗi khối, tiền xử lý số lượng của từng màu bên trong khối. Việc này có thể làm trong thời gian $O(N \cdot N/H(N))$.

Để chứng minh nếu điểm mốc x là tổ tiên của điểm mốc y , thì mọi điểm trong khối của x đều là tổ tiên của mọi điểm trong khối của y . Vì vậy lúc truy vấn, chỉ cần duyệt cây mốc một lần là giải được trường hợp thứ nhất.

Cuối cùng xét trường hợp thứ hai. Tiền xử lý với mỗi điểm ngoài: điểm mốc đầu tiên gặp được khi đi lên, rồi đưa điểm ngoài này vào tập của điểm mốc đó. Khi đó có thể xử lý tương tự trường hợp thứ nhất: tiền xử lý số lần xuất hiện của mỗi màu trong tập điểm ngoài do từng điểm mốc đại diện. Lúc truy vấn, khi DFS tới một điểm, dùng số lượng màu x trên đường từ điểm đó tới gốc và số lượng màu y trong tập của điểm đó để cập nhật đáp án.

Khi $H(N) = \sqrt{N}$, thời gian của bài này là $O(N\sqrt{N})$, bộ nhớ là $O(N\sqrt{N})$.

9.3 Thảo luận theo kích thước

Ý tưởng cơ bản của thảo luận theo kích thước là chia truy vấn, sửa đổi, bậc, v.v. thành hai loại: lớn hơn $H(N)$ và không lớn hơn $H(N)$, rồi xử lý riêng. Tức là kết hợp hai cách vết cạn kiểu $O(\text{size})$, hoặc $O(\text{size} \log \text{size})$, với $O(N/\text{size})$. Vì vậy độ khó tư duy thấp, và do kết hợp hai cách vết cạn nên trong phòng thi cũng dễ cài.

9.3.1 Ví dụ 4: *Children Trips*

Vẫn là bài trên cây N đỉnh, mỗi cạnh dài 1 hoặc 2; mỗi truy vấn cho x, y, z , yêu cầu tìm ít điểm nhất để khoảng cách giữa hai điểm liên tiếp trên đường từ x tới y đều không vượt quá z .

Xét cách vết cạn: mỗi lần nhị phân vị trí của a_i tiếp theo. Cách này là $O(N \log N)$, vì số a_i có thể nhiều nhất $O(N)$. Nhưng khi $z > H(N)$, số a_i nhiều nhất chỉ là $O(N/H(N))$, có thể vết cạn trực tiếp; còn khi $z \leq H(N)$, chỉ có không quá $H(N)$ loại z . Nếu tiền xử lý đáp án cho các z nhỏ này thì xong.

Với mỗi z nhỏ, tiền xử lý với từng điểm: đi về gốc một đoạn dài z sẽ tới điểm nào; nếu không có điểm cách đúng z thì chọn điểm cách $z - 1$. Sau đó dùng nhảy bội để biết mỗi điểm đi lên 2^k bước, mỗi bước dài z , thì tới đâu. Việc này cần thời gian và bộ nhớ $O(NH(N) \log N)$. Khi truy vấn, có thể như thuật toán nhảy bội thông thường: đi lên tới khi nhảy tiếp sẽ vượt quá LCA thì dừng; đoạn dư cuối cùng vết cạn trong $O(H(N))$.

Chọn $H(N) = \sqrt{N}$, bài toán được giải trong thời gian $O(N\sqrt{N} \log N)$ và bộ nhớ $O(N\sqrt{N} \log N)$.

Đây là lời giải chính thức của bài, độ khó tư duy khá thấp và dễ nghĩ ra. Tuy nhiên nhị phân giữa hai điểm trên cây tương đối phiền và không dễ cài đặt. Cách ở mục 1 có thể giải bài này thuận tiện hơn.

9.3.2 Ví dụ 5: *Regions*

Vẫn là bài cây N đỉnh có màu; mỗi truy vấn cho x, y , hỏi số cặp i, j sao cho màu của i là x , màu của j là y , và i là tổ tiên của j .

Trước hết xét trường hợp đặc biệt: trong mọi truy vấn, x đều giống nhau. Khi đó chỉ cần một DFS đơn giản để tính đáp án cho từng y ; trong quá trình DFS, ghi lại trên đường từ điểm hiện tại tới gốc có

bao nhiêu điểm màu x .

Tiền xử lý thứ tự DFS, khi đó các điểm của mỗi cây con là một đoạn có đầu trái là chính đỉnh đó. Đồng thời với mỗi màu, tiền xử lý các đầu trái và đầu phải của các đoạn đại diện bởi những điểm mang màu đó, rồi sắp xếp.

Với mỗi truy vấn, giả sử hai màu có lần lượt A và B điểm. Ta tính mỗi điểm nằm trong bao nhiêu đoạn rồi cộng đáp án. Vì các danh sách đã sắp xếp, có thể trộn chúng trong $O(A + B)$, sau đó quét một lần để nhận đáp án.

Tiếp theo xét một cách vết cạn cho trường hợp số màu ít. Với mỗi màu x , DFS một lần có thể tiền xử lý đáp án cho mọi truy vấn trong $O(NC)$, với C là số màu; chỉ cần trong DFS ghi lại từ điểm hiện tại tới gốc có bao nhiêu màu x . Tương tự, nếu cố định y , quét mọi điểm một lần cũng tính được đáp án; chỉ cần tiền xử lý tổng tiền tố thì hỏi đoạn trong $O(1)$.

Kết hợp hai cách vết cạn: với những màu xuất hiện quá $H(N)$ lần, tiền xử lý đáp án khi màu đó làm một trong hai màu x hoặc y . Với những màu còn lại, lúc truy vấn dùng cách vết cạn thứ nhất để trả lời trực tiếp.

Khi $H(N) = \sqrt{N}$, thời gian là $O(N\sqrt{N})$, bộ nhớ là $O(N\sqrt{N})$. Lời giải chính thức còn nêu một cách phân loại khác, đạt thời gian $O(N\sqrt{N} \log N)$ và bộ nhớ $O(N)$. Cách này đã được Wang Yuetong nhắc tới trong luận văn năm 2014, nên ở đây không trình bày thêm.

9.4 Xây dựng lại định kỳ

Ý tưởng của xây dựng lại định kỳ là chia khối theo thao tác: cứ sau $H(N)$ thao tác thì xây lại trạng thái một lần. Khi truy vấn, dựa vào lần xây lại gần nhất và mọi sửa đổi trong khoảng giữa để tính đáp án.

Chú ý rằng chia khối theo thao tác không cần offline; chỉ cần sau một khoảng thời gian cố định lại tiền xử lý một lượt.

9.4.1 Ví dụ 2: *Chef and Churu*

Vẫn là bài N số a_i , N hàm f_i , trong đó mỗi hàm là tổng của một đoạn a_i :

$$f_i = \sum_{x=l_i}^{r_i} a_x.$$

Có N thao tác, mỗi thao tác có thể sửa một số a_i hoặc hỏi một tổng $\sum_{x=l_i}^{r_i} f_x$.

Trước hết chia các thao tác thành khối; cứ $H(N)$ thao tác xây lại một lần. Mỗi lần xây lại, dùng $O(N)$ tính tổng tiền tố của mảng gốc và $O(N)$ tính giá trị mọi f_i . Tổng chi phí là $O(N \cdot N/H(N))$.

Tiếp theo chỉ cần tính đóng góp của mỗi thao tác cho các truy vấn trong cùng khối thao tác. Vì truy vấn có tính trừ được, có thể tách mỗi truy vấn thành các truy vấn tiền tố, rồi sắp xếp các truy vấn tiền tố theo độ dài.

Sau khi sắp xếp, quét theo đầu phải tăng dần và duy trì một cấu trúc hỗ trợ cộng đoạn $O(\sqrt{N})$, hỏi điểm $O(1)$. Cấu trúc này đã nhắc tới phía trước: chia a_i thành các khối cỡ $H(N)$; mỗi lần sửa chỉ cần sửa $O(N/H(N))$ khối và $O(H(N))$ điểm lẻ, còn hỏi chỉ cần cộng đáp án trong khối và đáp án điểm lẻ.

Với mỗi truy vấn, liệt kê các sửa đổi nằm trong cùng khối thao tác và đứng trước nó, nhiều nhất $O(H(N))$ thao tác. Vì có thể hỏi trong $O(1)$ ảnh hưởng của một vị trí lên truy vấn đó, phần đóng góp này được tính trong $O(NH(N))$ trên toàn khối.

Tổng thời gian là $O(N\sqrt{N})$, bộ nhớ là $O(N)$.

Nếu bắt buộc trực tuyến, có thể thay thao tác sắp xếp rồi quét bằng cách tiền xử lý và ghi vào một cây

đoạn bền vững. Khi đó thêm một hệ số log là hoàn thành thuật toán trực tuyến, thời gian $O(N\sqrt{N}\log N)$, nhưng bộ nhớ cũng cần $O(N\sqrt{N}\log N)$.

9.4.2 Ví dụ 6: bài toán phần tử lớn thứ k trên cây có hỗ trợ link, cut

Cho một rừng N đỉnh và N thao tác. Mỗi lần có thể thêm một cạnh hoặc xóa một cạnh, bảo đảm sau thao tác vẫn là rừng; hoặc hỏi phần tử lớn thứ k trên một đường đi.¹⁹

Trước hết xét trường hợp không có thao tác link, cut. Có thể dùng cây đoạn bền vững: mỗi đỉnh duy trì một cây đoạn bền vững xây từ cây của cha nó rồi thêm trọng số của chính đỉnh đó. Khi truy vấn, mỗi đường có thể tách thành hai đường từ một điểm tới gốc, rồi dùng tổng của hai đường này để nhị phân trên cây đoạn bền vững.

Giả sử ta duy trì cây đoạn bền vững của thời điểm cách đây K thao tác. Khi đó có thể dùng link-cut tree để tách một đường trên cây hiện tại thành nhiều nhất $O(K)$ đường trên cây ở thời điểm cách đây K thao tác. Dùng tổng của các đường này để nhị phân trên cây đoạn bền vững là được.

Vậy cứ mỗi $\sqrt{N}$ thao tác xây lại cây đoạn bền vững một lần, thời gian là $O(N\sqrt{N}\log N)$.

9.5 Tổng kết

Ưu thế của chia khối nằm ở tính thông dụng cao: nó có thể hỗ trợ duy trì nhiều loại thông tin. Các ví dụ ở những mục trước hầu như đều dùng hai ý tưởng khác nhau để giải. Tuy nhiên độ phức tạp thời gian cao là nhược điểm chí mạng của chia khối. Dù vậy, khi chưa nghĩ ra lời giải tốt hơn, thử chia khối là hoàn toàn đáng làm.

9.6 Lời cảm ơn

- Xin cảm ơn China Computer Federation đã cung cấp cho tôi nền tảng học tập và giao lưu.
- Xin cảm ơn thầy Li Jian vì nhiều năm quan tâm và chỉ dẫn.
- Xin cảm ơn các bạn Cen Ruoxu, Zhang Yuhao, Du Yuhan và những bạn khác đã giúp đỡ trong quá trình viết bài này.

9.7 Tài liệu tham khảo

1. Mugurel Ionut Andreica, “Novel $O(H(N) + N/H(N))$ Algorithmic Techniques for Several Types of Queries and Updates on Rooted Trees and Lists”.
2. “21-st International Olympiad in Informatics Tasks and Solutions”, từ www.ioi2009.org.
3. Wang Yuetong, “Thuật toán căn: không chỉ là chia khối”, luận văn đội tuyển quốc gia năm 2014, Trường Ngoại ngữ Nam Kinh.
4. Xu Yinzhan, “Ứng dụng cây đoạn trên một lớp bài toán chia để trị”, luận văn đội tuyển quốc gia năm 2014, Trường Học Quân Hàng Châu.

10 Thuật toán liên quan tới cactus và ứng dụng

Wang Yisong

Trường Học Quân Hàng Châu, Chiết Giang

¹⁹Một bài kinh điển.

Tóm tắt

Trong những năm gần đây, thi tin học xuất hiện khá nhiều bài toán liên quan tới cactus, chẳng hạn các bài toán cactus động. Bài viết này giới thiệu những thuật toán thường gặp trên cactus và đưa ra một số ví dụ, với hy vọng có thể gợi mở thêm các hướng ứng dụng.

10.1 Định nghĩa và cấu trúc của cactus

10.1.1 Định nghĩa cactus

Nếu trong một đồ thị vô hướng liên thông, mỗi cạnh thuộc nhiều nhất một chu trình đơn, và đồ thị không có khuyên, ta gọi đồ thị đó là một cactus.

10.1.2 Số cạnh của cactus

Với một cactus có n đỉnh, xét một cây khung của nó; cây khung có $n - 1$ cạnh. Những cạnh còn lại đều là cạnh không thuộc cây. Mỗi cạnh không thuộc cây tương ứng với một đường đi trên cây khung và tạo thành một chu trình đơn. Vì mỗi cạnh thuộc nhiều nhất một chu trình đơn, giao của hai đường đi tương ứng với hai cạnh không thuộc cây bất kỳ là rỗng, tức các đường đi này không chồng lên nhau. Do đó nhiều nhất có $n - 1$ đường đi như vậy. Vì thế cactus n đỉnh có nhiều nhất $2n - 2$ cạnh và ít nhất $n - 1$ cạnh.

10.1.3 Cấu trúc của cactus

Hình a. Bản gốc vẽ một cactus gốc 1: các chu trình gồm 1 - 6, 2 - 3 - 4 - 5, 8 - 9 - 10 và 10 - 11, cùng các nhánh cây gắn vào những đỉnh này.

Tương tự khái niệm cha của đỉnh trên cây, ta định nghĩa cha của đỉnh và cha của chu trình trên cactus.

Nếu mọi đường đi đơn từ một đỉnh u tới gốc của cactus đều có cạnh đầu tiên giống nhau, thì giống như trên cây, cha của u là đỉnh thứ hai trên một đường đi đơn từ u tới gốc. Ngược lại, cha của u là chu trình chứa cạnh đầu tiên trên một đường đi đơn từ u tới gốc. Chẳng hạn trong hình, cha của đỉnh 8 là đỉnh 1; cha của đỉnh 10 là chu trình 8 - 9 - 10; cha của đỉnh 11 là chu trình 10 - 11.

Cha của một chu trình là đỉnh trên chu trình gần gốc nhất. Chẳng hạn cha của chu trình 8 - 9 - 10 là đỉnh 8, còn cha của chu trình 10 - 11 là đỉnh 10.

10.1.4 Con

Tương tự con của đỉnh trên cây, ta định nghĩa con của đỉnh và con của chu trình trên cactus.

Con của một đỉnh có thể là một đỉnh, cũng có thể là một chu trình. Chẳng hạn con của đỉnh 1 gồm đỉnh 2, đỉnh 8 và chu trình 1 - 6.

Con của một chu trình là mọi đỉnh trên chu trình trừ cha của chu trình đó. Ví dụ, con của chu trình 8 - 9 - 10 là đỉnh 9 và đỉnh 10.

10.1.5 Đỉnh cha và đỉnh mẹ

Với một đỉnh nằm trên chu trình, ta gọi hai đỉnh kề nó trên chu trình lần lượt là đỉnh cha và đỉnh mẹ. Cần phân biệt “cha” với “đỉnh cha”. Chẳng hạn trên chu trình 2 - 3 - 4 - 5, đỉnh cha của 3 là 2, đỉnh mẹ của 3 là 4; đỉnh cha của 4 là 3, đỉnh mẹ của 4 là 5. Bắt đầu từ một con bất kỳ của chu trình, đi theo các đỉnh cha và đỉnh mẹ là có thể duyệt hết chu trình.

10.1.6 Cách duyệt một cactus

Tương tự quá trình duyệt cây, ta bắt đầu DFS từ gốc. Giả sử hiện đang ở đỉnh x , và tiếp theo cần thăm đỉnh y .

Nếu y chưa được thăm, xử lý giống như trên cây: chỉ cần đặt cha của y là x .

Nếu y đã được thăm, và thời điểm thăm đầu tiên của y sớm hơn thời điểm thăm đầu tiên của x , tức ta đã phát hiện một chu trình. Cha của chu trình này là y , còn các con là mọi đỉnh trên đường đi từ x tới y sau khi bỏ y . Ta duyệt chu trình này và đặt đỉnh cha, đỉnh mẹ cho mọi đỉnh trên chu trình.

Nếu y đã được thăm, và thời điểm thăm đầu tiên của y muộn hơn thời điểm thăm đầu tiên của x , tức y nằm trên một chu trình đã được xử lý; có thể bỏ qua trường hợp này.

10.2 DP trên cactus

Tương tự DP trên cây, ta cũng có thể thực hiện DP trên cactus.

Bắt đầu DP từ gốc cactus. Giả sử hiện DP tới đỉnh u ; trước hết xử lý thông tin ở u , sau đó liệt kê từng con của u và từng con của các chu trình có cha là u , rồi tiếp tục DP.

10.2.1 Ví dụ 1: một bài toán kinh điển

Cho một cactus, mỗi cạnh có độ dài. Hãy tính khoảng cách từ đỉnh 1 tới mọi đỉnh, trong đó khoảng cách giữa hai đỉnh là độ dài đường đi ngắn nhất giữa chúng. Ràng buộc $n \leq 10^5$.

Trước hết DFS một lượt để xác định cấu trúc cactus.

Sau đó bắt đầu DP từ đỉnh 1. Giả sử đã biết khoảng cách từ đỉnh 1 tới đỉnh u . Ta liệt kê từng con của u . Nếu con đó là một đỉnh, ta tính được khoảng cách từ đỉnh 1 tới đỉnh này và tiếp tục DP từ đó. Nếu con đó là một chu trình, ta liệt kê từng con x trên chu trình, tính khoảng cách từ x tới đỉnh 1, rồi tiếp tục DP từ x .

Độ phức tạp thời gian là $O(n)$.

10.2.2 Ví dụ 2: một bài toán kinh điển khác

Cho một cactus, mỗi cạnh có độ dài. Hãy tính đường kính của cactus. Đường kính là khoảng cách của cặp đỉnh xa nhau nhất, trong đó khoảng cách giữa hai đỉnh là độ dài đường đi ngắn nhất giữa chúng. Ràng buộc $n \leq 10^5$.

Trường hợp cây. Vì cây cũng là cactus, trước hết xét trường hợp cây. Một cách kinh điển là chọn tùy ý một đỉnh, tìm đỉnh xa nó nhất, rồi từ đỉnh vừa tìm được lại tìm một đỉnh xa nhất; khoảng cách giữa hai đỉnh này là đáp án.

Cách làm này sai trên cactus.

Hình b. Bản gốc minh họa một chu trình có $2k$ đỉnh, mọi cạnh trên chu trình dài 1, cộng thêm một cạnh nhánh. Đáp án là $k + 1$, nhưng cách hai lần tìm đỉnh xa nhất chỉ ra đáp án này nếu điểm ban đầu rơi vào $O(1)$ đỉnh đặc biệt.

Một cách làm khác. Với cây, xét DP cây: với mỗi x , tính độ sâu lớn nhất trong cây con gốc x , rồi dùng tổng độ sâu lớn nhất và lớn nhì trong các con khác nhau của mỗi đỉnh để cập nhật đáp án.

Với cactus, có thể làm tương tự DP cây. Trước hết DFS một lượt để xác định cấu trúc cactus. Sau đó với mỗi đỉnh x , tính độ sâu lớn nhất của cactus con x . Ở đây cactus con x là thành phần liên thông chứa x sau khi xóa mọi cạnh nằm trên các đường đi đơn từ gốc tới x .

Với mỗi đỉnh, dùng độ sâu lớn nhất của hai con khác nhau để cập nhật đáp án. Với mỗi chu trình, còn cần dùng tổng độ sâu lớn nhất của từng cặp đỉnh trên chu trình cộng với độ dài đường đi ngắn nhất giữa cặp đó để cập nhật đáp án. Không mất tính tổng quát, liệt kê một đỉnh; khi đó những đỉnh còn lại mà đường ngắn nhất đi theo chiều kim đồng hồ tạo thành một đoạn trên chu trình, và đầu còn lại của đoạn này cũng di chuyển theo chiều kim đồng hồ khi đỉnh đang xét di chuyển theo chiều kim đồng hồ. Đây là bài toán RMQ có hai đầu mút cùng đơn điệu, có thể duy trì bằng hàng đợi đơn điệu.

Cũng có thể không dùng hàng đợi đơn điệu. Chia chu trình thành hai phần theo nhánh mà khoảng cách từ cha của chu trình đi xuống ngắn hơn. Với cặp đỉnh nằm cùng một phần, đường ngắn nhất chắc chắn đi trong phần đó. Với mỗi đỉnh, trong phần kia, các đỉnh mà đường ngắn nhất đi theo chiều kim đồng hồ tạo thành một tiền tố, còn các đỉnh mà đường ngắn nhất đi ngược chiều kim đồng hồ tạo thành một hậu tố; chỉ cần xử lý trực tiếp giá trị lớn nhất của tiền tố và hậu tố.

Độ phức tạp thời gian là $O(n)$.

10.2.3 Ví dụ 3: *mx's cactus*

Cho một cactus n đỉnh và q truy vấn. Mỗi truy vấn cho một tập đỉnh, yêu cầu in khoảng cách của cặp đỉnh xa nhau nhất trong tập đó. Khoảng cách giữa hai đỉnh là độ dài đường đi ngắn nhất giữa chúng.

$$n \leq 3 \cdot 10^5, \quad q \leq 3 \cdot 10^5, \quad \text{tot} \leq 3 \cdot 10^5,$$

trong đó tot là tổng kích thước các tập đỉnh. Giới hạn thời gian 5 giây, bộ nhớ 512 MB.²⁰

Thuật toán vét cạn. Xử lý từng truy vấn bằng vét cạn. Với trường hợp cây, mỗi truy vấn chỉ cần làm một lần DP cây. Với trường hợp cactus, mỗi truy vấn dùng DP trên cactus như ví dụ trước. Độ phức tạp là $O(nq)$.

Một tính chất. Trong quá trình DP cây, nếu một truy vấn cho cnt đỉnh, số lần cập nhật đáp án thật sự có hiệu lực là $cnt - 1$, vì ban đầu có cnt đỉnh mang đáp án, và mỗi lần cập nhật đáp án sẽ làm giảm đi một đỉnh “có đáp án”. DP trên cactus cũng có tính chất tương tự: số lần cập nhật đáp án có hiệu lực là $O(cnt)$. Vì thế tổng số lần cập nhật có hiệu lực trên mọi truy vấn là $O(n)$.

Trường hợp cây. Dùng $f[x][i]$ biểu thị khoảng cách lớn nhất từ các đỉnh thuộc tập của truy vấn thứ i , nằm trong cây con x , tới đỉnh x . Khi đó quá trình DP cây tương đương với việc gộp các mảng này và cập nhật đáp án của những truy vấn tương ứng.

Bài toán trở thành: với mỗi đỉnh, gộp các mảng DP của từng con và nhanh chóng tìm những chỉ số truy vấn cần được cập nhật đáp án.

Điều này tương đương với việc duy trì nhiều tập số, hỗ trợ gộp hai tập và trả về giao của chúng trong lúc gộp. Có thể dùng mảng để lưu các số trong tập, rồi gộp heuristic: mỗi lần gộp thì chen thô tập nhỏ vào tập lớn. Vì cần tìm giao, có thể mở một bảng băm cho mỗi tập, hoặc dùng một bảng băm toàn cục, rồi kiểm tra khi gộp.

Độ phức tạp thời gian $O(n \log n)$, bộ nhớ $O(n)$.

²⁰Nguồn: UOJ #87, *mx's cactus*.

Trường hợp cactus. Với những phần không phải chu trình, xử lý giống trường hợp cây.

Với mỗi chu trình, trước hết tính mảng DP của từng con trên chu trình, sau đó dùng giao của từng cặp mảng DP để cập nhật đáp án.

Vì không thể xóa các phần tử của một tập khỏi tập khác, ta không dùng được hàng đợi đơn điệu có thao tác xóa. Có thể chia chu trình thành hai phần theo nhánh mà đi từ cha của chu trình xuống gần hơn. Với mỗi phần của chu trình, gộp các mảng DP theo thứ tự, rồi lấy giao với các mảng DP của phần kia để cập nhật đáp án. Sau khi cập nhật đáp án trên chu trình, cần hoàn tác mọi thao tác gộp; điều này có thể làm bằng cách ghi lại các thao tác phát sinh trong quá trình gộp. Cuối cùng gộp toàn bộ mảng DP của mọi con trên chu trình.

Không xét các lần gộp mảng DP khi cập nhật đáp án trên chu trình, tổng thời gian là $O(n \log n)$, vì mỗi đỉnh truy vấn đóng vai trò phần tử của “tập nhỏ hơn” nhiều nhất $O(\log n)$ lần. Khi xét quá trình cập nhật trên chu trình, bản chất của nó giống việc gộp toàn bộ mảng DP trên chu trình nên không ảnh hưởng tới độ phức tạp. Do đó tổng thời gian vẫn là $O(n \log n)$.

Giả sử trên một chu trình có nhiều tập, tổng kích thước là N . Có thể chứng minh chi phí gộp lần lượt các tập theo thứ tự không vượt quá $O(N)$, nên số thao tác cần ghi lại khi gộp cũng là $O(N)$. Tổng bộ nhớ vì vậy là $O(n)$.

10.3 Chia để trị theo điểm trên cactus

Tương tự chia để trị theo điểm trên cây, ta có thể chia để trị theo điểm trên cactus.

Mỗi lần chọn một đỉnh, gọi là “trọng tâm”, sao cho sau khi xóa đỉnh này và mọi cạnh trên các chu trình chứa đỉnh này, kích thước thành phần liên thông lớn nhất là nhỏ nhất. Sau đó thống kê thông tin liên quan tới trọng tâm và các chu trình chứa trọng tâm, rồi đệ quy chia để trị từng thành phần liên thông còn lại.

Độ phức tạp. Xét thành phần hiện tại có kích thước N . Nếu chọn đỉnh x , và tồn tại một đỉnh y sao cho khi lấy x làm gốc, kích thước cactus con y lớn hơn $N/2$, thì chọn y chắc chắn tốt hơn chọn x . Vì vậy sau mỗi lần chia để trị, thành phần liên thông lớn nhất giảm ít nhất một nửa. Số tầng chia để trị là $O(\log n)$.

10.3.1 Ví dụ 1: Vua bọ chết xuống Giang Nam

Cho một cactus. Với mỗi $l = 1, 2, \dots, n$, hãy tính số đường đi đơn bắt đầu từ đỉnh 1 và đi qua đúng l cạnh. Lấy đáp án modulo 998244353, một số nguyên tố bằng $7 \cdot 17 \cdot 2^{23} + 1$. Ràng buộc $n \leq 10^5$.²¹

Thuật toán vét cạn. Gọi $f[x][l]$ là số đường đi độ dài l bắt đầu từ đỉnh x và chỉ được đi theo hướng xa gốc hơn. Với mỗi con y của x , nếu y là một đỉnh, cộng $f[y][l-1]$ vào $f[x][l]$. Nếu con đó là một chu trình, liệt kê từng con z trên chu trình và cộng $f[z][l-d_1] + f[z][l-d_2]$ vào $f[x][l]$, trong đó d_1, d_2 là độ dài hai đường đi từ x tới z .

Chia để trị cactus. Ta nhận thấy $f[u]$ có thể xem như một đa thức. Mỗi lần chuyển từ một con v của u tương đương với nhân $f[v]$ với x , hoặc với $x^{d_1} + x^{d_2}$, rồi cộng vào $f[u]$. Điều này gợi ý dùng chia để trị.

Trong mỗi bước chia để trị, sau khi tìm trọng tâm u , trước hết đệ quy xử lý từng thành phần liên thông. Sau đó tính đa thức từ gốc tới trọng tâm, ký hiệu $f(x)$, rồi cộng đáp án của mọi con của u , ký hiệu $g(x)$. Chỉ cần cộng $f(x) \cdot g(x)$ với đáp án của từng thành phần liên thông là được.

²¹Nguồn: UOJ #23, UR #1: Vua bọ chết xuống Giang Nam.

Khi tính $f(x)$, nếu nhân các đa thức từ trọng tâm tới gốc bằng chia để trị kết hợp FFT, độ phức tạp là $O(n \log^2 n)$. Thật ra có thể ghi lại đa thức từ gốc tới trọng tâm trong quá trình đệ quy, rồi nhân ngược từng đa thức một. Khi đó độ phức tạp thỏa

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

Tổng độ phức tạp là $O(n \log^2 n)$.

10.3.2 Ví dụ 2: *Tree and Sets*

Cho một cactus n đỉnh, mỗi cạnh có một độ dài l , các cạnh khác nhau có thể có độ dài khác nhau.

Có q tập đỉnh. Mỗi tập có thể được mô tả bằng hai số nguyên u, d ($1 \leq u \leq n$). Một đỉnh v thuộc tập này khi và chỉ khi khoảng cách giữa v và u không vượt quá d , trong đó khoảng cách giữa hai đỉnh là độ dài đường đi ngắn nhất giữa chúng.

Cần xây một đồ thị có hướng không chu trình, thỏa:

- DAG có ít nhất $n + q$ đỉnh, nhiều nhất 1200000 đỉnh và 2400000 cạnh.
- Với mỗi cạnh từ u tới v , ta có $u > n$ và $u \neq v$.
- Với mỗi đỉnh x có số hiệu nằm trong tập đỉnh thứ i ($1 \leq i \leq q$), có đúng một đường đi từ đỉnh $n + i$ tới đỉnh x .
- Với mỗi đỉnh $x \in \{1, 2, \dots, n\}$ nhưng không thuộc tập đỉnh thứ i , không tồn tại đường đi từ đỉnh $n + i$ tới đỉnh x .

Ràng buộc $1 \leq n, q \leq 10000$.²²

Trường hợp cây. Trước hết thêm các đỉnh ảo để bậc của mọi đỉnh không vượt quá 3, đồng thời với mỗi đỉnh, nhiều nhất một tập đỉnh có u bằng đỉnh đó.

Sau đó chia để trị theo điểm trên cây. Mỗi lần chia để trị, với từng cây con của trọng tâm, sắp xếp mọi đỉnh trong cây con đó theo khoảng cách tới trọng tâm, rồi tiền xử lý tổng tiền tố và nối các cạnh tương ứng cho từng tập đỉnh.

Vì số tầng chia để trị theo điểm là $O(\log n)$, tổng số cạnh được nối là $O(n \log n)$, giả sử $q = O(n)$.

Trường hợp cactus. Vẫn có thể thêm đỉnh ảo để bậc của mọi đỉnh không vượt quá 3. Khi đó mỗi đỉnh của cactus thuộc nhiều nhất một chu trình.

Mỗi lần chia để trị có thể chọn một đỉnh, hoặc chọn một chu trình, làm “trọng tâm”. Để chứng minh số tầng chia để trị vẫn là $O(\log n)$.

Khi trọng tâm là một đỉnh, xử lý giống trường hợp cây.

Khi trọng tâm là một chu trình, tương tự DP trên cactus, chia chu trình này thành hai phần. Cả trong từng phần lần giữa hai phần đều là một bài toán tổng tiền tố/hậu tố. Lấy bài toán tổng tiền tố trong một phần làm ví dụ. Xem phần này là một dãy, gọi $\text{size}[i]$ là kích thước cactus con tại vị trí i của dãy. Mỗi lần chia để trị trên đoạn $[l, r]$, đặt

$$\sum_{i=l}^r \text{size}[i] = N.$$

Chọn mid sao cho

$$\sum_{i=l}^{\text{mid}-1} \text{size}[i] \leq N/2, \quad \sum_{i=\text{mid}+1}^r \text{size}[i] \leq N/2.$$

²²Nguồn: đợt thi đối kháng đội tuyển quốc gia năm 2015.

Sau đó đệ quy xử lý $[l, mid)$ và $(mid, r]$, rồi dựng các cạnh giữa ba phần $[l, mid)$, mid , $(mid, r]$. Trong phép chia đệ trị trên chu trình này, với mỗi i , cactus con thứ i xuất hiện trong $O(\log(N/\text{size}[i])) + O(1)$ lần chia đệ trị. Vì vậy với mỗi đỉnh, tổng số lần nó xuất hiện trong mọi lần chia đệ trị theo điểm và chia đệ trị trên chu trình là $O(\log n)$.

Do đó tổng số cạnh vẫn là $O(n \log n)$.

10.4 Link-cut cactus

10.4.1 Bài toán cactus động

Bài toán cactus động là một lớp bài toán cần duy trì thông tin động trên cactus. Việc duy trì động có thể là thay đổi hình dạng, hoặc thay đổi các thông tin liên quan.

Nhiều bài toán cây động có thể giải bằng link-cut tree. Tương tự link-cut tree, ta có thể xây link-cut cactus, dùng để giải khá nhiều bài toán cactus động.

Cấu trúc của link-cut cactus. Link-cut tree duy trì một phân rã theo xích của cây, nên ta cũng có thể duy trì một phân rã theo xích của cactus.

Nếu không có chu trình, cấu trúc giống link-cut tree: mỗi đỉnh có một *preferred child*, nối bằng cạnh thực; các con khác nối bằng cạnh ảo.

Hình c. Bản gốc minh họa trường hợp không có chu trình: cạnh đỏ là cạnh thực, cạnh đen là cạnh ảo.

Với một chu trình, định nghĩa cha của nó là đỉnh trên chu trình gần gốc của cactus nhất, ký hiệu A . Định nghĩa *preferred child* của chu trình là đỉnh trên chu trình được access gần đây nhất, ký hiệu B .

Ta nối đường đi ngắn nhất giữa A và B bằng cạnh thực.

Xích tạo bởi những đỉnh còn lại trên chu trình cũng được nối bằng cạnh thực; ta gọi xích này là xích phụ, tức xích *extra* trong hình nguồn.

Hình d. Bản gốc vẽ chu trình với hai đỉnh A, B , một đường $A-B$ được chọn làm xích chính và phần còn lại là xích *extra*.

Cách duy trì thông tin cạnh trên splay. Ta thêm một đỉnh vào mỗi cạnh để biểu diễn cạnh đó trên splay. Với hai đoạn đầu của xích phụ nối tới A và B , bản gốc tô đen các cạnh này; ta trực tiếp gắn chúng vào hai đầu của xích phụ để duy trì.

Trường hợp biên. Nếu mọi đỉnh trên chu trình đều nằm trên xích AB , thì xích phụ chỉ còn một cạnh. Tuy nhiên ta đã thêm một đỉnh trên cạnh này để duy trì nó, nên thực tế không cần thảo luận riêng trường hợp này.

Một trường hợp đặc biệt. Giả sử lần access trước là tới đỉnh A . Khi đó cạnh kề A trên xích AB trở thành cạnh ảo, nhưng đỉnh splay biểu diễn cạnh này vẫn nằm trên splay của xích AB . Cần đặc biệt xử lý trường hợp này.

Thao tác access. Thao tác này tương tự access của link-cut tree.

Giả sử hiện access tới đỉnh x . Trước hết tìm cạnh e đứng ngay trước x trên splay.

Nếu e không nằm trên chu trình, xử lý giống link-cut tree.

Ngược lại, trước hết cắt hai đầu trái phải của xích AB của chu trình này, chú ý xét trường hợp đặc biệt phía trên; sau đó nối xích AB với xích phụ.

Tiếp theo đưa x lên gốc splay, chọn phía ngắn hơn để nối lại làm xích AB mới, và đặt phía còn lại thành xích phụ.

Thao tác đổi gốc. Tương tự thao tác đổi gốc của link-cut tree: sau access, đặt nhãn đảo trên đường từ gốc tới đỉnh đó.

Cần chú ý thao tác này làm hoán đổi hai đỉnh A và B của các chu trình mà đường đi này đi qua, đồng thời hướng của xích phụ trở nên không đúng.

Cách giải quyết là với mỗi chu trình được access, kiểm tra thứ tự trước sau của A và B trong splay. Nếu bị đảo, hoán đổi A, B và đặt nhãn đảo cho xích phụ.

Cách sử dụng link-cut cactus. Lấy truy vấn giá trị nhỏ nhất của trọng số cạnh trên một đường đi ngắn nhất từ u tới v làm ví dụ: trước hết đổi gốc thành u , sau đó access đỉnh v , rồi trả về giá trị nhỏ nhất của trọng số cạnh trên splay này.

Cần chú ý giữa u và v có thể có nhiều đường đi ngắn nhất; có thể duy trì điều này bằng cách đặt nhãn trên splay khi access tới chu trình.

Độ phức tạp thời gian. Tương tự link-cut tree, với cactus n đỉnh và $O(n)$ thao tác access, số lần chuyển đổi *preferred child* của đỉnh và chu trình là $O(n \log n)$. Vì mỗi lần chuyển đổi *preferred child* dùng splay, tổng thời gian là $O(n \log^2 n)$.

10.4.2 Ví dụ 1: *Push the Flow!*

Cho một cactus n đỉnh, không có cạnh bội hoặc khuyên, mỗi cạnh có một dung lượng.

Có q thao tác; mỗi thao tác hoặc sửa dung lượng của một cạnh, hoặc hỏi lưu lượng cực đại giữa hai đỉnh.

$$1 \leq n \leq 100000, \quad 0 \leq q \leq 200000.$$

23

Cách dùng link-cut cactus. Với trường hợp không có chu trình, lưu lượng cực đại giữa hai đỉnh chính là giá trị nhỏ nhất của dung lượng cạnh trên đường đi giữa hai đỉnh đó.

Với mỗi chu trình, lưu lượng cực đại giữa hai đỉnh A, B bằng giá trị nhỏ nhất trên xích AB cộng với giá trị nhỏ nhất trên xích phụ. Vì vậy có thể cộng giá trị nhỏ nhất của xích phụ vào dung lượng của từng cạnh trên xích AB .

Khi truy vấn, chỉ cần đổi gốc rồi access.

Khi sửa dung lượng của một cạnh, đổi gốc thành một đầu mút của cạnh này, rồi access đầu mút còn lại. Khi đó cạnh cần sửa chắc chắn không nằm trên xích phụ của một chu trình, nên có thể sửa trực tiếp.

Độ phức tạp thời gian $O((n + q) \log^2 n)$, bộ nhớ $O(n)$.²⁴

²³Nguồn: CODECHEF PUSHFLOW.

²⁴Bài này cũng có thể giải bằng link-cut tree; vì đây không phải trọng tâm nên bản gốc lược qua.

10.4.3 Ví dụ 2: một bài toán kinh điển

Cho một cactus, cần hỗ trợ thêm cạnh, xóa cạnh, và sửa hoặc hỏi trên đường đi ngắn nhất giữa hai đỉnh hay trên một cactus con.

Khi lấy x làm gốc, cactus con y là thành phần liên thông chứa y sau khi xóa mọi cạnh nằm trên các đường đi đơn từ x tới y .

Thao tác sửa là cộng một số vào trọng số đỉnh trên một xích hoặc một cactus con, hoặc gán toàn bộ các trọng số này thành cùng một số.

Truy vấn là giá trị lớn nhất, nhỏ nhất, hoặc tổng trọng số đỉnh trên một xích hay một cactus con.

Một bài toán con. Cho một cây, cần hỗ trợ thêm cạnh, xóa cạnh, và sửa hoặc hỏi trên một đường đi hay một cây con.

Có thể dùng self-adjusting top trees để làm, độ phức tạp thời gian $O(n \log n)$. Cách làm này được giới thiệu chi tiết trong tài liệu tham khảo [1] và [2].

Thuật toán 1. Dùng link-cut cactus để duy trì hình dạng của cactus.

Với mỗi chu trình, cắt cạnh nối đỉnh B trên chu trình với xích phụ. Nếu chu trình này chỉ có hai đỉnh thì không cắt cạnh đó.

Như vậy ta thu được một cây khung của cactus, có thể duy trì bằng self-adjusting top trees, gọi tắt là top tree.

Khi thao tác trên xích, đổi gốc và access trên link-cut cactus, rồi thực hiện thao tác xích trên top tree.

Khi thao tác trên cactus con, đổi gốc rồi access; lúc đó cactus con này là một cây con trên top tree, nên thực hiện trực tiếp thao tác cây con của top tree.

Trong thao tác access của link-cut cactus, $O(\log n)$ chu trình bị ảnh hưởng bởi thay đổi *preferred child*. Ta cần thay đổi cạnh bị cắt trên những chu trình đó, tức thực hiện $O(\log n)$ thao tác link và cut trên top tree. Độ phức tạp khâu hao là $O(\log^2 n)$.

Vì dùng top tree nên hằng số hơi lớn.

Thuật toán 2. Xét việc duy trì trực tiếp bằng link-cut cactus.

Tương tự top tree, trên mỗi đỉnh của link-cut cactus có thể duy trì thông tin của cactus con. Ta gọi cách làm này là “top cactus”.

Một kỹ thuật nhỏ: gọi thông tin cần duy trì là Info, nhãn là Tag. Khi đó chỉ cần cài đặt $\text{Info} + \text{Info}$, $\text{Info} * \text{Tag}$ và $\text{Tag} * \text{Tag}$. Chú ý Tag có thể biểu diễn dưới dạng $a * x + b$, giúp đơn giản hóa mã.

Trên mỗi đỉnh của top cactus, duy trì các giá trị sau:

- x : thông tin của chính đỉnh này.
- sum: tổng thông tin của mọi đỉnh trên xích do splay gốc tại đỉnh này biểu diễn, không gồm thông tin cactus con.
- sub: tổng thông tin cactus con của mọi đỉnh trên xích do splay gốc tại đỉnh này biểu diễn, không gồm thông tin trên xích.
- ex: với cạnh đầu tiên của xích AB của mỗi chu trình, ex biểu diễn giá trị all của xích phụ của chu trình đó.
- all: $\text{all} = \text{sum} + \text{sub} + \text{ex}$.

- chain tag: nhãn trên mọi đỉnh của xích do splay gốc tại đỉnh này biểu diễn, không gồm nhãn cactus con.
- sub tag: nhãn trên cactus con của mọi đỉnh thuộc xích do splay gốc tại đỉnh này biểu diễn, không gồm nhãn trên xích.
- ex tag sum: tổng các sub tag đã đặt trên xích AB của chu trình kể từ lần access trước, nhưng chưa có trên xích phụ.

Trong đó sub cần liệt kê từng cactus con của đỉnh này để tính. Vì bậc của một đỉnh có thể là $O(n)$, không thể tính sub bằng vết cạn; nên với mỗi đỉnh, dùng một cây cân bằng, chẳng hạn splay, để duy trì sub.

Đẩy nhãn xuống. chain tag chỉ được đẩy xuống trong nội bộ splay.

Khi sub tag được đẩy xuống cactus con, nó được tách thành dạng chain tag + sub tag.

ex tag sum duy trì những nhãn đã có trên xích AB nhưng chưa có trên xích phụ. Khi access tới một chu trình, đẩy các nhãn này xuống xích phụ của chu trình.

Thao tác access. Mỗi lần access tới đỉnh x :

- Nếu x không nằm trên chu trình, đổi cạnh nối x với *preferred child* cũ của nó thành cạnh ảo, và thêm *preferred child* cũ vào cây cân bằng duy trì cactus con. Sau đó đổi cạnh nối x với *preferred child* mới thành cạnh thực, xóa *preferred child* mới khỏi cây cân bằng duy trì cactus con, và truyền nhãn cactus con cho nó.
- Nếu x nằm trên chu trình, cắt hai đầu trên dưới của chu trình, tìm ex tag sum trên xích AB , rồi truyền nó cho xích phụ. Sau đó đưa x lên gốc splay, chọn phía ngắn hơn để nối lại, đồng thời cập nhật thông tin và nhãn tương ứng.

Độ phức tạp thời gian. Bản thân link-cut cactus có độ phức tạp $O(n \log^2 n)$.

Tổng số vòng lặp trong access là $O(n \log n)$, nên chi phí duy trì thông tin cactus con của mỗi đỉnh cũng là $O(n \log^2 n)$.

Tổng độ phức tạp thời gian là $O(n \log^2 n)$, nhưng hằng số nhỏ hơn thuật toán trước.

10.4.4 Ví dụ 3: một bài toán kinh điển mở rộng

Cho một cactus, cần hỗ trợ thêm cạnh, xóa cạnh, và sửa hoặc hỏi trên đường đi ngắn nhất giữa hai đỉnh hay trên một cactus con.

Khi lấy x làm gốc, cactus con y là thành phần liên thông chứa y sau khi xóa mọi cạnh nằm trên các đường đi đơn từ x tới y .

Thao tác sửa là cộng một số vào trọng số đỉnh trên một xích hoặc một cactus con, hoặc gán toàn bộ các trọng số này thành cùng một số.

Truy vấn là trọng số đỉnh lớn thứ k trên một xích hoặc một cactus con.

Một bài toán con. Cho một cây, cần hỗ trợ thêm cạnh, xóa cạnh, và sửa hoặc hỏi trọng số đỉnh lớn thứ k trên một đường đi hay một cây con.

Top tree không duy trì được truy vấn lớn thứ k . Ta xét một cách kinh điển khác để duy trì thông tin đường đi và cây con: thứ tự DFS.

Vì có sửa đường đi và hỏi lớn thứ k , cấu trúc duy trì thứ tự DFS có thể là danh sách chia khối. Giả sử kích thước khối là T . Khi đó một thao tác tách hoặc gộp trên danh sách chia khối cần $O(\log n + T)$ thời gian; một truy vấn cần $O((n/T) \log^2 n)$ thời gian. Chú ý có thể dùng cây cân bằng để duy trì dãy các khối, nhờ đó độ phức tạp sửa là $O(\log n + T)$.

Nếu mỗi thao tác của bài toán gốc có thể chuyển thành $f(n)$ thao tác tách, gộp trên thứ tự DFS và $O(1)$ truy vấn, thì chọn kích thước khối

$$T = O\left(\sqrt{\frac{n \log^2 n}{f(n)}}\right)$$

sẽ cho độ phức tạp mỗi thao tác là

$$O\left(\sqrt{nf(n) \log^2 n}\right).$$

Hình dạng cây không đổi, gốc cố định. Dùng link-cut tree để duy trì một phân rã theo xích của cây, rồi duy trì một thứ tự DFS tương ứng.

Trong thứ tự DFS này, mọi xích trong phân rã theo xích đều là một đoạn liên tiếp.

Trong quá trình access của link-cut tree, ta chỉnh thứ tự DFS tương ứng. Giả sử hiện cần đổi *preferred child* của đỉnh x thành y ; khi đó chỉ cần lấy đoạn thứ tự DFS tương ứng với cây con y ra và đặt ngay sau đỉnh x .

Như vậy sau khi access đỉnh x , xích từ gốc tới x là một đoạn trong thứ tự DFS, và cây con x cũng là một đoạn; do đó có thể thao tác trên xích và cây con.

Hình dạng cây không đổi, có thao tác đổi gốc.

Hình e. Bản gốc dùng các hình bầu dục biểu diễn đỉnh và tam giác biểu diễn cây con. Khi gốc là a , thứ tự DFS là $a b c C B A$; khi gốc là c , thứ tự DFS là $c b a A B C$.

Giả sử cần đổi gốc từ a sang c . Trước hết access gốc mới, phát sinh $O(f(n))$ thao tác trên thứ tự DFS. Nếu làm trực tiếp, có vẻ phải dùng $O(\text{chiều cao cây})$ thao tác thứ tự DFS để đổi gốc.

Ta cưỡng bức đặt nhãn đảo cho đoạn thứ tự DFS tương ứng với đường đi từ gốc cũ tới gốc mới, và cho phần còn lại của thứ tự DFS. Khi đó thứ tự DFS trở thành

$$c b a r(A) r(B) r(C),$$

trong đó $r(X)$ là thứ tự DFS sau khi đảo của cây con X . Lưu ý toàn bộ thứ tự DFS của một số cây con đã bị đảo.

Trong mỗi vòng lặp của access, kiểm tra xem có xảy ra tình huống này hay không. Nếu có, dùng $O(1)$ thao tác trên thứ tự DFS để đảo lại thứ tự DFS của cây con đó. Như vậy đạt được $f(n) = O(\log n)$.

Hình dạng cây có thể thay đổi. Với thao tác thêm cạnh và xóa cạnh, sau khi đổi gốc và access những đỉnh tương ứng, trực tiếp nối cả thứ tự DFS của cây vào hoặc tách nó ra.

Vẫn có $f(n) = O(\log n)$. Vì vậy bài toán con này có thể giải trong

$$O\left(\sqrt{n \log^3 n}\right)$$

thời gian cho mỗi thao tác.

Trường hợp cactus. Ta vẫn dùng link-cut cactus để duy trì hình dạng của cactus. Một thao tác có thể chuyển thành $O(\log n)$ thao tác của bài toán trên cây, nên thu được cận trên $f(n) = O(\log^2 n)$. Tuy nhiên khi viết chương trình sẽ thấy $f(n)$ gần với $O(\log n)$ hơn.

Sau đây là chứng minh $f(n) = O(\log n)$: vì link-cut tree duy trì một cây khung của cactus, và các đỉnh được access là như nhau, phân rã theo xích trên LCT tương tự phân rã theo xích do link-cut cactus duy trì. Khi đó trong một lần access của link-cut cactus, ngoài lần access đầu tiên của LCT, mỗi lần access của LCT chỉ thay đổi phân rã theo xích của một chu trình và đi qua $O(1)$ cạnh ảo. Điều này cho thấy tổng số vòng lặp access của LCT là $O(\log n)$, nên $f(n) = O(\log n)$.

Vì vậy bài này được giải trong

$$O\left(\sqrt{n \log^3 n}\right)$$

thời gian cho mỗi thao tác.

10.5 Lời cảm ơn

- Xin cảm ơn CCF đã cung cấp cơ hội học tập và giao lưu.
- Xin cảm ơn huấn luyện viên Xu Xianyou vì đã bồi dưỡng tôi.
- Xin cảm ơn các bạn học đã giúp tôi hoàn thành bài viết này.

Tài liệu tham khảo của bài viết

1. Huang Zhi'ao, “Bàn về các bài toán liên quan tới cây động và một số mở rộng đơn giản”, luận văn đội tuyển quốc gia năm 2014.
2. Tarjan, Robert E., and Renato F. Werneck. “Self-adjusting top trees.”

11 Bàn về thuật toán matching trên đồ thị và ứng dụng

Chen Haobo

Trường Trường Quận, Trường Sa, Hồ Nam

Tóm tắt

Thuật toán matching trên đồ thị là một lớp thuật toán thường dùng trong thi tin học và có phạm vi ứng dụng rất rộng. Bài viết này tổng kết các thuật toán matching trên đồ thị, chia đại thể thành hai phần: matching cực đại và matching trọng số lớn nhất. Trong mỗi phần, trước hết trình bày cách làm trên đồ thị hai phía, sau đó mở rộng sang đồ thị tổng quát, đồng thời nêu một số mở rộng và ứng dụng.

11.1 Các định nghĩa liên quan tới matching

Trong lý thuyết đồ thị, xét đồ thị $G = (V, E)$, trong đó V là tập đỉnh và E là tập cạnh.

Một tập cạnh $M \subseteq E$ sao cho không có hai cạnh nào chung đỉnh được gọi là một *matching* của đồ thị. Kích thước của matching là số cạnh trong matching. Matching có số cạnh nhiều nhất là *matching cực đại*. Khi cạnh có trọng số, matching có tổng trọng số cạnh lớn nhất là *matching trọng số lớn nhất*.

Cạnh thuộc matching gọi là cạnh matching; cạnh không thuộc matching gọi là cạnh không matching. Một đỉnh được xem là nằm trong matching khi và chỉ khi nó là đầu mút của một cạnh matching. Đỉnh

như vậy gọi là đỉnh đã ghép; đỉnh còn lại gọi là đỉnh chưa phủ. Đặc biệt, nếu mọi đỉnh đều đã ghép thì matching được gọi là matching hoàn hảo. Với một cạnh matching (u, v) , ta nói u và v là bạn ghép của nhau.

Nếu không nói rõ thêm, sau đây dùng n để chỉ số đỉnh của đồ thị và m để chỉ số cạnh.

11.2 Đường tăng

Giả sử P là một đường đi đơn trong đồ thị. Nếu:

1. hai đầu của P đều là đỉnh chưa phủ;
2. các cạnh trên P luân phiên là cạnh matching và cạnh không matching,

thì P được gọi là một *đường tăng*. Đường đi đơn chỉ thỏa mãn điều kiện thứ hai được gọi là *đường luân phiên*.

Trên một đường tăng, số cạnh không matching nhiều hơn số cạnh matching đúng một cạnh. Nếu đảo trạng thái các cạnh trên đường này, tức biến cạnh matching thành cạnh không matching và ngược lại, kích thước matching hiện tại tăng thêm 1 mà vẫn hợp lệ. Thao tác này gọi là *tăng* theo đường tăng.

Khi tăng theo một đường tăng, mọi đỉnh trên đường đó đều trở thành đỉnh đã ghép; vì vậy quá trình tăng liên tục mở rộng tập đỉnh đã ghép.

Định lý 1. Một matching M là matching cực đại của đồ thị G khi và chỉ khi trong G không tồn tại đường tăng.

Chứng minh. Vì đảo một đường tăng làm kích thước matching tăng thêm 1, chiều cần thiết là hiển nhiên.

Xét một matching M' lớn hơn M , và xét hiệu đối xứng

$$D = M \oplus M' = (M \cup M') \setminus (M \cap M').$$

Theo định nghĩa matching, trong D , bậc của mỗi đỉnh không quá 2; nếu một đỉnh có bậc 2, hai cạnh kề của nó lần lượt đến từ M và M' . Vì vậy D gồm một số chu trình và đường đi, trong đó các cạnh của M và M' luân phiên xuất hiện. Mỗi chu trình luân phiên có độ dài chẵn và chứa số cạnh của M bằng số cạnh của M' .

Bỏ mọi chu trình luân phiên đi, vì $|M'| > |M|$, trong D vẫn phải có một đường luân phiên mà số cạnh thuộc M' nhiều hơn số cạnh thuộc M . Với matching M , đường đó chính là một đường tăng. Do đó sẽ tồn tại matching lớn hơn thì tồn tại đường tăng, suy ra chiều đủ.

Từ định lý trên, ta có một khung thuật toán tìm matching cực đại: lặp lại việc duyệt các đỉnh chưa phủ, tìm một đường tăng rồi tăng theo nó, cho tới khi không còn tìm được đường tăng.

Thực ra, nếu tại một thời điểm không tìm được đường tăng xuất phát từ một đỉnh chưa phủ x , thì sau nhiều lần tăng khác, vẫn không tồn tại đường tăng xuất phát từ x . Giả sử sau một lần tăng theo đường $a - b$, xuất hiện đường tăng P_x xuất phát từ x . Khi đó P_x phải giao với đường $a - b$. Xét lần đầu P_x gặp $a - b$; vì $a - b$ là đường luân phiên nên kiểu cạnh ở hai phía điểm giao khác nhau, nghĩa là trước khi tăng, từ x đã có thể đi tới một trong các đỉnh a, b , tức một đỉnh chưa phủ. Điều này mâu thuẫn.

Hình a. Bản gốc minh họa một đường tăng mới xuất hiện sau khi đảo đường $a - b$. Đường mới từ x buộc phải cắt $a - b$, nên trước khi đảo cũng đã có đường tăng xuất phát từ x .

Vì vậy có thể tối ưu khung thuật toán: với mỗi đỉnh chưa phủ, chỉ cần tìm một lần đường tăng xuất phát từ nó.

11.3 Matching cực đại trên đồ thị hai phía

11.3.1 Thuật toán đường tăng cơ bản

Để mô tả thuận tiện, chia đỉnh của đồ thị hai phía thành hai phần trái và phải; mọi cạnh đều nối từ trái sang phải.

Vì đường tăng có độ dài lẻ, hai đầu của nó nằm ở hai phía khác nhau. Không mất tính tổng quát, ta tìm đường tăng từ từng đỉnh chưa phủ bên trái. Trên đường tăng, bước thứ lẻ đi qua cạnh không matching, bước thứ chẵn đi qua cạnh matching. Do đó nếu định hướng cạnh không matching từ trái sang phải và cạnh matching từ phải sang trái, bài toán trở thành: trong đồ thị có hướng, từ đỉnh đã cho tìm một đường đơn đi tới một đỉnh chưa phủ. Trong đồ thị có hướng, “tối được bằng đường đơn” tương đương với “tối được”, nên chỉ cần xét bài toán khả đạt.

Sau khi định hướng như trên, với một đỉnh chưa phủ u bên trái, chạy DFS từ u , đánh dấu mỗi đỉnh đã thăm để mỗi đỉnh được thăm nhiều nhất một lần; khi gặp một đỉnh chưa phủ thì dừng. Một lần tìm đường tăng tốn $O(m)$. Khi không tìm được đường tăng, cây DFS sinh ra còn được gọi là *cây luân phiên*. Vì có $O(n)$ đỉnh chưa phủ cần thử, tổng độ phức tạp là $O(nm)$.

11.3.2 Dùng Dinic tìm matching cực đại

Matching cực đại trên đồ thị hai phía có thể chuyển thành mô hình luồng mạng. Thêm nguồn và đích; nối nguồn tới mỗi đỉnh bên trái bằng cung dung lượng 1, nối mỗi đỉnh bên phải tới đích bằng cung dung lượng 1, và thay mỗi cạnh ban đầu bằng một cung từ trái sang phải dung lượng 1. Luồng cực đại chính là matching cực đại; các cung giữa hai phía có luồng đầy tương ứng với cạnh matching.

Dùng thuật toán Dinic có thể tìm matching cực đại trong $O(m\sqrt{n})$ thời gian. Mỗi vòng Dinic gồm BFS $O(m)$ xây đồ thị mức và DFS đa tăng $O(m)$ vì mọi dung lượng đều bằng 1. Trong $\sqrt{n}$ vòng đầu, tổng thời gian là $O(m\sqrt{n})$.

Sau $\sqrt{n}$ vòng, độ dài đường tăng ngắn nhất từ nguồn tới đích đã tăng nghiêm ngặt qua các vòng. Xét hiệu đối xứng giữa matching cực đại và matching hiện tại; nó gồm các chu trình luân phiên và đường luân phiên rời nhau. Mỗi đường luân phiên tương ứng với một đường tăng và có độ dài ít nhất $\sqrt{n}$, nên số đường như vậy không quá $O(\sqrt{n})$. Vì thế phần còn lại của Dinic cũng chỉ cần $O(\sqrt{n})$ vòng, tổng độ phức tạp vẫn là $O(m\sqrt{n})$.

11.3.3 Một số mở rộng

Trong đồ thị bất kỳ, xét hai matching cực đại khác nhau M, M' và hiệu đối xứng $D = M \oplus M'$. D gồm các chu trình luân phiên và đường luân phiên. Vì M, M' đều cực đại, các chu trình và đường này đều có độ dài chẵn; nếu có đường luân phiên lẻ thì đó là một đường tăng. Như vậy, từ một matching cực đại có thể nhận được một matching cực đại khác bằng cách xor với một số đường chẵn và chu trình chẵn rời nhau. Đặc biệt, hiệu đối xứng của hai matching hoàn hảo chỉ gồm các chu trình luân phiên rời nhau.

Đỉnh bắt buộc trong matching cực đại hai phía. Một đỉnh bắt buộc là đỉnh chắc chắn nằm trong mọi matching cực đại. Do hai phía đối xứng, chỉ xét các đỉnh bên trái.

Tìm một matching cực đại M . Các đỉnh bắt buộc cần tìm chắc chắn là đỉnh đã ghép trong M . Một đỉnh đã ghép p có thể trở thành chưa ghép trong một matching cực đại khác khi và chỉ khi từ p có một

đường luân phiên bắt đầu bằng cạnh matching và kết thúc ở một đỉnh chưa phủ cùng phía.

Xét ngược lại: đỉnh hướng cạnh matching từ phải sang trái, cạnh không matching từ trái sang phải. Từ mọi đỉnh chưa phủ bên trái chạy DFS, đánh dấu các đỉnh tới được. Cuối cùng, mỗi đỉnh đã ghép bên trái mà không được đánh dấu là đỉnh bắt buộc. Độ phức tạp là $O(n + m)$.

Cạnh bắt buộc trong matching cực đại hai phía. Cần tìm các cạnh chắc chắn nằm trong mọi matching cực đại. Tìm một matching cực đại M ; cạnh bắt buộc chỉ có thể là cạnh thuộc M . Với một cạnh $e \in M$, nếu có một matching cực đại khác không chứa e , thì e nằm trong hiệu đối xứng của hai matching, tức nằm trên một đường luân phiên chắn hoặc một chu trình luân phiên chắn.

Nếu e nằm trên đường luân phiên chắn, một đầu của đường phải là đỉnh chưa phủ, nên một đầu của e là đỉnh không bắt buộc. Vì thế chỉ cần kiểm tra hai đầu của e có đỉnh không bắt buộc hay không. Nếu e nằm trên chu trình luân phiên chắn, định hướng cạnh matching từ trái sang phải, cạnh không matching từ phải sang trái, rồi dùng Tarjan tìm thành phần liên thông mạnh để biết e có nằm trong một chu trình hay không. Tổng thời gian là $O(n + m)$.

Tập độc lập lớn nhất trong đồ thị hai phía. Tập độc lập lớn nhất là tập chọn được nhiều đỉnh nhất sao cho không có cạnh nào nối giữa hai đỉnh được chọn. Gọi M là matching cực đại. Mỗi cạnh trong M buộc có ít nhất một đầu không thuộc tập độc lập, nên kích thước tập độc lập không vượt quá $n - |M|$.

Sau đây là cách dựng tập độc lập đạt đúng cận này. Đưa mọi đỉnh chưa phủ vào tập độc lập. Với mỗi cạnh matching $e = (u, v)$, để đạt cận trên, đúng một trong hai đầu u, v phải được chọn; xem quyết định “chọn trái hay chọn phải” là một biến x_e . Các cạnh không matching giữa đầu mút của những cạnh matching khác nhau sinh ra quan hệ kéo theo giữa các biến này. Ngoài ra, khi một đỉnh chưa phủ đã được chọn, mọi cạnh nối từ nó tới đỉnh đã ghép sẽ ép giá trị biến của cạnh matching chứa đỉnh đó.

Nếu đồ thị kéo theo sinh mâu thuẫn thì từ hai cách ép trái ngược sẽ khôi phục được một đường tăng, mâu thuẫn với việc M đã cực đại. Do đó có thể gán giá trị: từ các biến bị đỉnh chưa phủ ép, DFS lan truyền giá trị; các biến còn lại chọn tùy ý. Số biến là $O(|M|) = O(n)$, số quan hệ được dựng từ cạnh của đồ thị gốc là $O(m)$, nên thời gian là $O(n + m)$.

Mặt khác, tập độc lập lớn nhất cũng có thể mô hình hóa bằng luồng và lát cắt nhỏ nhất. Điều này dẫn tới đẳng thức kinh điển trong đồ thị hai phía:

$$\text{tập độc lập lớn nhất} = n - \text{matching cực đại}.$$

Phủ đỉnh nhỏ nhất trong đồ thị hai phía. Phủ đỉnh là tập đỉnh sao cho mọi cạnh của đồ thị đều có ít nhất một đầu mút nằm trong tập này. Theo định nghĩa, phần bù của mọi phủ đỉnh là một tập độc lập, và phần bù của mọi tập độc lập là một phủ đỉnh. Vì vậy bỏ đi tập độc lập lớn nhất sẽ thu được phủ đỉnh nhỏ nhất. Suy ra trong đồ thị hai phía:

$$\text{phủ đỉnh nhỏ nhất} = n - \text{tập độc lập lớn nhất} = \text{matching cực đại}.$$

Matching hoàn hảo từ điển nhỏ nhất trong đồ thị hai phía. Gọi match_i là chỉ số đỉnh bên phải ghép với đỉnh thứ i bên trái. Cần cực tiểu hóa từ điển dãy $\text{match}_1, \dots, \text{match}_n$.

Trước hết tìm một matching hoàn hảo bất kỳ. Vì hiệu đối xứng của hai matching hoàn hảo chỉ gồm các chu trình luân phiên, ta duyệt các đỉnh trái theo thứ tự tăng dần, quyết định match_u bằng tham lam. Với một cạnh hiện có (u, v) , kiểm tra liệu có thể đổi matching sao cho u ghép với v hay không. Điều này tương đương với việc cạnh (u, v) nằm trên một chu trình luân phiên, tức hai đỉnh tương ứng thuộc cùng một thành phần liên thông mạnh trong đồ thị định hướng cạnh matching và không matching luân phiên. Chọn v nhỏ nhất thỏa mãn rồi xóa u khỏi đồ thị để khóa quyết định này.

Matching chỉ thay đổi $O(n)$ lần nên chỉ cần tính thành phần liên thông mạnh $O(n)$ lần; mỗi lần tìm chu trình có thể dùng DFS. Độ phức tạp là $O(nm)$.

11.3.4 Ứng dụng

Trong đồ thị hai phía, ta đã có

$$\text{matching cực đại} = \text{phủ đỉnh nhỏ nhất} = n - \text{tập độc lập lớn nhất}.$$

Nhờ đó các mô hình hai phía như bàn cờ theo hàng-cột, tô màu đen trắng thủ công, hoặc các đồ thị chứng minh được không có chu trình lẻ, đều thường giải được bằng matching.

Problem 1. Chessboard. Cho một bàn cờ $n \times n$ và n quân xe; cần đặt chúng sao cho mỗi hàng và mỗi cột có nhiều nhất một quân. Với mỗi quân, tọa độ được giới hạn trong một hình chữ nhật. Hỏi vị trí của những quân nào là xác định duy nhất.

Tọa độ x, y độc lập, chỉ cần xét một chiều. Bài toán trở thành: với một hoán vị P , vị trí xuất hiện của số x bị giới hạn trong đoạn $[l_x, r_x]$; cần tìm số nào có vị trí cố định. Dựng đồ thị hai phía, bên trái là các số, bên phải là các vị trí, nối số x tới mọi vị trí trong đoạn cho phép. Ta cần tìm các cạnh bắt buộc trong matching hoàn hảo.

Vì để đảm bảo có nghiệm nên có matching hoàn hảo, bước tìm đỉnh bắt buộc có thể bỏ qua. Định hướng cạnh matching về bên trái, cạnh không matching về bên phải, rồi dùng Tarjan tìm thành phần liên thông mạnh. Nếu thành phần liên thông mạnh chứa một đỉnh bên trái có kích thước lớn hơn 1, cạnh ra của đỉnh đó nằm trên chu trình, nên vị trí của số tương ứng không xác định.

Do cạnh nối theo đoạn và chỉ cần xét liên thông, dùng cây đoạn trên phía vị trí để tối ưu dựng cạnh, tổng số cạnh $O(n \log n)$. Matching hoàn hảo ban đầu có thể tìm bằng tham lam kinh điển: sắp các số theo l , quét vị trí từ trái sang phải và luôn chọn số có r nhỏ nhất. Tổng thời gian $O(n \log n)$.

Problem 2. Minimum Diameter. Cho $n \leq 1000$ điểm trên mặt phẳng, cần xóa $k < n$, $k < 30$ điểm sao cho khoảng cách lớn nhất giữa hai điểm còn lại là nhỏ nhất.

Liệt kê cặp điểm a, b sẽ trở thành cặp xa nhất còn lại. Lấy $|ab|$ làm bán kính, vẽ hai đường tròn tâm a, b . Mọi điểm còn lại phải nằm trong giao của hai hình tròn này. Trong các điểm nằm bên trong, nối cạnh giữa mọi cặp có khoảng cách lớn hơn $|ab|$, nghĩa là hai điểm đó không thể cùng giữ lại. Cần giữ lại nhiều điểm nhất, tức tìm tập độc lập lớn nhất; kiểm tra số điểm phải xóa có vượt k hay không.

Mỗi cạnh cấm như vậy đều băng qua đường trung trực của ab , vì hai điểm nằm cùng một phía không thể cách nhau hơn $|ab|$. Vì thế đây là bài toán tập độc lập lớn nhất trên đồ thị hai phía. Khi matching cực đại vượt k , tập độc lập quá nhỏ và vòng kiểm tra hiện tại có thể dừng. Một lần tìm đường tăng tốn $O(n^2)$, tối đa thử k đường tăng, cộng với liệt kê cặp a, b , thu được thuật toán đa thức $O(kn^4)$. Thuật toán đầy đủ của bài gốc còn cần tối ưu khác; ở đây ý tưởng matching là phần đáng chú ý.

Problem 3. The Square Div One. Cho ma trận $n \times m$, $n, m < 50$. Cần điền mỗi ô một số từ 1 tới $\max(n, m)$ sao cho trong mỗi hàng và mỗi cột không có số lặp. Hãy tìm phương án có thứ tự từ điển nhỏ nhất, so sánh theo thứ tự từng hàng.

Một ý tưởng trực tiếp là điền từng hàng từ trên xuống. Ở hàng hiện tại, với mỗi cột đã biết các số đã xuất hiện, nối vị trí với các số còn có thể điền rồi tìm matching từ điển nhỏ nhất. Tuy nhiên cách đó có thể khiến các hàng sau vô nghiệm: mỗi số phải xuất hiện $\max(n, m)$ lần; nếu một số xuất hiện quá ít ở các hàng đầu, các hàng còn lại không đủ để bù.

Khi đang điền một hàng, đặt $m' = \max(n, m)$. Gọi r_x là số lần số x còn cần xuất hiện, và h là số hàng chưa xác định. Điều kiện cần và đủ để còn có nghiệm là

$$\sum_x r_x = hm, \quad r_x \leq h \quad \text{với mọi } x.$$

Tính cần thiết hiển nhiên. Với tính đủ, nếu $h = 1$ thì rõ; nếu $h > 1$, ép điền tất cả các số có $r_x = h$ trong hàng hiện tại. Do tổng $\sum r_x = hm$ và mỗi $r_x \leq h$, số vị trí là đủ; sau khi điền xong, điều kiện vẫn đúng cho hàng kế tiếp, nên quy nạp được.

Vì thế mỗi hàng chỉ cần bảo đảm mọi số có $r_x = h$ được điền một lần. Dựng đồ thị hai phía: bên trái là vị trí của hàng, bên phải là các số. Nối vị trí với số nếu số đó có thể điền ở vị trí ấy; nếu $r_x = h$, ép số x phải được chọn, còn nếu $r_x < h$ thì có thể bỏ qua. Dùng cách đã nêu để tìm matching hoàn hảo từ điển nhỏ nhất.

11.4 Matching cực đại trên đồ thị tổng quát

11.4.1 Thuật toán blossom

Khác biệt duy nhất giữa đồ thị tổng quát và đồ thị hai phía là đồ thị tổng quát có thể có chu trình lẻ. Điều này khiến ta không thể chuyển bài toán thành khả đạt trong đồ thị có hướng.

Hình b. Bản gốc minh họa một chu trình lẻ có các mũi tên luân phiên: dù từ s tới được t , đường đi không đơn giản làm một số đỉnh bị dùng bởi hai cạnh matching sau khi đảo, nên không thể chỉ xét khả đạt.

Vấn đề nằm ở chu trình lẻ. Nếu chu trình trong hình là chẵn, việc chỉ xét khả đạt không quay lại cùng một đỉnh với cùng trạng thái, vì đi một số chẵn cạnh trở về sẽ không đổi loại cạnh cần đi tiếp.

Ta vẫn bắt đầu như trên đồ thị hai phía: mỗi lần chọn một đỉnh chưa ghép s , từ s tìm đường tăng bằng BFS/DFS theo quy tắc luân phiên. Trên cây luân phiên sinh ra, đánh dấu chẵn-lẻ cho các đỉnh, gốc là đỉnh chẵn. Đỉnh chẵn được đưa vào cây bằng cạnh matching từ một đỉnh lẻ, và mỗi lần ta duyệt các cạnh ra từ đỉnh chẵn.

Giả sử đang ở đỉnh u , xét cạnh tới v .

- Nếu v chưa thăm: nếu v là đỉnh chưa phủ thì tìm được đường tăng; nếu không, đưa bạn ghép của v vào cây.
- Nếu v đã thăm: cạnh đang xét là cạnh không thuộc cây của cây luân phiên, tạo thành một chu trình. Nếu đó là chu trình chẵn thì bỏ qua; trường hợp chỉ có chu trình chẵn chính là đồ thị hai phía. Nếu đó là chu trình lẻ, co toàn bộ chu trình thành một *blossom*, tức một đỉnh mới, rồi tiếp tục tìm đường tăng trên đồ thị mới.

Ta phân tích vì sao việc co blossom là đúng. Gọi đồ thị ban đầu là G , đồ thị sau khi co là G' . Cần chứng minh hai mệnh đề:

1. Nếu G có đường tăng, thì G' cũng có đường tăng.
2. Nếu G' có đường tăng, thì G cũng có đường tăng.

Trong cây luân phiên, xét một cạnh không thuộc cây (u, v) sinh ra chu trình lẻ, và đặt $h = \text{LCA}(u, v)$. Chu trình lẻ này là chu trình luân phiên, chỉ riêng tại h có hai cạnh kề cùng kiểu, đều là cạnh không matching. Cạnh cây đi vào h là cạnh matching; ngoài h , mọi cạnh từ đỉnh trên chu trình đi ra ngoài đều là cạnh không matching. Vì vậy nếu đi vào h bằng cạnh matching, ta có thể đi tới bất kỳ đỉnh nào khác trên chu trình rồi đi ra bằng cạnh không matching, theo một trong hai chiều; chiều ngược lại cũng đúng.

Hình c. Bản gốc vẽ cây luân phiên, cạnh ngoài cây (u, v) , và đỉnh $h = \text{LCA}(u, v)$. Phần chu trình lẻ được co thành một đỉnh mới; chu trình trước khi co gọi là blossom cũ.

Với mệnh đề thứ nhất, gọi đoạn đường màu đỏ từ s tới h trong hình gốc là S . Gọi G_S, G'_S là các đồ thị thu được sau khi đảo trạng thái cạnh trên S trong G, G' . Matching trong hai đồ thị này vẫn hợp lệ và cùng kích thước với matching ban đầu. Nếu G có đường tăng, matching trong G_S không cực đại, nên theo định lý đường tăng, G_S có đường tăng. Nếu đường tăng đó không đi qua blossom cũ, nó cũng tồn tại trong G'_S . Nếu có đi qua, ta cho nó vừa vào blossom thì đi thẳng trong blossom tới h ; theo tính chất vừa phân tích, đó vẫn là đường tăng hợp lệ. Khi ánh xạ sang G'_S , đoạn qua blossom tương ứng với việc đi qua đỉnh mới. Vì G'_S có đường tăng và kích thước matching của G' không đổi, G' cũng có đường tăng.

Với mệnh đề thứ hai, nếu đường tăng trong G' không đi qua đỉnh mới thì nó cũng là đường tăng trong G . Nếu nó đi qua đỉnh mới, nó chỉ có một cạnh vào và một cạnh ra, một cạnh matching và một cạnh không matching. Ánh xạ hai cạnh đó về G ; cạnh matching nhất định là cạnh nối với h . Như đã phân tích, từ h có thể đi trong blossom tới mọi vị trí cần thiết rồi đi ra bằng cạnh không matching, hoặc ngược lại. Nhờ đó khôi phục được một đường tăng trong G .

Hơn nữa, nếu đang tìm đường tăng từ s và gặp chu trình lẻ, thì hãy trước khi co có đường tăng xuất phát từ s , sau khi co vẫn có đường tăng xuất phát từ s . Vì vậy điểm bắt đầu không đổi.

Khi cài đặt, thường không co đỉnh một cách tường minh. Dùng DSU để ghi mỗi đỉnh hiện thuộc blossom có gốc nào. Ngoài mảng ghép đôi, dùng mảng next ghi đỉnh kế tiếp trên đường tăng cuối cùng; trong mỗi blossom, các next tạo thành một danh sách hai chiều mô tả cách đi bên trong blossom. Chương trình chính dùng BFS mở rộng cây luân phiên, hàng đợi chứa mọi đỉnh chẵn. Khi gặp chu trình lẻ, dùng DSU co blossom và đánh dấu các đỉnh lẻ trên chu trình thành đỉnh chẵn để đưa vào hàng đợi. Một lần tìm đường tăng tốn $O(m)$; cần $O(n)$ lần, nên tổng thời gian là $O(nm)$.

11.4.2 Ứng dụng

Problem 1. Trò chơi trên đồ thị vô hướng. Alice và Bob chơi trên một đồ thị vô hướng đơn có $n \leq 200$ đỉnh, Alice đi trước. Ban đầu có một quân ở đỉnh s . Mỗi lượt, nếu quân đang ở đỉnh u , người chơi phải chuyển quân tới một đỉnh kề với u . Các đỉnh đã đi qua bị xóa vĩnh viễn khỏi đồ thị. Ai không đi được thì thua. Biết hai người đều chơi tối ưu, hỏi có bao nhiêu đỉnh s là trạng thái thắng cho người đi trước.

Kết luận: s là trạng thái thắng khi và chỉ khi s nằm trong mọi matching cực đại.

Nếu tồn tại matching cực đại mà s chưa ghép, thì sau khi người đi trước đi từ s tới v , người đi sau chỉ cần đi tiếp tới bạn ghép của v . Người đi sau luôn có nước đi, trừ khi tới một đỉnh chưa phủ; nhưng nếu điều đó xảy ra thì ta đã tìm được một đường luân phiên nối hai đỉnh chưa phủ, tức một đường tăng, mâu thuẫn với matching cực đại.

Ngược lại, nếu s nằm trong mọi matching cực đại, lấy một matching cực đại bất kỳ và để người đi trước dùng đúng chiến lược vừa mô tả cho người đi sau. Nếu người đi trước hết nước, đã sinh ra một đường luân phiên chỉ có một đầu chưa phủ; đảo đường này sẽ làm s không còn được ghép mà kích thước matching không đổi, mâu thuẫn với điều kiện s bắt buộc.

Vì vậy bài toán trở thành: trong đồ thị tổng quát, xác định các đỉnh nằm trong mọi matching cực đại. Tìm một matching cực đại bất kỳ. Với mỗi đỉnh đã ghép u , tạm xóa nó khỏi đồ thị và kiểm tra có đường tăng hay không. Nếu có, u không bắt buộc.

Chú ý sau khi xóa u , nếu tồn tại đường tăng thì bạn ghép v của u là một đầu của đường tăng. Nếu không, sau khi xóa u , matching giảm 1, tăng theo đường tăng lại tăng 1, rồi vẫn có thể ghép u với v để tạo matching lớn hơn matching cực đại ban đầu, mâu thuẫn. Do đó khi xét u , chỉ cần tìm đường tăng từ v . Tìm matching cực đại ban đầu tốn $O(nm)$; sau đó duyệt $O(n)$ đỉnh, mỗi lần blossom kiểm tra đường tăng

từ một điểm tốn $O(m)$, tổng $O(nm)$.

Problem 2. Lấy điểm trên đồ thị topo. Cho một DAG có $n \leq 200$ đỉnh. Mỗi lượt có thể lấy đồng thời nhiều nhất hai đỉnh có bậc vào 0; bậc vào chỉ được tính lại sau khi lượt kết thúc, và hai đỉnh được lấy cùng lượt không được có quan hệ topo với nhau. Hỏi ít nhất bao nhiêu lượt để lấy hết mọi đỉnh.

Với hai đỉnh u, v , nếu một đỉnh tới được đỉnh kia thì chúng không thể lấy cùng lượt. Dựng đồ thị vô hướng G' , nối cạnh giữa mọi cặp đỉnh không có quan hệ topo, biểu thị rằng cặp đó có thể lấy cùng một lượt. Vì số đỉnh cố định, cực tiểu số lượt tương đương cực đại số lượt lấy được hai đỉnh, gọi là R . Matching cực đại trong G' hiển nhiên là cận trên của R ; ta chứng minh cận này luôn đạt được.

Xét các đỉnh bậc vào 0 của DAG ban đầu và quan hệ ghép trong G' . Nếu một đỉnh không có bạn ghép, lấy nó ngay. Nếu bạn ghép của nó cũng có bậc vào 0, lấy cả hai. Trường hợp còn lại, mọi đỉnh bậc vào 0 đều có bạn ghép bị “đè” bởi quan hệ topo. Xét các bạn ghép ấy theo thứ tự topo; trong số chúng, phải có ít nhất hai đỉnh cực đại, tức không bị bạn ghép khác đè. Nếu chỉ có một đỉnh cực đại z , vì z không có bậc vào 0, phải có một đỉnh bậc vào 0 tới được z ; điều này kéo theo chuỗi đè ngược và mâu thuẫn với tính cực đại duy nhất.

Lấy hai đỉnh cực đại đó là z, t , và bạn ghép bậc vào 0 của chúng là x, y . Từ các cạnh matching $x - z$ và $y - t$, đổi thành $x - y$ và $z - t$: z, t có thể ghép với nhau do không có quan hệ topo, và x, y đều có bậc vào 0 nên có thể lấy cùng lượt. Như vậy luôn điều chỉnh được matching cực đại mà kích thước không đổi và có một cạnh ghép hai đỉnh bậc vào 0. Lặp lại sẽ đạt đúng R lượt lấy hai đỉnh. Chứng minh trên cũng cho cách dựng nghiệm. Độ phức tạp $O(nm)$ nếu dùng blossom trực tiếp cho G' .

11.5 Matching trọng số lớn nhất trên đồ thị hai phía

Matching trọng số lớn nhất là matching có tổng trọng số cạnh lớn nhất. Nếu thêm đỉnh giả vào phía ít đỉnh hơn để hai phía bằng nhau, và xem cạnh không tồn tại có trọng số 0, bài toán matching trọng số lớn nhất có thể chuyển thành bài toán matching hoàn hảo trọng số lớn nhất. Sau đây xét matching hoàn hảo trọng số lớn nhất.

11.5.1 Thuật toán KM

Thuật toán KM, còn gọi là Hungarian algorithm, tìm matching hoàn hảo trọng số lớn nhất trong $O(n^3)$ thời gian. Trước hết nêu hai khái niệm.

Một bộ *nhãn đỉnh khả thi* là cách gán mỗi đỉnh u một giá trị $l(u)$ sao cho với mọi cạnh (u, v) ,

$$w(u, v) \leq l(u) + l(v).$$

Đồ thị *đẳng thức* ứng với một bộ nhãn khả thi là đồ thị con sinh bởi toàn bộ các đỉnh, chỉ giữ những cạnh thỏa

$$w(u, v) = l(u) + l(v).$$

Định lý 2. Với một bộ nhãn khả thi, nếu đồ thị đẳng thức có matching hoàn hảo thì matching đó là matching hoàn hảo trọng số lớn nhất của đồ thị hai phía ban đầu.

Chứng minh. Với matching hoàn hảo bất kỳ M , tổng trọng số của nó thỏa

$$\text{val}(M) = \sum_{(u,v) \in M} w(u, v) \leq \sum_{(u,v) \in M} (l(u) + l(v)) = \sum_i l(i).$$

Nếu M' là matching hoàn hảo trong đồ thị đẳng thức, thì

$$\text{val}(M') = \sum_{(u,v) \in M'} (l(u) + l(v)) = \sum_i l(i).$$

Vậy không matching hoàn hảo nào có tổng trọng số lớn hơn M' .

Mục tiêu là liên tục điều chỉnh nhãn sao cho đồ thị đẳng thức có matching hoàn hảo. Gọi n là số đỉnh mỗi phía. Đặt $l_x(i)$ cho nhãn đỉnh trái thứ i , $l_y(j)$ cho nhãn đỉnh phải thứ j , và $w(i,j)$ cho trọng số cạnh giữa chúng. Khởi tạo nhãn khả thi, chẳng hạn

$$l_x(i) = \max_j w(i,j), \quad l_y(j) = 0.$$

Mỗi lần chọn một đỉnh trái chưa ghép rồi thử tìm đường tăng trong đồ thị đẳng thức, giống thuật toán matching cực đại. Nếu tìm được thì tăng ngay. Nếu không, ta có một cây luân phiên. Gọi S, T là tập đỉnh trái và phải trong cây; gọi S', T' là phần còn lại.

Trong đồ thị đẳng thức, không có cạnh từ S tới T' ; nếu có cạnh không matching thì cây còn mở rộng được, còn cạnh matching từ S ra ngoài là không thể vì mọi đỉnh trong S trừ gốc đều được ghép với một đỉnh trong T . Ngược lại, mọi cạnh từ S' tới T đều là cạnh không matching.

Nếu giảm nhãn của mọi đỉnh trong S đi a và tăng nhãn của mọi đỉnh trong T lên a , các cạnh matching và các cạnh của cây luân phiên vẫn nằm trong đồ thị đẳng thức; một số cạnh từ S sang T' có thể mới được thêm vào. Chọn

$$a = \min\{l_x(u) + l_y(v) - w(u,v) \mid u \in S, v \in T'\}.$$

Nếu a nhỏ hơn giá trị này, đồ thị đẳng thức không đổi.

Khi có cạnh mới (u,v) vào đồ thị đẳng thức, nếu v chưa ghép thì tìm được đường tăng; nếu v đã ghép thì đưa v và bạn ghép của nó vào cây luân phiên. Mỗi lần sửa nhãn hoặc tìm được đường tăng, hoặc cây luân phiên tăng thêm hai đỉnh, nên nhiều nhất $O(n)$ lần sửa nhãn cho một đường tăng.

Để cài đặt hiệu quả, với mỗi đỉnh phải $v \in T'$, duy trì

$$\text{slack}(v) = \min_{u \in S} (l_x(u) + l_y(v) - w(u,v)).$$

Khi thêm một đỉnh trái vào S , cập nhật mọi slack trong $O(n)$; khi sửa nhãn, trừ a khỏi các slack tương ứng. Như vậy mỗi lần mở rộng cây tốn $O(n)$, mỗi đường tăng tốn $O(n^2)$, và toàn bộ thuật toán tốn $O(n^3)$.

11.5.2 Ứng dụng

KM giải trực tiếp matching trọng số lớn nhất và matching hoàn hảo trọng số lớn nhất trên đồ thị hai phía. Nếu đổi dấu trọng số cạnh, nó cũng giải matching trọng số nhỏ nhất.

Mô hình nhãn đỉnh của KM còn cho bài toán đối ngẫu: tìm các dãy l_x, l_y sao cho

$$l_x(i) + l_y(j) \geq w(i,j)$$

với mọi cạnh (i,j) , và cực tiểu hóa

$$\sum_{i=1}^n l_x(i) + \sum_{j=1}^n l_y(j).$$

Các nhãn do KM tìm được chính là nghiệm tối ưu, vì tổng nhãn của mọi nghiệm khả thi đều không nhỏ hơn trọng số matching hoàn hảo lớn nhất, còn KM đạt đúng cận dưới này.

11.6 Matching trọng số lớn nhất trên đồ thị tổng quát

11.6.1 Thuật toán

Tương tự mô hình nhân trong KM cho đồ thị hai phía, với đồ thị tổng quát ta đặt một nhân đỉnh z_i cho mỗi đỉnh i , và một nhân không âm z_B cho mỗi tập đỉnh lẻ B . Với cạnh $e = (u, v)$, đặt

$$f_e = z_u + z_v + \sum_{B: u, v \in B} z_B,$$

trong đó w_e là trọng số của cạnh e .

Nếu với một matching M thỏa:

1. $f_e \geq w_e$ với mọi $e \in E$;
2. $f_e = w_e$ với mọi $e \in M$;
3. nếu $z_B > 0$ thì trong M có đúng $(|B| - 1)/2$ cạnh nằm hoàn toàn trong B ,

thì M là matching trọng số lớn nhất.

Lý do là với matching bất kỳ M' , tổng trọng số của nó không vượt quá giới hạn

$$\text{LIMIT} = \sum_i z_i + \sum_B \frac{|B| - 1}{2} z_B.$$

Trong đó z_i là đóng góp của nhân đỉnh, còn z_B là cận trên đóng góp của nhân tập đỉnh lẻ. Matching M đạt đúng giới hạn này, nên tối ưu. Định nghĩa của B có vẻ lạ, nhưng thực chất z_B là trọng số duy trì cho mỗi blossom đã co.

Mục tiêu là đồng thời tìm bộ nhân và matching M thỏa các điều kiện trên. Một bộ nhân thỏa điều kiện 1, 2 được gọi là hợp lệ; đồ thị gồm các cạnh $f_e = w_e$ gọi là đồ thị đẳng thức. Khởi tạo $M = \emptyset$, nhân đỉnh $z_i = \max_e w_e/2$, còn mọi $z_B = 0$; chỉ cần ghi và duy trì những blossom có $z_B > 0$. Ban đầu chỉ vi phạm điều kiện thứ ba.

Thuật toán gồm nhiều pha như khi tìm matching cực đại. Trong mỗi pha, tìm đường tăng trong đồ thị đẳng thức. Khác với đồ thị hai phía, ta bắt đầu BFS đồng thời từ mọi đỉnh chưa ghép, xem các gốc là đỉnh chắn; hàng đợi chứa các đỉnh chắn. Khi co blossom, mọi đỉnh bên trong blossom được đánh dấu chắn và các đỉnh chắn mới được đưa vào hàng đợi. Nếu tìm được đường tăng thì tăng. Nếu không, điều chỉnh nhân z .

Để giữ điều kiện thứ ba, sau một pha cần bảo vệ các blossom đã co có $z_B > 0$, tức không bung chúng ra. Vì vậy trong BFS cây luân phiên, ta xem blossom như một đỉnh; ngoài blossom chắn, cũng có thể gặp blossom lẻ.

Khi không tìm được đường tăng trong đồ thị đẳng thức, điều chỉnh nhân giống KM:

$$z_i \leftarrow z_i - a \quad \text{nếu } i \text{ là đỉnh chắn,}$$

$$z_i \leftarrow z_i + a \quad \text{nếu } i \text{ là đỉnh lẻ.}$$

Tuy nhiên mọi đỉnh trong cùng một blossom có cùng loại; nếu chỉ sửa nhân đỉnh, cạnh bên trong blossom có thể rời đồ thị đẳng thức hoặc nhân trở nên không hợp lệ. Vì thế cần nhân blossom:

$$z_B \leftarrow z_B + 2a \quad \text{nếu } B \text{ là blossom chắn,}$$

$$z_B \leftarrow z_B - 2a \quad \text{nếu } B \text{ là blossom lẻ.}$$

Ở đây chỉ xét các blossom ngoài cùng; blossom có thể lồng nhau. Cách sửa này giữ các cạnh trong blossom ở lại đồ thị đẳng thức.

Giá trị a là nhỏ nhất trong bốn loại sau:

1. z_i , với i là đỉnh chắn;
2. $f_e - w_e$, với $e = (u, v)$ nối một đỉnh chắn với một đỉnh chưa thăm;
3. $(f_e - w_e)/2$, với $e = (u, v)$ nối hai đỉnh chắn không thuộc cùng blossom;
4. $z_B/2$, với B là blossom lẻ.

Giá trị này không làm các cạnh trong blossom, cạnh của cây luân phiên hay cạnh matching rời khỏi đồ thị đẳng thức, đồng thời không làm bộ nhãn mất hợp lệ.

Nếu cực tiểu rơi vào loại 1, sau lần sửa này điều kiện thứ ba được cải thiện. Nếu rơi vào loại 4, một số blossom lẻ có trọng số về 0. Những blossom này chắc chắn hình thành từ các pha trước; trước đây giữ lại chúng để bảo toàn điều kiện thứ ba, còn bây giờ có thể bung ra. Sau khi bung, các blossom con của chúng trở thành blossom trong cây luân phiên hiện tại, và mọi đỉnh chắn mới được đưa vào hàng đợi.

Hình d. Bản gốc so sánh trước và sau khi bung một blossom lẻ có $z_B = 0$; các blossom con trở lại cây luân phiên hiện tại.

Nếu cực tiểu rơi vào loại 2 hoặc 3, ít nhất một cạnh mới được thêm vào đồ thị đẳng thức. Nếu cạnh mới nối hai đỉnh chắn thuộc hai cây khác nhau thì tìm được đường tăng. Như vậy cuối cùng luôn tìm được đường tăng. Kết thúc một pha, bung mọi blossom chắn trên cây luân phiên có trọng số 0, rồi sang pha kế tiếp.

Độ phức tạp cơ bản là $O(n^4)$. Bằng cách duy trì biến slack tương tự KM trên đồ thị hai phía, có thể đạt $O(n^3)$.

11.6.2 Ứng dụng: bài toán người đưa thư Trung Quốc trên đồ thị vô hướng

Cho một đồ thị vô hướng đơn có $n \leq 200$ đỉnh, các cạnh có trọng số. Cần tìm một đường đi khép kín ngắn nhất đi qua mọi cạnh ít nhất một lần.

Ta cần thêm một số cạnh với tổng trọng số nhỏ nhất sao cho đồ thị có chu trình Euler, tức mọi đỉnh đều có bậc chẵn. Trọng số cạnh thêm giữa hai đỉnh là khoảng cách ngắn nhất giữa chúng trong đồ thị gốc. Dùng Floyd $O(n^3)$ để tính mọi cặp khoảng cách ngắn nhất; bài toán còn lại là tìm matching hoàn hảo trọng số nhỏ nhất giữa các đỉnh bậc lẻ. Đổi trọng số bằng một hằng số lớn trừ đi khoảng cách sẽ chuyển nó thành matching trọng số lớn nhất trên đồ thị tổng quát.

11.7 Tổng kết

Bài viết đã thảo luận bốn kiểu bài toán matching cực đại: từ matching cực đại đến matching trọng số lớn nhất, từ đồ thị hai phía đến đồ thị tổng quát. Với mỗi loại, bài viết giới thiệu một số thuật toán đa thức.

Cốt lõi của matching là tìm đường tăng. Với matching trọng số lớn nhất, mô hình nhãn đỉnh phần nào chuyển bài toán về dạng tìm matching cực đại trong đồ thị đẳng thức.

Thực ra nội dung về matching trên đồ thị còn rộng hơn nhiều: có nhiều thuật toán hiệu quả hơn và nhiều hướng nghiên cứu lịch sử mà bài viết chưa đề cập. Bạn đọc quan tâm có thể tự tìm hiểu thêm; hy vọng bài viết này đóng vai trò gợi mở.

11.8 Lời cảm ơn

- Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

- Xin cảm ơn Lü Kaifeng, Huang Zhi'ao, Guo Xiaoxu, Luo Yuping, Yu Xingjiang, Hu Jiachen và Zhang Tianyang vì đã giúp tôi trong quá trình viết bài.
- Xin cảm ơn các huấn luyện viên trong đội tuyển và thầy Xu Guangming vì đã bồi dưỡng tôi.
- Xin cảm ơn các bạn học ở Trường Trường Quận vì đã giúp đỡ và ủng hộ tôi.
- Xin cảm ơn gia đình đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo của bài viết

1. Wikipedia, “Berge’s lemma”.
2. Wikipedia, “Hopcroft–Karp algorithm”.
3. Matthew Kusner, “Edmonds’s Blossom Algorithm”.
4. Zvi Galil, “Efficient Algorithms for Finding Maximal Matching in Graphs”.
5. Liu Rujia, *Sách nhập môn kinh điển về thi thuật toán*, Nhà xuất bản Đại học Thanh Hoa.
6. Chen Lijie, “Codeforces VK Cup 2012 Round 3 D. Minimum Diameter”.
7. Huang Zhi'ao, “Matching trọng số lớn nhất”.
8. Fan Haoqiang, “Thuật toán blossom cho matching trên đồ thị vô hướng”.